ORACLE
University

Developing Web Applications with JavaScript, HTML5, and CSS

Activity Guide

D1104149GC10

1006272023

Table of Contents

Practices for Lesson 1: Introduction to Web Applications

Overview

In these practices, you will:

- Create a Web Application project with WebStorm IDE
- Create HTML, CSS, and JavaScript files
- Link files together to form a web application

Practice 1-1: Create a Web Application Project with WebStorm IDE

Overview

In this practice, you will use the WebStorm IDE to create a Web Application project.

Tasks

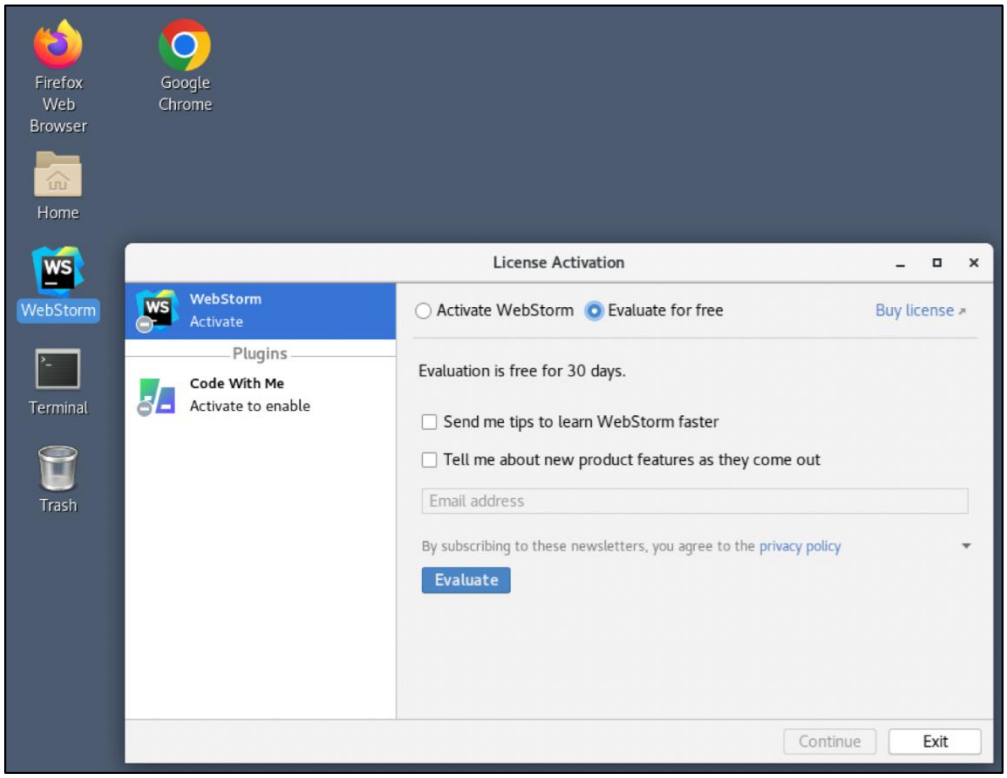
1. Create a Web Application project to develop DukeLabs web application.
   a. Launch WebStorm IDE.

      Hint: Click the WebStorm icon located on the desktop.

   b. Start your product trial.

      Hints:

      - Select the “Evaluate for free” option.
      - Click the “Evaluate” button.

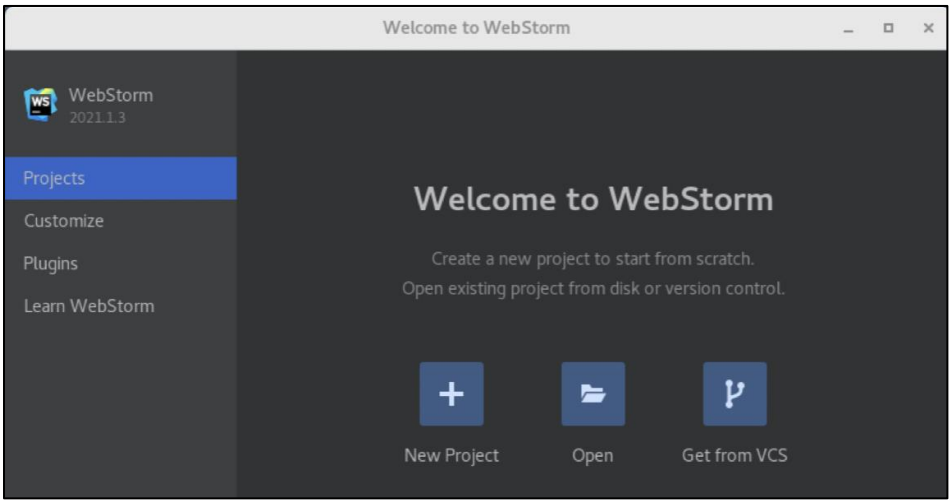


   c. After you have started the evaluation, click the “Continue” button.

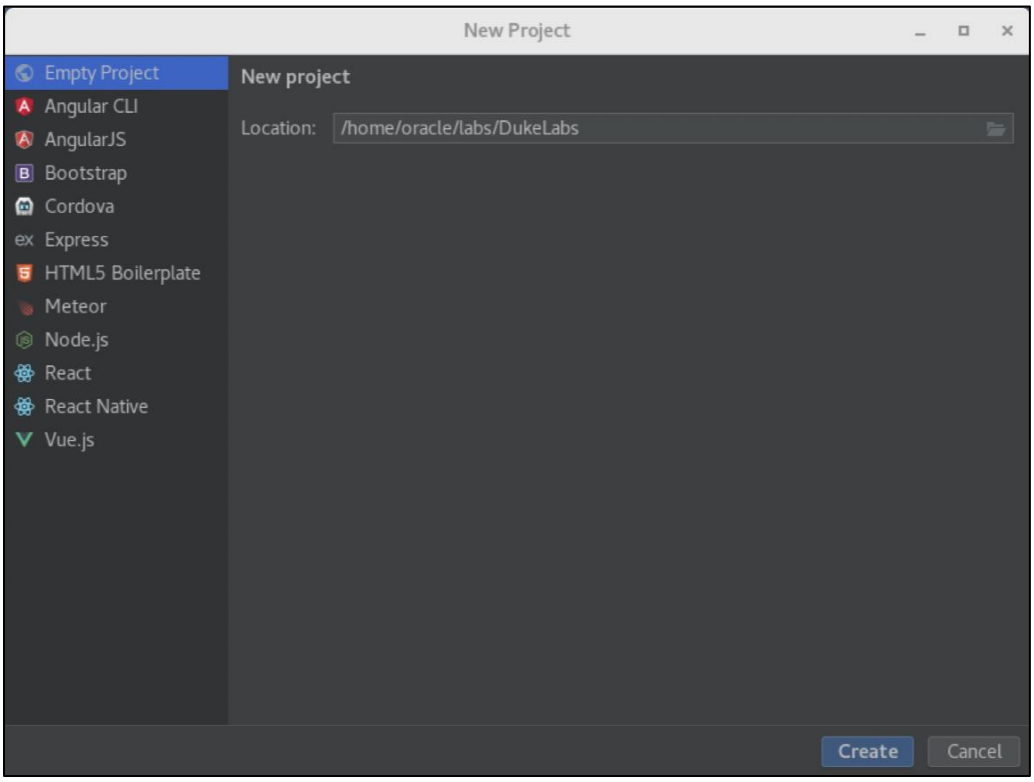
d. Create new project to contain artefacts related to DukeLabs web application.

Hints:

- Click the “New Project” icon.



- Choose an “Empty” project.
- Set project location as /home/oracle/labs/DukeLabs.

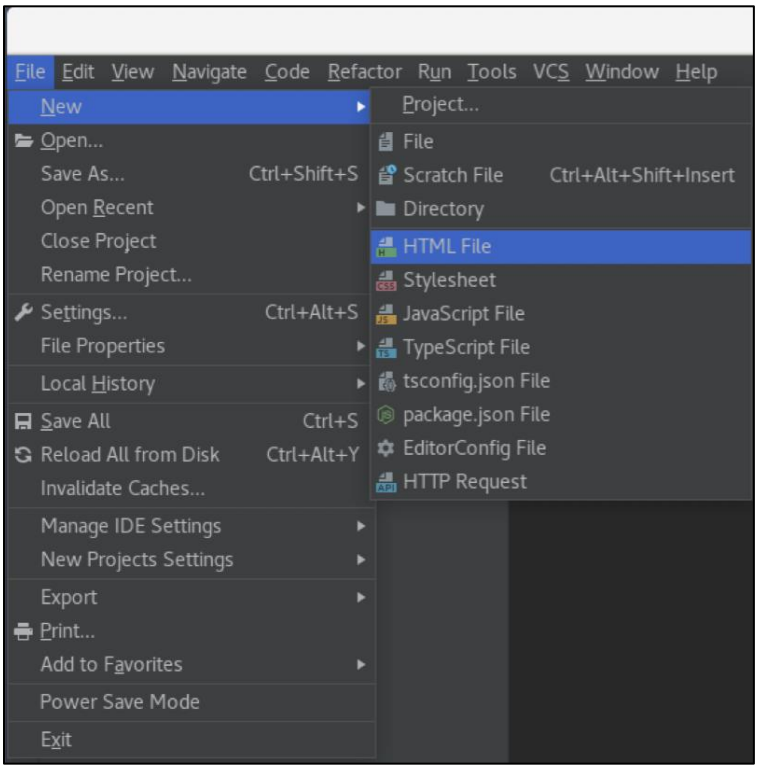


- Click the “Create” button.

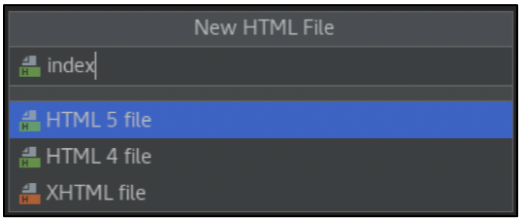
2. Create a HTML document to act as a front page of the DukeLabs web application.
   a. Create a new HTML document called `index.html` in the DukeLabs folder.

      Hints:

      – Use File → New → HTML File



      – Set file name as index and press the Enter key.



   b. Modify page title text to be “Welcome to DukeLabs”.

```
<title>Welcome to DukeLabs</title>
```

   c. Add header, main, and footer sections inside the html page body, and a nav element inside of the header.

```
<body>
  <header>
    <nav></nav>
  </header>
  <main></main>
  <footer></footer>
</body>
```

d. Add a reference to the `duke.png` image inside a page header.

Hints:

– Open a terminal.

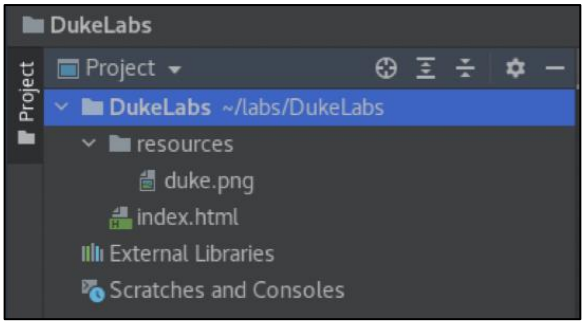
– Create resources folder in the DukeLabs folder.

```
mkdir /home/oracle/labs/DukeLabs/resources
```

– Copy `duke.jpg` file from the `/home/oracle/labs/files` folder into the `/home/oracle/labs/DukeLabs/recourses` folder.

```
cp labs/files/duke.png labs/DukeLabs/resources/
```

– Return to the WebStorm project window. The resources folder and the image file should appear in your project.



– Add this image reference to the header element, just before the nav element.

```
<header>
  <img src="resources/duke.png">
  <nav></nav>
</header>
```

e. Add a div element containing a slogan "We do science here!" text inside the header element, immediately after the img, but just before the nav.

```
<header>
  <img src="resources/duke.png">
  <div>We do science here!</div>
  <nav></nav>
</header>
```

f. Create a bullet list inside a nav element with two list items.

```
<nav>
  <ul>
    <li></li>
    <li></li>
  </ul>
</nav>
```

g. Add a contact email link to the first list item and a link referencing the `about.html` page to the second list item.

Hints:

- Switch to the terminal window.
- Copy the `about.html` file from the `/home/oracle/labs/files` folder into the `/home/oracle/labs/DukeLabs/` folder.

```
cp labs/files/about.html labs/DukeLabs/
```

- Return to the WebStorm project window. The `about.html` file should appear in your project.
- Use a (anchor) elements to produce two links.
- Use fictional email "mailto:duke@dukelabs.org" as a contact.

```
<nav>
  <ul>
    <li>
      <a href="mailto:duke@dukelabs.org">Contact</a>
    </li>
    <li>
      <a href="about.html">About</a>
    </li>
  </ul>
</nav>
```

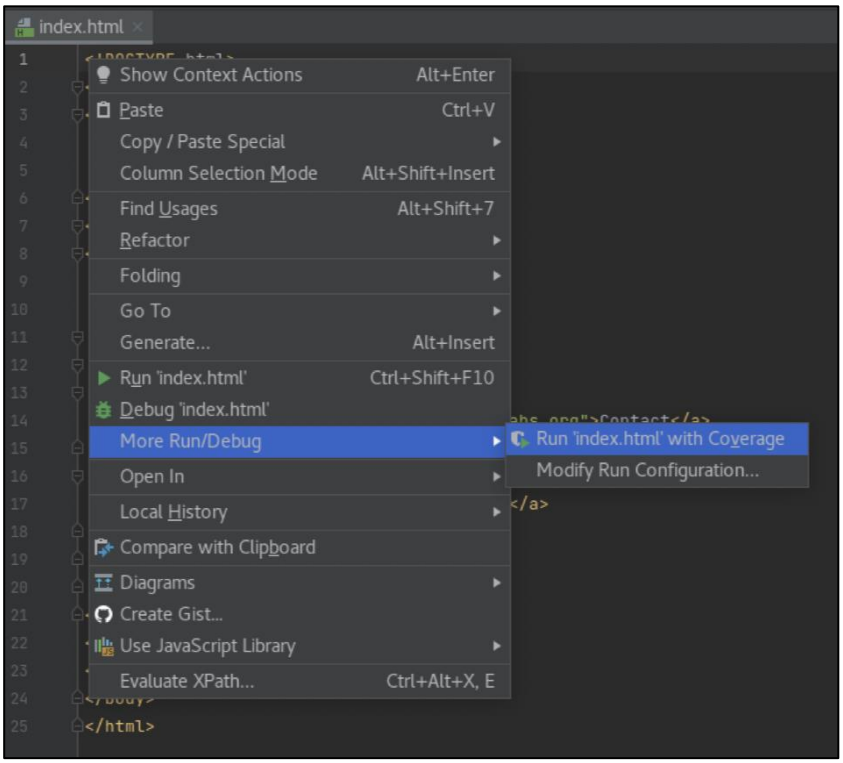
h. Add "Experiments and Measurements" text inside the main element.

```
<main>Experiments and Measurements</main>
```

i. Add the “Copyright ©2022” text inside the footer element.

```
<footer>Copyright ©2022</footer>
```

3. Test the DukeLabs web application.
   a. Right-click the `index.html` file.
   b. Select the “More Run/Debug” menu.
   c. Click the "Run index.html with Coverage" menu.

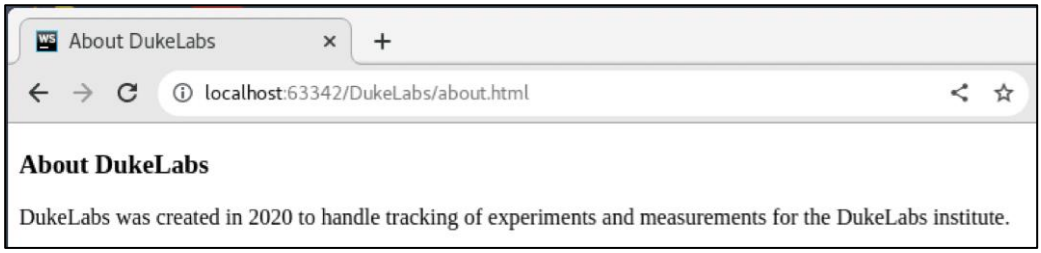


Note:

– Use password `oracle` when prompted to enter the keychain password.

d. Observe the page appearing in the browser.

e. Click the About link to navigate to the about page.

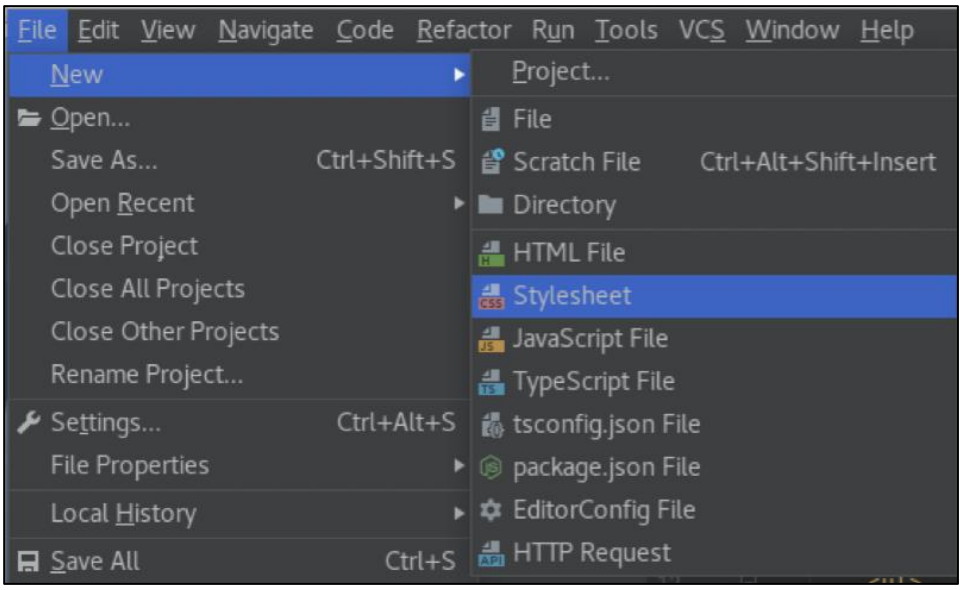


f. Navigate back to the index page using the browser's back button.

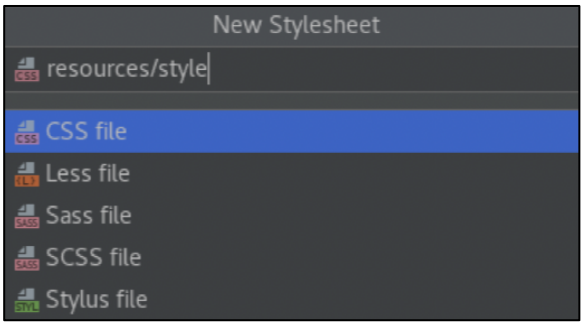
4. Create a CSS stylesheet document within the resources folder and apply it to the `index.html` page.

   a. Switch back to the WebStorm project window and ensure that the DukeLabs folder is selected.

   b. Use File → New → Stylesheet.



   c. Set file name as resources/style and press the Enter key.



   d. In this stylesheet file, add style for a header element, that displays its text centered and in red color.

```
header {
  color:red;
  text-align:center;
}
```

   e. Open an `index.html` file and link `style.css` to it.

   Hints:

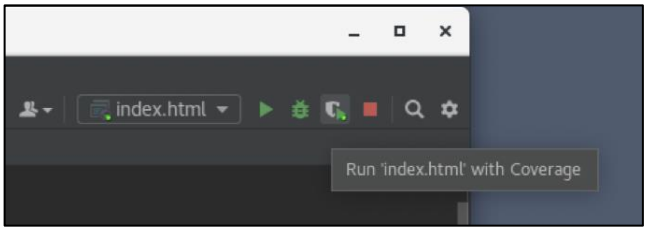
   – Place stylesheet reference inside the `index.html` head element.

```
<head>
  <meta charset="UTF-8">
  <title>Welcome to DukeLabs</title>
  <link rel="stylesheet" href="resources/style.css">
</head>
```

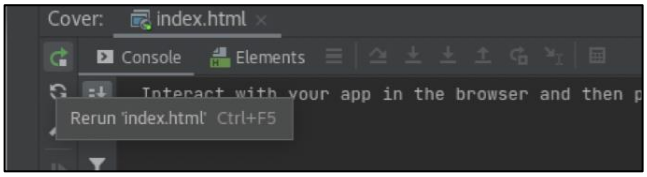
5. Test DukeLabs web application again to see how application of the style affects it.
   a. Switch to the browser window used in the earlier test of the application.
   b. Refresh the browser page to see changes.

Notes:

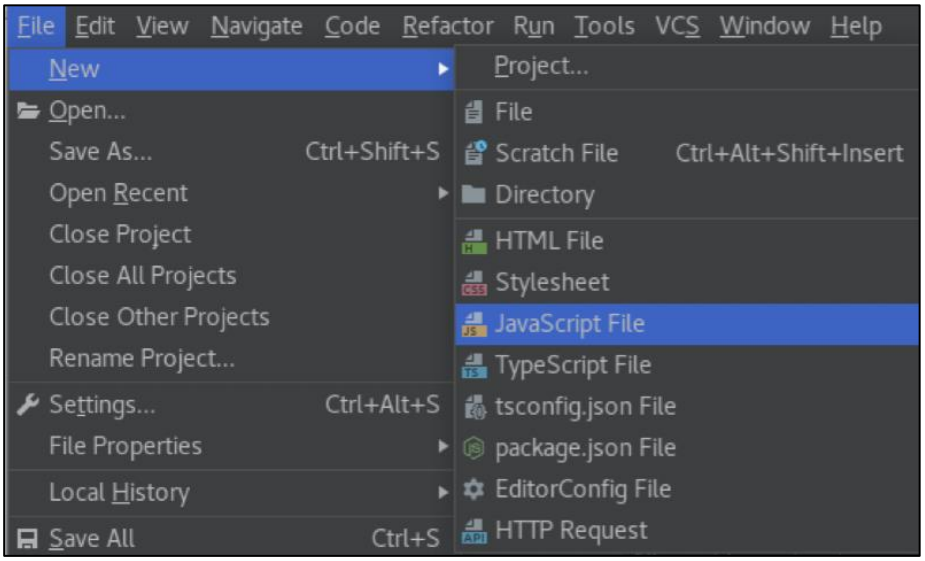
- Alternatively, you may choose to run application again from WebStorm using the "Run 'index.html' with coverage" button.



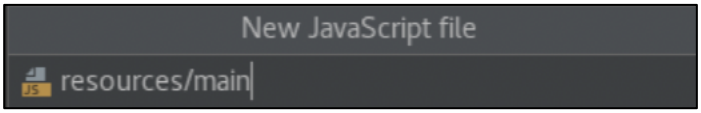
- Or click a "Rerun 'index.html'" in the console window displayed by WebStorm.



6. Create a JavaScript file and apply it to the `index.html` page.
   a. Switch back to the WebStorm project window and ensure that the DukeLabs folder is selected.
   b. Use File → New → JavaScript File.



   c. Set file name as resources/main and press the Enter key.



   d. In this JavaScript file, add a snippet of code that prints a hello message to the console.

```
console.log("Hello from DukeLabs!");
```

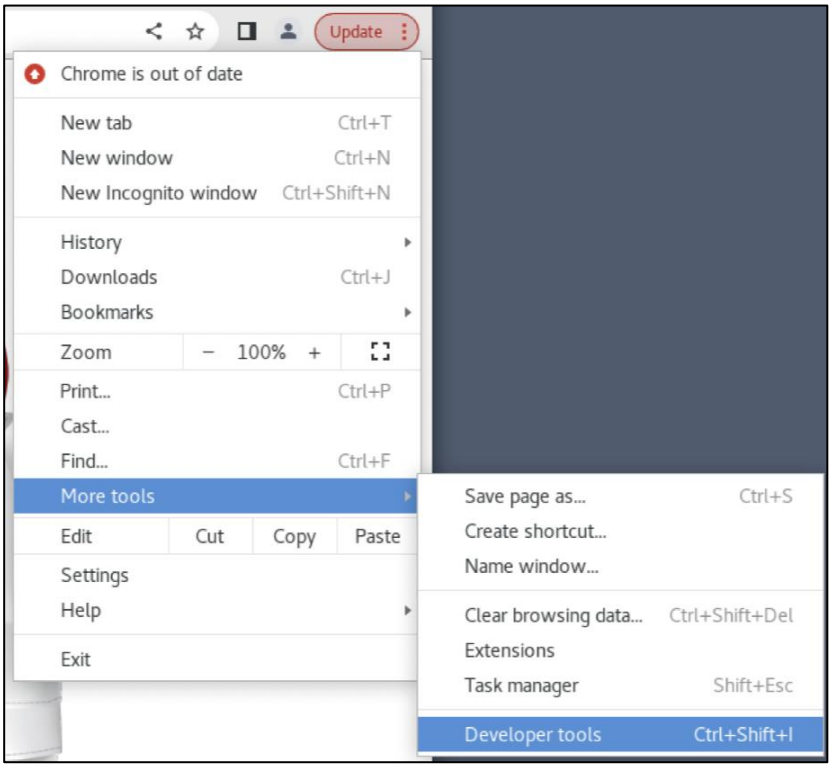
   e. Open the `index.html` document and link it to the `main.js` file.

   Hints:

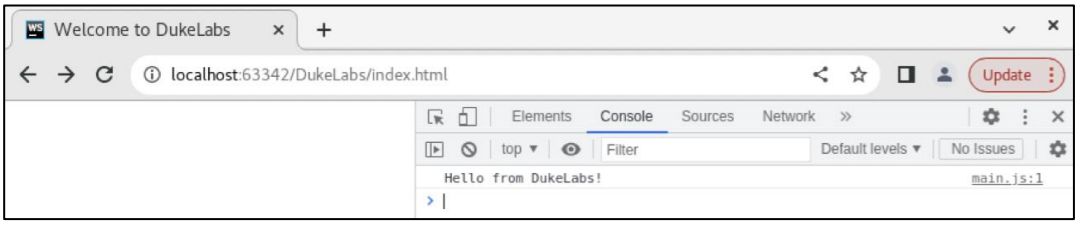
   – Place a JavaScript file reference inside the `index.html` head element.

```
<head>
  <meta charset="UTF-8">
  <title>Welcome to DukeLabs</title>
  <link rel="stylesheet" href="resources/style.css">
  <script src="resources/main.js"></script>
</head>
```

7. Test DukeLabs web application again to observe console output produced by the JavaScript code.
   a. Switch to the browser window used in the earlier test of the application.
   b. Click the three vertical dots to the side of the "Update" button.
   c. Select More Tools -> Developer Tools menu.



   d. Click the Console tab.
   e. Refresh the browser page.



Notes:

- Alternatively, you may choose to run the application again from WebStorm using “Run 'index.html' with coverage” button, or simply click “Rerun 'index.html'” in the console window displayed by WebStorm.
- You should also be able to see the console output produced by the JavaScript code snippet in the console window displayed by WebStorm as well as in the browsers JavaScript console.

Practices for Lesson 2: Introduction to JavaScript

Overview

In these practices, you will:

- Declare variables, define and invoke functions, create and use objects
- Explore JavaScript language constructs such as conditions and loops

Practice 2-1: Explore JavaScript Language Construct

Overview

The DukeLabs application needs to be able to manipulate data and perform different calculations to support its tracking of the scientific experiments and their measurements. During this practice, you'll implement JavaScript objects that represent experiments and measurements, define functions to process values and perform calculations, add variables and constants to represent data that would support actions performed by functions.

Tasks

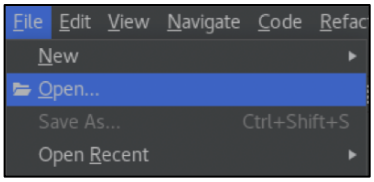
1. If you have not completed the previous practice, you may open a solution application for Lesson 1, which is the starting point of the Lesson 2 practice. Otherwise, you may continue to use same WebStorm project that you have worked on in the previous exercise.

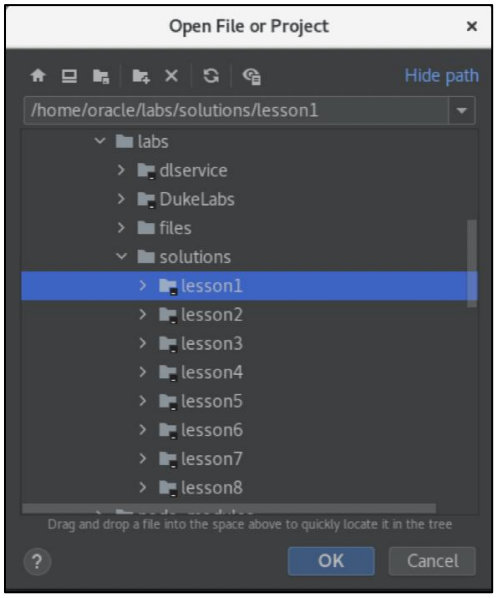
   Hints:

   - Launch WebStorm IDE if it is not already running.

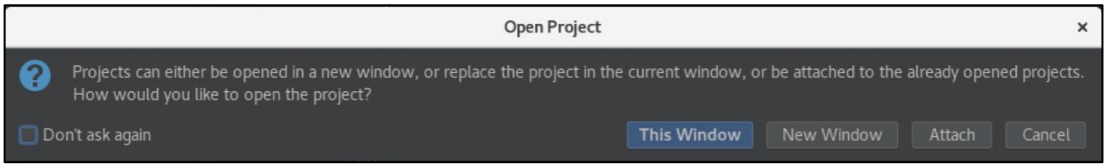
   

   - Use File → Open menu and browse to the `/home/oracle/labs/solutions/lesson1` folder.
   - Click the "OK" button.

   - Click the "This Window" button.

2. Define constants to support function calculations.
   a. Open the `main.js` file and immediately after the line of code that prints a hello message to the console, add a section that defines constants.
   b. These constants will be used later in this practice to support the following calculations:

      A pound to kilogram conversion that requires:

      - A conversion ratio of 2.2

      A duration of time calculation, which requires:

      - A number of milliseconds in a second 1000
      - A number of seconds in a minute 60
      - A number of seconds in an hour 3600

      Mark the beginning of this section of code with a comment that indicates its purpose.

```
/* --- Constants Section --- */
const LB_PER_KG = 2.2;
const MILLI_PER_SECOND = 1000;
const SECONDS_PER_MINUTE = 60;
const SECONDS_PER_HOUR = 3600;
```

3. Add two experiment objects that represent an ongoing and complete experiments, providing details such as the ID, task description, budget, start date and time of the experiment and a completion indicator. The end date and time of the experiment is only applicable if this experiment is considered complete.
   a. In the `main.js` file add a new section to describe objects. This section should be placed after the section that defined constants.
   b. Add an experiment object called experiment1, with the following properties:
      - id : 101
      - task : "Measure Weight"
      - budget : 123.45
      - startTime : new Date(2022,3,16,6,7)
      - complete : false

```
/* --- Objects Section --- */
// Create Experiment objects
let experiment1 = {
  id: 101,
  task: "Measure Weight",
  budget: 123.45,
  startTime: new Date(2022,3,16,6,7),
  complete: false
};
```

c. Add another experiment object called experiment2, with following properties:
   - id : 102
   - task : "Measure Length"
   - budget : 321.54
   - startTime : new Date(2022,4,1,14,30)
   - endTime : new Date(2022,4,2,21,12)
   - complete : true

```
let experiment2 = {
  id: 102,
  task: "Measure Length",
  budget: 321.54,
  startTime: new Date(2022,4,1,14,30),
  endTime: new Date(2022,4,2,21,12),
  complete: true
};
```

4. Add five measurement objects that represent measurements of weight and length, providing details such as the ID, unit of measure, measured value, and time when this measurement was recorded. The time value represents a period of time elapsed since the start of the experiment and should be expressed as a string value in ISO format "PT<hours>H<minutes>M<seconds>S".
   a. Add these measurement objects immediately after experiment objects.
   b. Add a measurement object called measurement1, with following properties:
      - id : 1
      - unit : "kg"
      - value : 42
      - time : PT2M12S

```
// Create Measurement objects
let measurement1 = {
  id: 1,
  unit: "kg",
  value: 42,
  time: 'PT2M12S'
};
```

   c. Add a measurement object called measurement2, with the following properties:
      - id : 2
      - unit : "kg"
      - value : 40
      - time : PT3M10S

```
let measurement2 = {
  id: 2,
  unit: "kg",
  value: 40,
  time: 'PT3M10S'
};
```

d. Add a measurement object called measurement3, with the following properties:
   - id : 3
   - unit : "kg"
   - value : 3
   - time : PT3M55S

```
let measurement3 = {
  id: 3,
  unit: "kg",
  value: 3,
  time: "PT3M55S"
};
```

e. Add a measurement object called measurement4, with the following properties:
   - id : 4
   - unit : "m"
   - value : 12
   - time : PT20M

```
let measurement4 = {
  id: 4,
  unit: "m",
  value: 12,
  time: "PT20M"
};
```

f. Add a measurement object called measurement5, with the following properties:
   - id : 5
   - unit : "m"
   - value : 10
   - time : PT1H20M10S

```
let measurement5 = {
  id: 5,
  unit: "m",
  value: 10,
  time: "PT1H22M10S"
};
```

5. Create an array of measurement objects and place the first three measurements in that array.
   a. The array of measurements should be placed after measurement object declarations.
   b. Create an array object called measurements and initialize it with the first three measurement objects.

```
// Create and populate an array of Measurement objects
let measurements = [measurement1, measurement2, measurement3];
```

6. Create functions that convert weight values between pound and kilograms.
   a. In the `main.js` file add a new section to describe function. This section should be placed after the section that defined objects.
   b. Create function to convert pounds to kilograms that accepts a value in pounds and returns this value divided by the pound to kilogram conversion ratio.

```
/* --- Function Section --- */
// Convert between pounds and kilograms
function lb2kg(lb) {
  return lb / LB_PER_KG;
}
```

   c. Create function to convert kilograms to pounds that accepts a value in kilograms and returns this value multiplied by the pound-to-kilogram conversion ratio.

```
function kg2lb(kg) {
  return kg * LB_PER_KG;
}
```

7. Add logic to test conversion functions.
   a. Create a new section in the `main.js` script file to test functions.
   b. Invoke function that converts kilograms to pounds to convert the value of the first measurement. Store the result of this conversion in a variable.

```
/* --- Test Section --- */
// Convert between pounds and kilograms
let lb = kg2lb(measurement1.value);
```

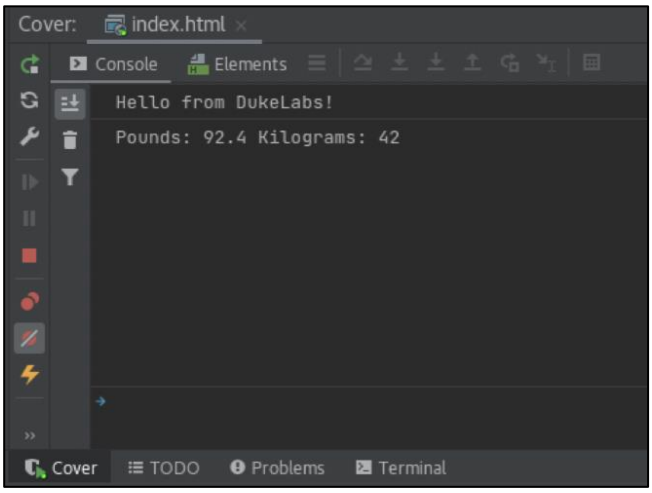
   c. Invoke function that converts pounds to kilograms to convert the value back. Store the result of this conversion in another variable.

```
let kg = lb2kg(lb);
```

   d. Print these pounds and kilogram values to the console.

```
console.log("Pounds: "+lb+" Kilograms: "+kg);
```

8. Test newly added logic.
    a. Run application if it is not already running. Otherwise simply switch to the browser window used in the earlier tests of the application and refresh the page.
    b. Observe the JavaScript console output. You should also be able to see the console output produced by the JavaScript code snippet in the console window displayed by WebStorm as well as in the browsers JavaScript console.



9. Create functions that convert time duration values between ISO formatted string and a value expresses in milliseconds.
    a. In the `main.js` file continue add new functions at the end of the function definitions section, just before the test section.

b. Create function to parse duration value expressed as an ISO formatted string and return the number of milliseconds for this duration.

Hints:

- Function should accept a string value of duration in ISO format.
- Duration format conforms to the regular expression:
  `/PT(?:([.,\d]+)H)?(?:([.,\d]+)M)?(?:([.,\d]+)S)?/`

  This expression expects text to start with letters PT, then have several digits followed by the letter H, which would indicate a number of hours, then several digits followed by the letter M, which indicates number of minutes, and finally several digits followed by the letter S, which indicates the number of seconds.
- Use function match of a regular expression object to retrieve hours, minutes, and seconds from the string.
- Function match returns an array of values that matched supplied pattern within a given string. Element at position 1 in this array would contain number of hours, position 2 number of minutes and position 3 number of seconds.
- If any of these values is not present in the string, then it should be substituted with zero.
- All of these values should be converted from string to number using the `parseInt` function.
- Calculate the number of milliseconds within this time period by adding a number of hours multiplied by a number of seconds per hours, number of minutes multiplied by a number of seconds per minute and multiply this by a number of milliseconds per second.
- Return the number of milliseconds.

```
// parse duration from ISO format "PT<hours>H<minutes>M<seconds>S"
// to milliseconds
function parseDuration(duration) {
  let durationPattern = /PT(?:([.,\d]+)H)?(?:([.,\d]+)M)?(?:([.,\d]+)S)?/;
  let matches = duration.match(durationPattern);
  let hours = (matches[1] === undefined) ? 0 : matches[1];
  let minutes = (matches[2] === undefined) ? 0 : matches[2];
  let seconds = (matches[3] === undefined) ? 0 : matches[3];
  return (parseInt(hours)*SECONDS_PER_HOUR+
          parseInt(minutes)*SECONDS_PER_MINUTE+
          parseInt(seconds))*MILLI_PER_SECOND;
}
```

c. Create function to format duration as an ISO string from a number of milliseconds.

Hints:

- Function should accept a value of duration in milliseconds.
- Function should return an equivalent string value in the ISO format.
- If the value of duration in milliseconds is 0, function should simply return "PT0S".
- Calculate total number of seconds by dividing duration by the number of milliseconds in a second and truncating the floating-point part of the result.
- Calculate number of hours by dividing total number of seconds by the number of seconds in an hour and truncating the floating-point part of the result.
- Calculate number of minutes by getting the remainder of division (modulus) of a total number of seconds by the number of seconds in an hour and truncating the floating-point part of the result and dividing by the number of seconds in a minute.
- Calculate number of seconds by getting the remainder of division (modulus) of a total number of seconds by the number of seconds in a minute and truncating the floating-point part of the result.
- The result string should start with letters "PT" and then contain number of hours followed by the letter H, then number of minutes followed by the letter M and number of seconds followed by the letter S.
- You should only attempt to concatenate hour, minute and second values to the result if these values are not zero.

```
// format duration from milliseconds into ISO format as
// "PT<hours>H<minutes>M<seconds>S"
function formatDuration(duration) {
  if (duration === 0) {
    return "PT0S";
  }
  let totalSeconds = Math.trunc(duration/MILLI_PER_SECOND);
  let hours = Math.trunc(totalSeconds / SECONDS_PER_HOUR);
  let minutes = Math.trunc((totalSeconds % SECONDS_PER_HOUR) /
                           SECONDS_PER_MINUTE);
  let seconds = Math.trunc(totalSeconds % SECONDS_PER_MINUTE);
  let result = "PT";
  if (hours != 0) {
    result+=hours+"H";
  }
  if (minutes != 0) {
    result+=minutes+"M";
  }
  if (seconds != 0) {
    result+=seconds+"S";
  }
  return result;
}
```

10. Add logic to test time period calculation functions.
    a. This logic should be added at the end of the function test section of the `main.js` script file.
    b. Invoke function that parses duration value and calculates the number of milliseconds. Invoke the `parseDuration` function, passing the value of the time property of the measurement3 as an argument. Store the milliseconds result value in a variable. Print the result to the console.

```
// Parse and format duration milliseconds and string representations
let durationMS = parseDuration(measurement3.time);
console.log("Measurement time offset from the start of the experiment:"+
durationMS);
```

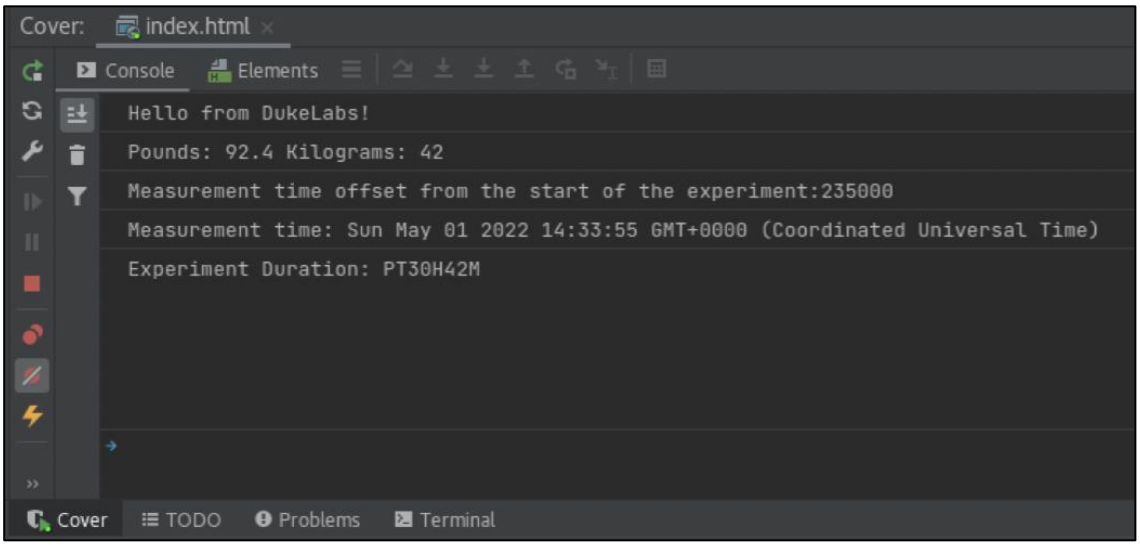
    c. Measurement object assumes that its time property represents a period of time elapsed since the start of the experiment. Calculate when this measurement was taken by adding duration in milliseconds to the start time of the experimant2. Store the result value in a variable. Convert the result value to Date and print the result to the console.

```
// Calculate measurement time
let measuremetTime = experiment2.startTime.getTime()+durationMS;
console.log("Measurement time: "+new Date(measuremetTime));
```

    d. Calculate overall duration of the experiment by subtracting the experiment end time from the experiment start time. Acquire start and end time values from the experimant2 object because it represents a complete experiment. The result of subtracting time values is a duration in milliseconds, which you should now convert into ISO string representation using the `formatDuration` function. Store the result value in a variable and print it to the console.

```
// Calculate experiment duration
let durationTime = formatDuration(experiment2.endTime-experiment2.startTime);
console.log("Experiment Duration: "+durationTime);
```

11. Test newly added logic.
    a. Run application if it is not already running; otherwise simply switch to the browser window used in the earlier tests of the application and refresh the page.
    b. Observe the JavaScript console output. You should also be able to see the console output produced by the JavaScript code snippet in the console window displayed by WebStorm as well as in the browsers JavaScript console.



12. Create a function that compares measurements.
    a. Add this function to the end of the function section in the `main.js` file, just before the test section.

b. Create `compareMeasurements` function that takes two measurement objects as parameters and returns a positive, a negative, or a zero-integer result indicating if one of the measurements is bigger, smaller, or equal to another.

Hints:

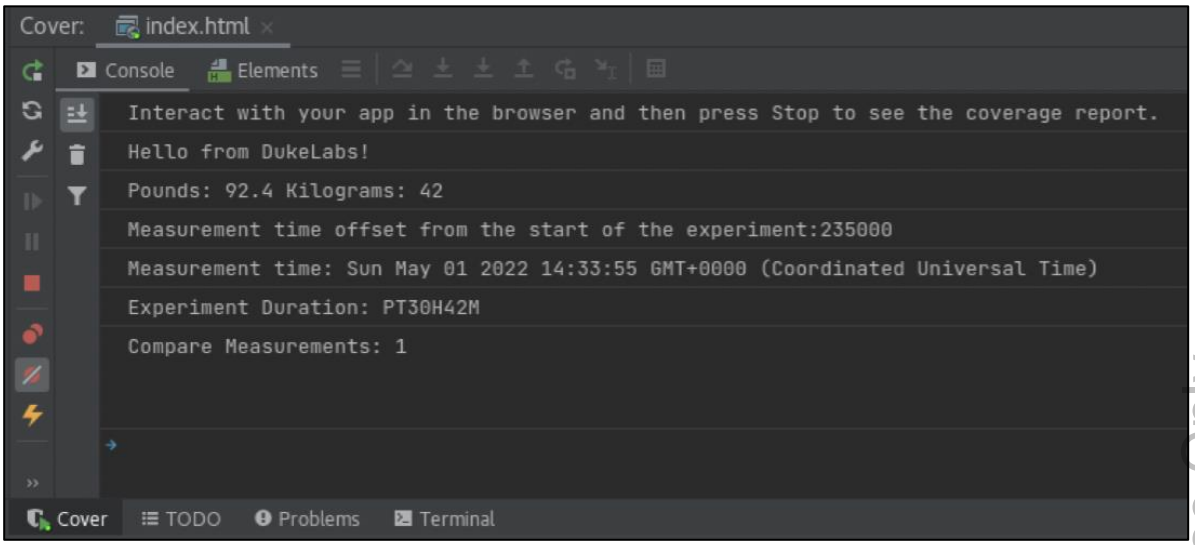
- Inside this newly added function declare a variable to hold the result of comparing measurements. Initialize this variable as 0.
- Check if the value of the first measurement is smaller than the value of second measurement, and if true assign -1 to the result.
- Check if measurements values are equal. If the first measurement value is not smaller then second, and if that is true, assign 0 to the result; otherwise, assign 1 to the result.
- Return the result.

```
// Compare two measurement objects
function compareMeasurements(m1, m2) {
  let result = 0;
  if (m1.value < m2.value) {
    result = -1;
  } else {
    if (m1.value == m2.value) {
      result = 0;
    } else {
      result = 1;
    }
  }
  return result;
}
```

13. Add logic to test the measurement-comparing function.
    a. This logic should be added at the end of the function test section of the `main.js` script file.
    b. Invoke the `compareMeasurements` function to compare `measurement1` and `measurement2` objects. Print the result to the console.

```
// Compare measurements:  smaller -1, equal 0, greater 1 value
console.log("Compare Measurements: " +
             compareMeasurements(measurement1, measurement2));
```

14. Test the newly added logic.
    a. Run application if it is not already running; otherwise simply switch to the browser window used in the earlier tests of the application and refresh the page.
    b. Observe the JavaScript console output. You should also be able to see the console output produced by the JavaScript code snippet in the console window displayed by WebStorm as well as in the browser's JavaScript console.



15. Create functions that calculate the average measurement value.
    a. Add these functions to the end of the function section in the `main.js` file, just before the test section.
    b. Create the `calculateAverageMeasurement` function that should take an array of measurement objects as an argument and return an average value of all measurements in that array.

       Hints:

       – Inside this newly added function, declare a variable to hold the result of the average calculation. Initialize this variable as 0.
       – Add a for-of iterator to step through all elements in the measurements array, extract each measurement value, parse it as a float number, and add to the result.
       – After the loop ends, divide the result by the number of elements in the measurements array. Round the result to a floating-point value with two-decimal points precision and return the result.

```
// Iterate through measurements array and calculate
// average value version 1 (assuming single unit of measure)
function calculateAverageMeasurement(measurements) {
  let result = 0;
  for (const measurement of measurements) {
    result += parseFloat(measurement.value);
  }
  result = (result/measurements.length).toFixed(2);
  return result;
}
```

c. Create the `calculateAverageMeasurements` function that should take an array of measurement objects as an argument and return an object containing an average value of measurements per unit of measure. At a minimum, consider two units of measure, for example, kilogram and meter.

Hints:

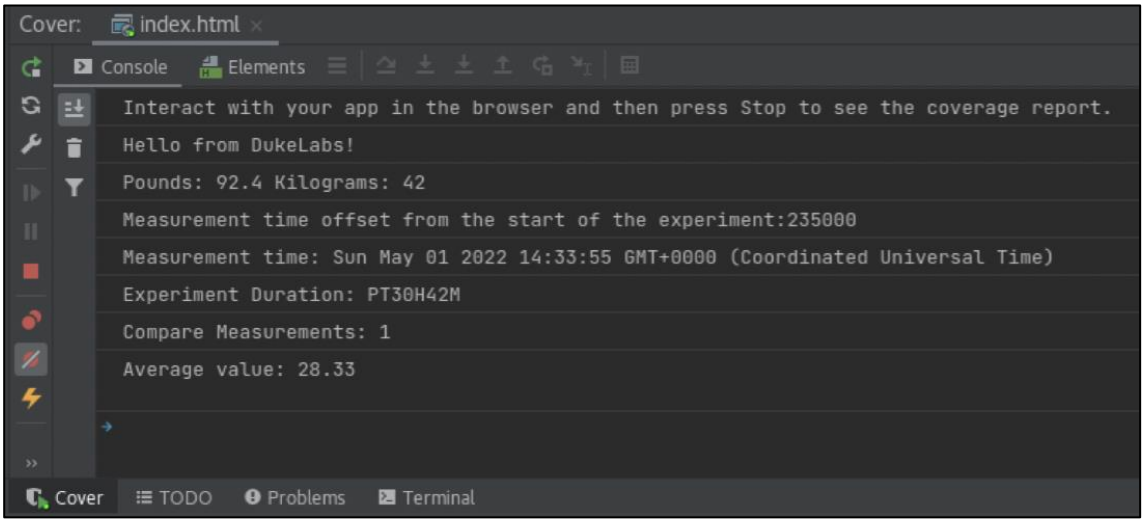
- Inside this newly added function, declare a variable to hold the result of the averages calculation. Initialize this variable as an object that contains the following properties:
  - A placeholder for a value of measurements in kilograms kgTotal: 0.0
  - A counter for a number of values in kilograms kgValue: 0
  - A placeholder for a value of measurements in meters mTotal: 0.0
  - A counter for a number of values in meters mValue: 0
- Add a for-of iterator to step through all elements in the measurements array.
- Inside this loop, add a switch that checks the unit of measure for each measurement.
- Add a case to the switch for each unit of measure (cover at least "kg" and "m" cases).
- Inside each case, extract a measurement value, parsed as float, add this value to a corresponding result object property and increment the corresponding result object counter, and then break out of the switch.
- After the loop ends, divide each total property in the result object by the corresponding counter value. Round the result to a floating-point value with two-decimal points precision and assign this back to the corresponding result object property. Return the result object.

```
// Iterate through measurements array and calculate
// average value version 2 (per each unit of measure)
function calculateAverageMeasurements(measurements) {
  let result = {
    kgTotal: 0.0,
    kgValues: 0,
    mTotal: 0.0,
    mValues: 0
  };
  for (const measurement of measurements) {
    switch (measurement.unit) {
      case "kg":
        result.kgTotal += parseFloat(measurement.value);
        result.kgValues++;
        break;
      case "m":
        result.mTotal += parseFloat(measurement.value);
        result.mValues++;
        break;
    }
  }
  result.kgTotal = (result.kgTotal/result.kgValues).toFixed(2);
  result.mTotal = (result.mTotal/result.mValues).toFixed(2);
  return result;
}
```

16. Add logic to test the average measurement calculation functions.
    a. This logic should be added at the end of the function test section of the `main.js` script file.
    b. Invoke the `calculateAverageMeasurement` function to compute an average value of all measurements in the measurements array. Assign the result to a valuable and print it to the console.

```
// Calculate measurements average value version 1 (assuming single unit)
let avgValue = calculateAverageMeasurement(measurements);
console.log("Average value: " + avgValue);
```

17. Test the newly added logic.
    a. Run the application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.
    b. Observe the JavaScript console output. You should also be able to see the console output produced by the JavaScript code snippet in the console window displayed by WebStorm as well as in the browser's JavaScript console.



    c. (Optionally) You may try to pass an empty array to the `calculateAverageMeasurement` function to test what happens if there are no elements in the array.

```
let avgValue = calculateAverageMeasurement([]);
```

    Note: The average value produced in this case would be NaN (Not a Number).
    d. Revert code back to use actual measurements array once you complete this test.
18. Add logic to test the average measurement calculation functions for multiple units of measure.
    a. This logic should be added at the end of the function test section of the `main.js` script file.
    b. Add two more measurements to the measurements array. There are three measurement objects already present in the array, so add `measurement4` at position 3 and `measurement5` at position 4.

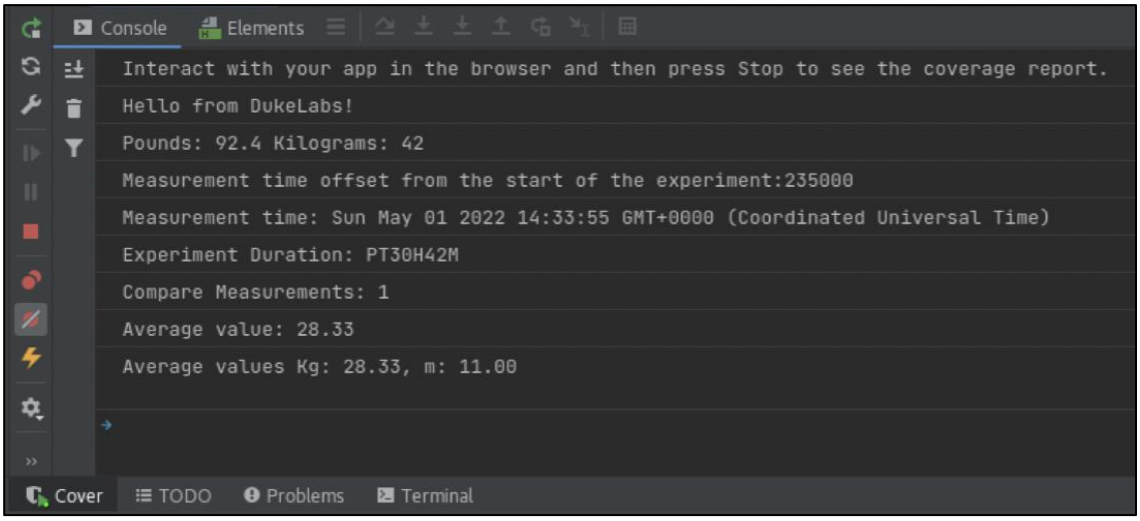
c. Invoke the `calculateAverageMeasurements` function to compute an average value of all measurements in the measurements array. Assign the result to a variable and print the value per each unit of measure to the console.

```
// Calculate measurements average value version 2 (per each unit)
measurements[3] = measurement4;
measurements[4] = measurement5;
let avgValues = calculateAverageMeasurements(measurements);
console.log("Average values Kg: " + avgValues.kgTotal+", m: "+
avgValues.mTotal);
```

19. Test the newly added logic.

a. Run the application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.

b. Observe the JavaScript console output. You should also be able to see the console output produced by the JavaScript code snippet in the console window displayed by WebStorm as well as in the browser's JavaScript console.



20. Create the `formatCurrency` function.

a. Add this function to the end of the function section in the `main.js` file, just before the test section.

b. Create the `formatCurrency` function that should take a numeric value as an argument and return string representation of that value formatted as currency. Use a number format object for the United Kingdom (British English) locale. You may choose a different localization, but then would have to consider adjusting the reset of the practice steps to match your choice of a locale.

Hints:

- Inside this newly added function, declare a format variable that references a Number format object initialized for the 'en-GB' locale, a currency style, with a minimum of 0 and maximum of 2 fraction digits.
- Format the `value` parameter by invoking the `format` function of the format object.
- Return the formatted result.

```
// format currency
function formatCurrency(value){
  const format = new Intl.NumberFormat('en-GB',
    {style: 'currency', currency: 'GBP',
    minimumFractionDigits: 0, maximumFractionDigits: 2});
  return format.format(value);
}
```

21. Create the `formatExperiment` and `formatMeasurement` functions.
    a. Add these functions to the end of the function section in the `main.js` file, just before the test section.

b. Create the `formatExperiment` function that should take an experiment object as an argument and return string representation of that object.

Hints:

- Inside this newly added function, declare a variable to hold the result for the experiment object formatting.
- Initialize this variable to contain a prompt "Experiment " followed by the concatenated experiment ID and task properties. Enclose task text in quotation marks. Insert a space between property values and also at the end of this string.
- Add prompt "Budget " to the result followed by the budget property of this experiment, formatted as currency. Add a space that the end of this string.
- Add the `startTime` property of the experiment formatted as text, using British English locale and a London time zone. Add a space that the end of this string.
- If the experiment is complete, add the end time property of the experiment formatted as text with the same locale and time zone; otherwise, add the words "on going".
- Return the formatted result.

```
// Format Experiment object
function formatExperiment(experiment) {
  let result = "Experiment "+experiment.id+" \""+experiment.task+"\" ";
  result += "Budget: "+formatCurrency(experiment.budget)+" ";
  result += experiment.startTime.toLocaleString("en-GB",
              {timeZone: "Europe/London"})+" ";
  result += (experiment.complete) ?
              experiment.endTime.toLocaleString("en-GB",
              {timeZone: "Europe/London"}) : "on going";
  return result;
}
```

c. Create the `formatMeasurement` function that should take a measurement object as an argument and return string representation of that object.

Hints:

- Inside this newly added function, declare a variable to hold the result for the experiment object formatting.
- Initialize this variable to contain a prompt "Measurement " followed by the concatenated measurement ID, unit, value, and time properties.
- Return the formatted result.

```
// Format Measurement object
function formatMeasurement(measurement) {
    let result = "Measurement "+measurement.id+" "+measurement.unit+
    " "+measurement.value+" "+measurement.time;
    return result;
}
```

22. Add logic to test the formatting functions.
    a. This logic should be added at the end of the function test section of the `main.js` script file.
    b. Invoke the `formatExperiment` function for each experiment and print the formatted result to the console.

```
// Print each experiment
console.log(formatExperiment(experiment1));
console.log(formatExperiment(experiment2));
```

    c. Sort object in the measurements array.

    Hints:

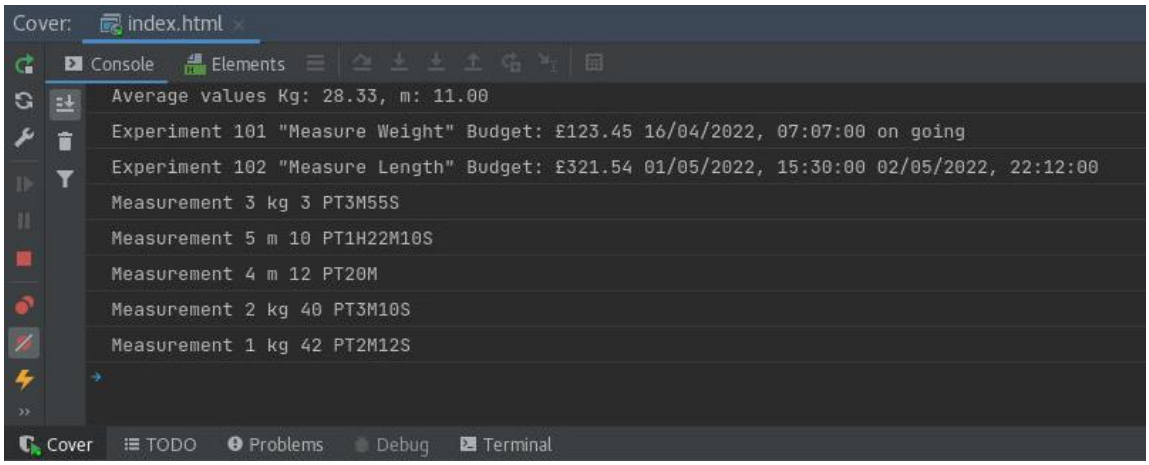
    – The `compareMeasurements` function can be used to implement sorting of the measurement objects. JavaScript array provides a sort method that compares objects in the array using any function that can return -1, 0, 1 values to establish the order of elements.
    – Invoke the `sort` function upon the measurements array and pass the `compareMeasurements` function as an argument.

```
// Sort measurements
measurements.sort(compareMeasurements);
```

    d. Iterate through the measurements array formatting each object in there and printing the results to the console.

```
// Iterate and print array of measurements
for (const measurement of measurements) {
  console.log(formatMeasurement(measurement));
}
```

23. Test the newly added logic.
    a. Run the application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.
    b. Observe the JavaScript console output. You should also be able to see the console output produced by the JavaScript code snippet in the console window displayed by WebStorm as well as in the browser's JavaScript console.



Practices for Lesson 3: Functions, Classes, and Objects

Overview

In these practices, you will:

- Define and use JavaScript classes
- Create and use properties, constructors, accessors, instance, and static functions
- Instantiate classes and place resulting object into collections

Practice 3-1: Refactor Code to Improve Its Cohesion and Structure

Overview

In this practice, you will refactor code of the DukeLabs application to improve its structure and cohesion. Practice introduces Experiment and Measurements classes to establish data structures and functionalities for the corresponding types of objects. Another important change is the introduction of the data object that would hold collections of experiment and measurement objects and provide access operations. This data object would act as a kind of in-memory "database" structure.

Tasks

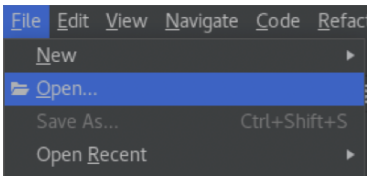
1. If you have not completed previous practice, you may open a solution application for Lesson 2, which is the starting point of the Lesson 3 practice. Otherwise, you may continue to use same WebStorm project that you have worked on in the previous exercise.

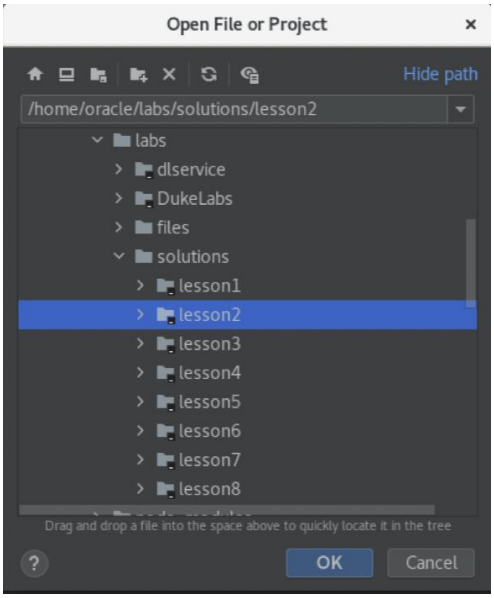
   Hints:

   - Launch WebStorm IDE if it is not already running.

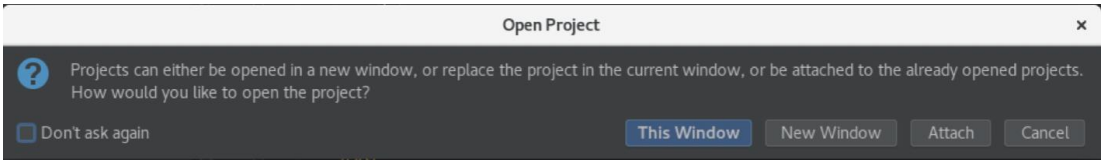
   

   - Use File → Open menu to and browse to the `/home/oracle/labs/solutions/lesson2` folder.
   - Click the "OK" button.

   - Click the "This Window" button.

2. Define the Experiment class.
   a. Open the `main.js` file and add a new class definition section immediately after the end of the constants section, before the object section.
   b. In this section, create an Experiment class with the following private properties: id, task, budget, startTime, endTime, complete. (Reminder: private property names start with # symbol).

```
/* Class definition section */
// define Experiment class
class Experiment {
  // declare private properties
  #id;
  #task;
  #budget;
  #startTime;
  #endTime;
  #complete;
  // the rest of the experiment class code will be placed here
}
```

   c. Inside this class, add a constructor that initializes all properties of the Experiment.

```
// define constructor to initialize
// all properties of an Experiment object
constructor(id, task, budget, startTime, endTime, complete) {
  this.#id = id;
  this.#task = task;
  this.#budget = budget;
  this.#startTime = startTime;
  this.#endTime = endTime;
  this.#complete = complete;
}
```

   d. Inside this class, add two static methods that instantiate experiment as either complete, or on-going. Both of these methods invoke the same Experiment constructor; however, the on-going method default the `endTime` as null and sets complete indicator as false.

```
// define static function to initialize a complete Experiment object
static createCompleteExperiment(id, task, budget, startTime, endTime) {
  return new Experiment(id, task, budget, startTime, endTime, true);
}
// define static function to initialize an ongoing Experiment object
static createOngoingExperiment(id, task, budget, startTime) {
  return new Experiment(id, task, budget, startTime, null, false);
}
```

e. Inside this class, add a `toString` function that produces text representation of the experiment object. The body of this function can be copied from the `formatExperiment` function. However, all references to the experiment object must be replaced with the current object "this" reference.

```
// copy formatExperiment function logic into the toString operation
// modify property references to print current objects private properties
toString() {
  let result = "Experiment "+this.#id+" \""+this.#task+"\" ";
  result += "Budget: "+formatCurrency(this.#budget)+" ";
  result += this.#startTime.toLocaleString("en-GB",
              {timeZone: "Europe/London"})+" ";
  result += (this.#complete) ?
               this.#endTime.toLocaleString("en-GB",
               {timeZone: "Europe/London"}) : "ongoing";
  return result;
}
```

f. Comment out the original `formatExperiment` function. (Select relevant lines of code and press CTRL+/ on a keyboard). This would break the existing code that invoked this function, but later in this practice you will be replacing this code with the use of a `toString` function anyway.

```
// // Format Experiment object
// function formatExperiment(experiment) {
//     let result = "Experiment "+experiment.id+" \""+experiment.task+"\" ";
//     result += "Budget: "+formatCurrency(experiment.budget)+" ";
//     result += experiment.startTime.toLocaleString("en-GB",
//     {timeZone: "Europe/London"})+" ";
//     result += (experiment.complete) ?
//     experiment.endTime.toLocaleString("en-GB", {timeZone: "Europe/London"})
//     : "on going";
//     return result;
// }
//
```

g. Inside this class, add a get property accessors for each property. Each accessor should have a name that corresponds to the property that it returns.

h. Also add a set accessor for the complete property that should accept the `endTime` value for the experiment as an argument. Assign this value to the corresponding property and set the complete property to true.

```
// create get property accessors
get id() {
  return this.#id;
}
get task() {
  return this.#task;
}
get budget() {
  return this.#budget;
}
get startTime() {
  return this.#startTime;
}
get endTime() {
  return (this.#complete) ? this.#endTime : "ongoing";
}
get complete() {
  return this.#complete;
}
// create set property accessor that completes the experiment
set complete(endTime) {
  this.#endTime = endTime;
  this.#complete = true;
}
```

3. Define the Measurement class.

a. In the `main.js` file, add the new Measurement class definition section immediately after the end of the Experiment class definition section, before the object section.

a. In this section, create a Measurement class with the following private properties: id, unit, value, time. (Reminder: private property names start with # symbol).

```
// define Measurement class
class Measurement {
  // declare private properties
  #id;
  #unit;
  #value;
  #time;
  // the rest of the measurement class code will be placed here
}
```

b. Inside the Measurement class, add a private static array containing SI units of measurement. (SI units are "s","m","kg","A","K","mol","cd").

```
// create private static array of SI units of measure
static #units = ["s","m","kg","A","K","mol","cd"];
```

c. Inside the Measurement class, add a private static property called maxId to store the current highest measurement id value and initialize it to 0.

```
// create a private static property to store current highest id value
static #maxId = 0;
```

d. Inside the Measurement class, add a constructor that initializes all private properties of the measurement. Accept unit, value, and time as arguments. Calculate the current measurement id by incrementing the maxId value. Validate that the unit of measure argument is actually an SI unit (check if it is present in the units array. If it is not, assign null to the unit property).

```
// define constructor to initialize
// all private properties of a Measurement object
// validate that unit of measure is an SI unit
constructor(unit, value, time) {
  this.#id = ++Measurement.#maxId;
  this.#unit = (Measurement.#units.indexOf(unit) == -1) ? null : unit;
  this.#value = value;
  this.#time = time;
}
```

e. Inside this class, add a `toString` function that produces text representation of the measurement object. The body of this function can be copied from the `formatMeasurement` function. However, all references to the measurement object must be replaced with the current object "this" reference.

```
// copy formatMeasurement function logic into the toString operation
// modify property references to print current objects private properties
toString() {
  let result = "Measurement "+this.#id+" "+this.#unit+" "+
                              this.#value+" "+this.#time;
   return result;
}
```

f. Comment out the original `formatMeasurement` function. (Select relevant lines of code and press CTRL+/ on a keyboard.) This would break the existing code that invoked this function, but later in this practice you will be replacing this code with the use of a `toString` function anyway.

```
// // Format Measurement object
// function formatMeasurement(measurement) {
//     let result = "Measurement "+measurement.id+" "+measurement.unit+
// " "+measurement.value+" "+measurement.time;
//     return result;
// }
```

g. Inside this class, add `get` property accessors for each property. Each accessor should have a name that corresponds to the property that it returns.

```
// create get property accessors
get id() {
  return this.#id;
}
get unit() {
  return this.#unit;
}
get value() {
  return this.#value;
}
get time() {
  return this.#time;
}
```

4. In the Objects section of the `main.js` script file, add a data object that will act as an in-memory database which is going to store experiment and measurement objects and provide capabilities to add and find experiments and measurements.

   a. At the start of the object section of the `main.js` file, add a new constant called data. This constant would reference the in-memory "database" for experiments and measurements.

```
/* --- Objects Section --- */
// An in-Memory "Database" with insert and search operations
const data = {
  // the rest of the data object code would be placed here
}
```

   b. Inside the data object, add the following properties:

   – A map of all data that is going to be stored as a set of instances of measurements indexed by an experiment object

   – A map of experiment objects indexed by experiment ID

   – A map of measurement objects indexed by measurement ID

   – Notice that properties end in a comma, because more properties would be added to this object in the next several steps of this practice.

```
// a Map comprised of a Set of Measurements indexed by an Experiment object
allData : new Map(),
// a Map comprised of Experiment objects indexed by ID
experiments : new Map(),
// a Map comprised of Measurement objects indexed by ID
measurements : new Map(),
```

   c. Next, inside the data object, add a function that will find and return an experiment object for a corresponding id value. This function should accept experiment ID as an argument, get the experiment with this ID from the experiments map and return it.

```
// Find an Experiment with a given ID
getExperiment(eId) {
  return this.experiments.get(eId);
},
```

   d. Next, inside the data object, add a function that will find and return a measurement object for a corresponding ID value. This function should accept measurement ID as an argument, get the measurement with this ID from the measurements map and return it.

```
// Find a Measurement with a given ID
getMeasurement(mId) {
  return this.measurements.get(mId);
},
```

   e. Next, inside the data object, add a function that will find and return all measurements for a given experiment ID. This function should accept experiment ID as an argument, get a set of all corresponding the measurements from all data map and return this set.

```
// Find all Measurement objects for a given experiment ID
getMeasurements(eId) {
  return this.allData.get(this.getExperiment(eId));
},
```

f. Next, inside the data object, add a function that will add an experiment object to the data object. This function should accept experiment object as an argument, place this object into the experiments map using this experiment ID as an index. Also, the same object should be placed into the map of all data together with an empty set that could be later used to add measurements to this experiment.

```
// Add a new Experiment object
addExperiment(experiment) {
  this.experiments.set(experiment.id, experiment);
  this.allData.set(experiment, new Set());
},
```

g. Next, inside the data object, add a function that will add a measurement object to the data object. This function should accept two arguments, an ID of the experiment and a measurement object that should belong to this experiment. The measurement object should be placed into the measurements map using this measurement ID as an index. Also, the same object should be placed into the map of all data by adding it to the set of measurements for the corresponding experiment.

```
// Add a new Measurement to an Experiment with a given ID
addMeasurement(eId, measurement) {
  this.measurements.set(measurement.id, measurement);
  this.allData.get(this.getExperiment(eId)).add(measurement);
}
```

Notice that this is the last property that will be added to the data object, thus it does not end in a comma symbol.

5. Replace the section that creates experiment and measurement objects, with code that instantiates Experiment and Measurement classes.
   a. Comment out all the lines of code that produced experiment and measurement objects. (Select relevant lines of code and press CTRL+/ on a keyboard.)

```
// // Create Experiment objects
// let experiment1 = {
//     id: 101,
//     task: "Measure Weight",
//     budget: 123.45,
//     startTime: new Date(2022,3,16,6,7),
//     complete: false
// };
// let experiment2 = {
//     id: 102,
//     task: "Measure Length",
//     budget: 321.54,
//     startTime: new Date(2022,4,1,14,30),
//     endTime: new Date(2022,4,2,21,12),
//     complete: true
// };
//
// // Create Measurement objects
// let measurement1 = {
//     id: 1,
//     unit: "kg",
//     value: 42,
//     time: 'PT2M12S'
// };
// let measurement2 = {
//     id: 2,
//     unit: "kg",
//     value: 40,
//     time: 'PT3M10S'
// };
// let measurement3 = {
//     id: 3,
//     unit: "kg",
//     value: 3,
//     time: "PT3M55S"
// };
// let measurement4 = {
//     id: 4,
//     unit: "m",
//     value: 12,
//     time: "PT20M"
// };
// let measurement5 = {
//     id: 5,
//     unit: "m",
//     value: 10,
//     time: "PT1H22M10S"
// };
```

b. Create two experiment objects using `createOngoingExperiment` and `createCompleteExperiment` static methods of the Experiment class. Use the same values as were used previously in the segment of code that you have just commented out.

```
// Instantiate Experiment objects
let experiment1 =
  Experiment.createOngoingExperiment(101, "Measure Weight", 123.45,
    new Date(2022,3,16,6,7));
let experiment2 =
  Experiment.createCompleteExperiment(102, "Measure Length", 321.54,
    new Date(2022,4,1,14,30), new Date(2022,4,2,21,12));
```

c. Create five measurement objects using Measurement constructor. Use the same values as were used previously in the segment of code that you have just commented out.

```
// Instantiate Measurement objects
let measurement1 = new Measurement("kg",42,'PT2M12S');
let measurement2 = new Measurement("kg",40,'PT3M10S');
let measurement3 = new Measurement("kg",3,"PT3M55S");
let measurement4 = new Measurement("m",12,"PT20M");
let measurement5 = new Measurement("m",10,"PT1H22M10S");
```

Note: The existing code in this script was referencing experiment and measurement objects using the same variable names. Thus, even though the actual initialization mechanism for these variables has been changed, the remaining code of the practice would not be affected. However, because object definitions themselves are now different, existing code that is using specific properties and behaviors of these objects can break.

For example, the existing code is using `formatExperiment` and `formatMeasurement` functions, which no longer exist, so it should be refactored to use the `toString` method instead. Likewise other functions that were previously designed as stand-alone, could be refactored as parts of the relevant object or class definitions. This could be a good design choice for a large application that can benefit from grouping functions together.

This practice does not ask you to actually perform such refactoring, but it should be noted that some of the functions, such as conversion, duration calculations, or sorting and so on can be moved inside Experiment, or Measurement classes, or placed inside the data object, which could improve overall code cohesion in a large application.

Also, notice that there is no longer a need to supply the measurement ID value to produce measurement objects, because this ID is autogenerated by the measurement constructor.

6. Place experiment and measurement objects into a data object.
   a. This code needs to be added after the instantiation of the experiment and measurement objects performed at the end of the object section of the `main.js` file.
   b. Invoke the `addExperiment` function of the data object to add two experiment objects to the data object.

```
// Add experiments and measurements to data object
data.addExperiment(experiment1);
data.addExperiment(experiment2);
```

c. Invoke the `addMeasurement` function of the data object to add all measurement objects to the data object. Associate measurements 1, 2 and 3 with the first experiment, and measurements 4 and 5 with a second experiment.

```
data.addMeasurement(experiment1.id,measurement1);
data.addMeasurement(experiment1.id,measurement2);
data.addMeasurement(experiment1.id,measurement3);
data.addMeasurement(experiment2.id,measurement4);
data.addMeasurement(experiment2.id,measurement5);
```

d. Comment out the line of code that defines a measurements array. (Select the relevant lines of code and press CTRL+/ on a keyboard.) – This array object is no longer needed, because all experiments and measurement are now linked together and contained within the data object and can always be retrieved from it later as required.

```
// Create and populate an array of Measurement objects
// let measurements = [measurement1, measurement2, measurement3];
```

e. Add a line of code that initializes the measurements array by retrieving measurements from the data object. Notice, coincidently measurements 1, 2 and 3 are associated with the experiment 101.

```
let measurements = Array.from(data.getMeasurements(101));
```

7. Test the application capabilities of retrieving experiment and measurement objects from the data object.
   a. In the test section of the `main.js` file, locate lines of code that reference experiment and measurement object references and replace these with `getExperiment` and `getMeasurement` function calls. Valid experiment ID values are 101 and 102, and measurement ID values are 1,2,3,4,5.

```
// let lb = kg2lb(measurement1.value);
let lb = kg2lb(data.getMeasurement(1).value);
let kg = lb2kg(lb);
console.log("Pounds: "+lb+" Kilograms: "+kg);

// Parse and format duration milliseconds and string representations
// let durationMS = parseDuration(measurement3.time);
let durationMS = parseDuration(data.getMeasurement(3).time);
console.log("Measurement time offset from the start of the experiment: "
+durationMS);

// Calculate measurement time
// let measuremetTime = experiment2.startTime.getTime()+durationMS;
let measuremetTime = data.getExperiment(102).startTime.getTime()+durationMS;
console.log("Measurement time: "+new Date(measuremetTime));

// Calculate experiment duration
// let durationTime =
//   formatDuration(experiment2.endTime-experiment2.startTime);
let durationTime =
  formatDuration(data.getExperiment(102).endTime -
                 data.getExperiment(102).startTime);
console.log("Experiment Duration: "+durationTime);

// Compare measurements:  smaller -1, equal 0, greater 1 value
// console.log("Compare Measurements: " +
//   compareMeasurements(measurement1, measurement2));
console.log("Compare Measurements: " +
compareMeasurements(data.getMeasurement(1), data.getMeasurement(2)));
```

b. Replace the code that added measurements 3 and 4 to the measurements array, with the logic that retrieves these two measurements from the data object and pushes them into the measurements array.

Hints:

- Locate lines of code that add measurements 3 and 4 to the measurements array in the test section of the `main.js` file and comment these lines out.
- Coincidently these two objects are associated with the experiment 102 so they could be retrieved from the data object as a set. Use the function `getMeasurements` of the data object to retrieve these two measurements.
- Form an array out of the set of measurements returned by the `getMeasurements` function.
- Iterate through this array elements and push each element into the measurements array.

```
// measurements[3] = measurement4;
// measurements[4] = measurement5;
for (const measurement of Array.from(data.getMeasurements(102))) {
    measurements.push(measurement);
}
```

8. Test functionality of the `toString` operations of the Experiment and Measurement classes.

a. Comment out lines of code that use the `formatExperiment` function to print experiments to the console.

```
// Print each experiment
// console.log(formatExperiment(experiment1));
// console.log(formatExperiment(experiment2));
```

b. Add code that retrieves experiment 101 from a data object and print it to the console using its `toString` operation.

```
// Retrieve experiments and measurements from a data object
console.log(data.getExperiment(101).toString());
```

c. Add code that retrieves measurements 1, 2 and 3 from a data object and print these to the console using their `toString` operation.

```
console.log(data.getMeasurement(1).toString());
console.log(data.getMeasurement(2).toString());
console.log(data.getMeasurement(3).toString());
```

d. Add code that retrieves experiment 102 and all of its measurements from a data object and print them all to the console.

```
console.log(data.getExperiment(102).toString());
for (const measurement of data.getMeasurements(102)) {
  console.log(measurement.toString());
}
```

e. Comment out the line of code that uses formatMeasurement function call to print measurements to the console.

```
// Iterate and print array of measurements
// for (const measurement of measurements) {
//     console.log(formatMeasurement(measurement));
// }
```

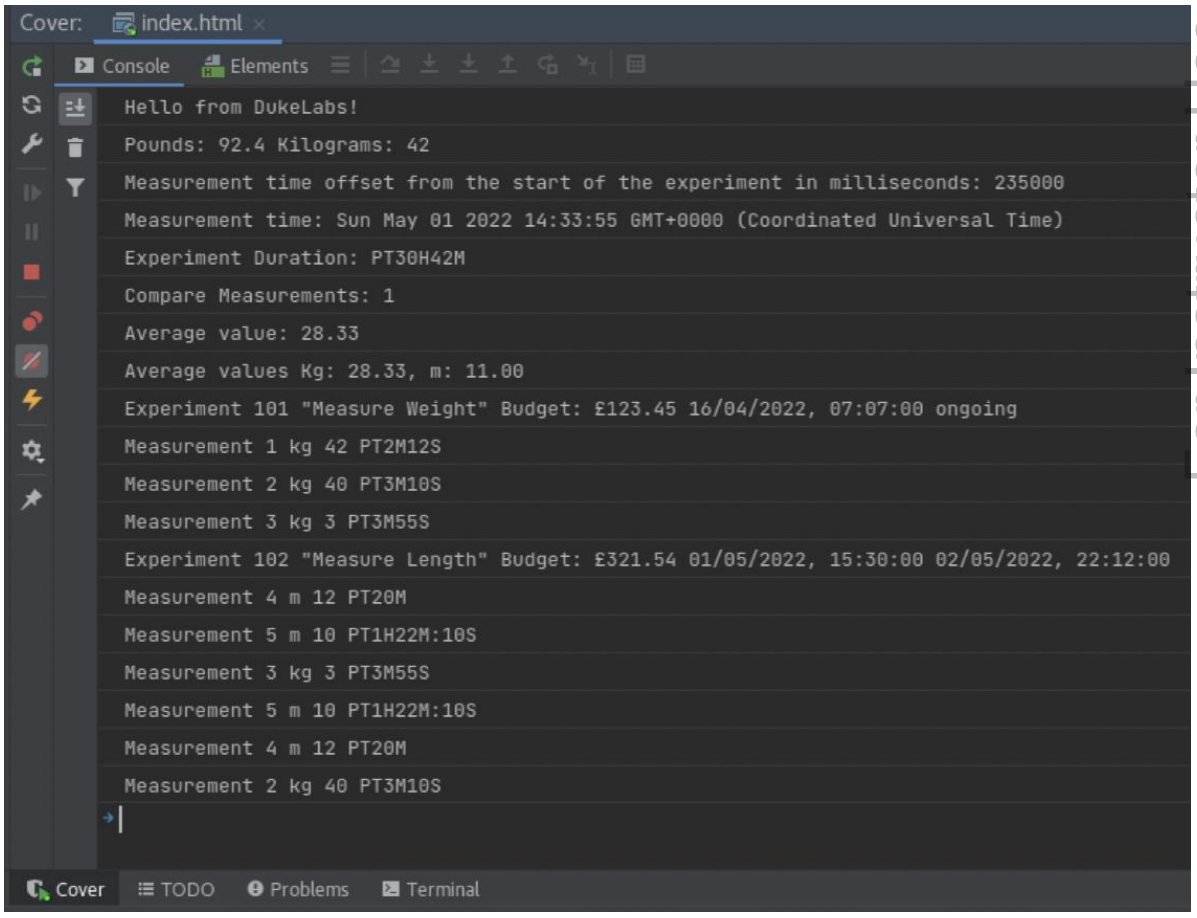
f. Add the line of code that uses `toString` to print measurements to the console.

```
for (const measurement of measurements) {
  console.log(measurement.toString());
}
```

9. Test the application logic. Changes made so far in this practice were almost exclusively focused on restructuring an improving code, replacing implementation behind existing functionalities, rather than adding new ones. Therefore, console output produced in this practice should be very similar to that of the previous practice.

   a. Run the application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.

   b. Observe the JavaScript console output. You should also be able to see the console output produced by the JavaScript code snippet in the console window displayed by WebStorm as well as in the browser's JavaScript console.



Practice 3-2: Dynamically Alter and Introspect Object Properties

Overview

In this practice, you will alter one of the experiment objects by adding a notes property to it. After that you will write code to dynamically introspect this experiment object to find information about its properties. Lastly, you will test how freezing an object prevents its alteration.

Tasks

1. Dynamically alter one of the experiment objects.
   a. Place code that alters the experiment at the end of the test section of the `main.js` file.
   b. Retrieve experiment with ID 101 from the data object and assign it to the experiment variable.

```
// Alter an experiment object and introspect its properties
let experiment = data.getExperiment(101);
```

   c. Add a notes property to this experiment. Initialize this property to contain any value, for example, "This is fun" string.

```
experiment.notes = "This is fun";
```

2. Dynamically discover object properties.
   a. Check if the experiment object has the property called "notes".

```
console.log(Object.hasOwn(experiment,"notes"));
```

   b. Check if the experiment object is extensible.

```
console.log(Object.isExtensible(experiment));
```

   c. Check if the experiment object is sealed.

```
console.log(Object.isSealed(experiment));
```

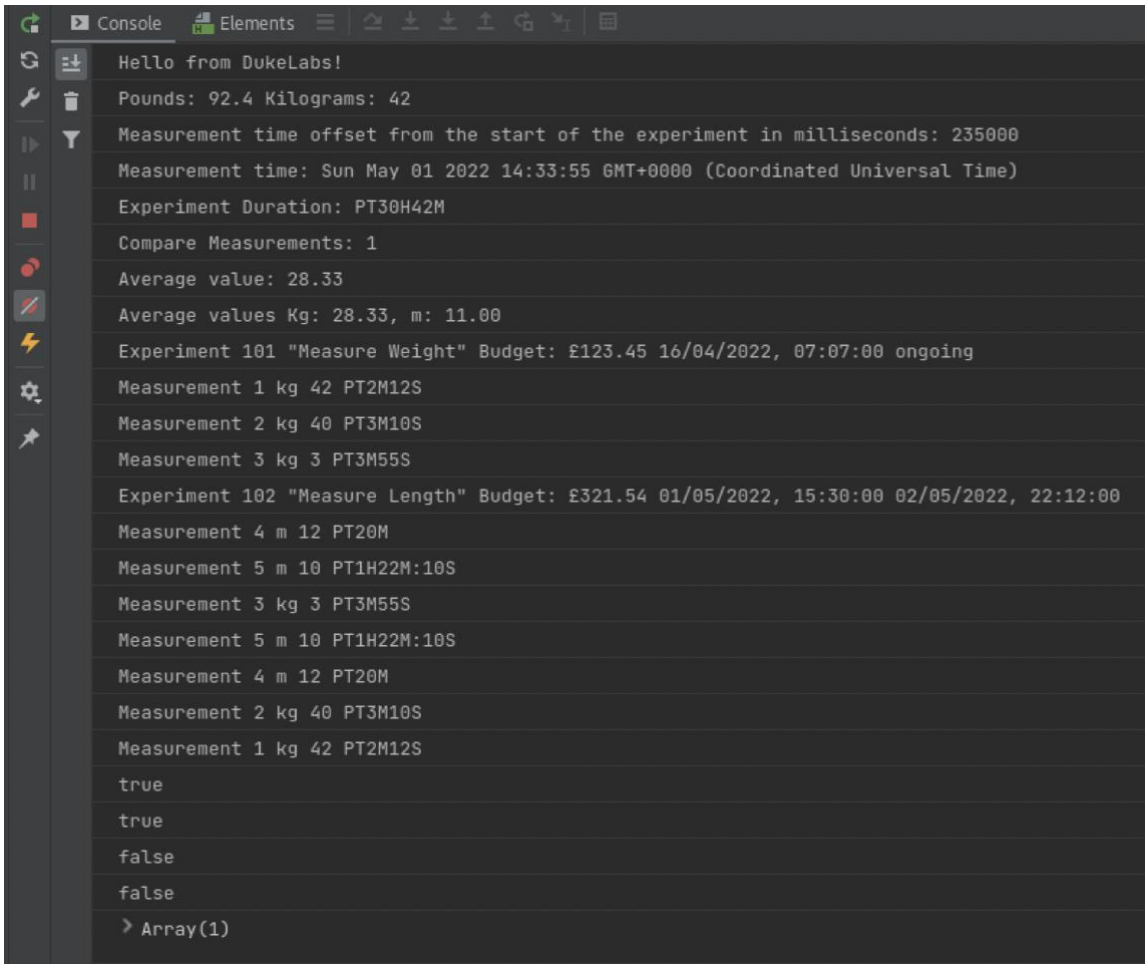
   d. Check if the experiment object is frozen.

```
console.log(Object.isFrozen(experiment));
```

   e. Acquire experiment object properties.

```
console.log(Object.entries(experiment));
```

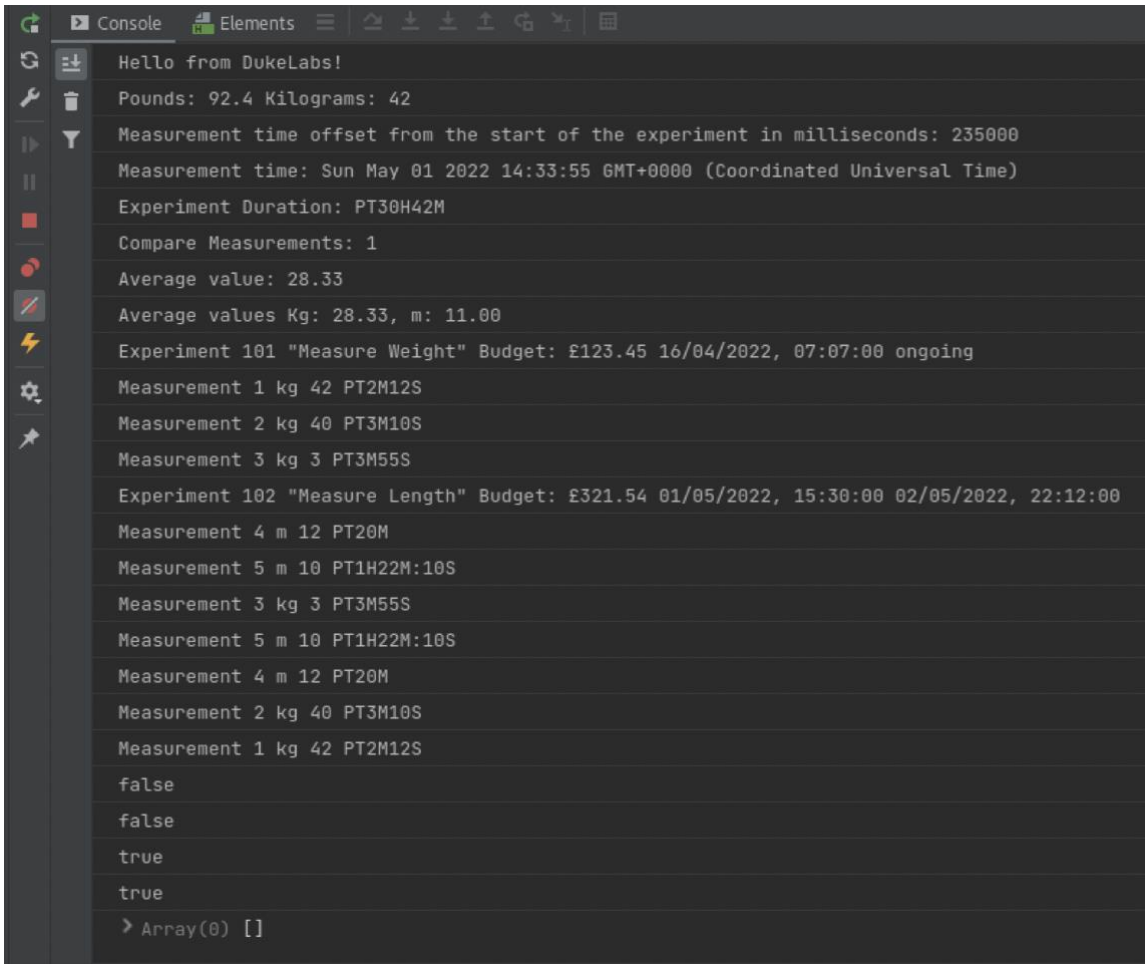
3. Test the newly added code.
   a. Run the application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.
   b. Observe the JavaScript console output. You should also be able to see the console output produced by the JavaScript code snippet in the console window displayed by WebStorm as well as in the browser's JavaScript console.



4. Prevent dynamic alteration of the experiment object.
   a. Insert a new line of code immediately after the line that retrieves the experiment 101 object.
   b. Invoke the `Object.freeze` function to prevent dynamic modifications of this experiment object.

```
// Alter an experiment object and introspect its properties
let experiment = data.getExperiment(101);
Object.freeze(experiment);
```

5. Test the newly added code.
   a. Run the application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.
   b. Observe the JavaScript console output. You should also be able to see the console output produced by the JavaScript code snippet in the console window displayed by WebStorm as well as in the browser's JavaScript console.



Practices for Lesson 4: Extending Classes and Objects

Overview

In these practices, you will:

- Create a class that extends another class
- Add additional function to the prototype

Practice 4-1: Extending Classes and Altering Prototypes

Overview

In this practice, you will create a thought experiment class that represents a variant (subtype) of the experiment. Another task will be to alter experiment class prototype to add additional function that can locate measurements that belong to this experiment.

Tasks

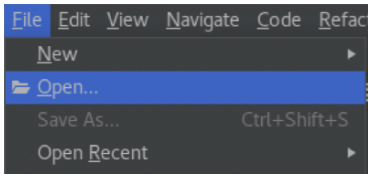
1. If you have not completed the previous practice, you may open a solution application for Lesson 3, which is the starting point of the Lesson 4 practice. Otherwise, you may continue to use same WebStorm project that you have worked on in the previous exercise.

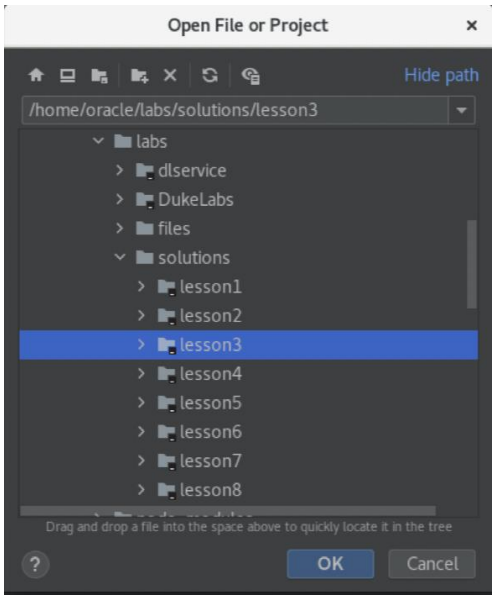
   Hints:

   - Launch WebStorm IDE if it is not already running.

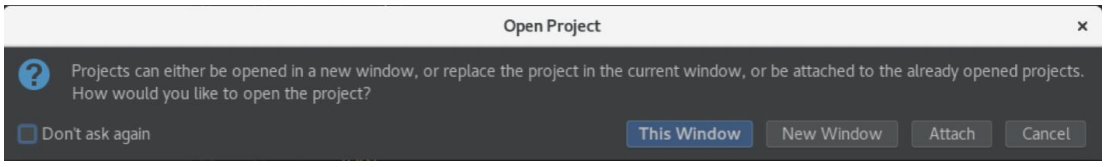
   

   - Use File → Open menu to and browse to `/home/oracle/labs/solutions/lesson3` folder.
   - Click the "OK" button.

   - Click the "This Window" button.

2. Create the `ThoughExperiment` class that extends Experiment.
   a. This code needs to be placed at the end of the class definition section in the `main.js` file.
   b. Create a `ThoughtExperiment` class that extends the Experiment class, with a private property that represents an array of thoughts.

```
// define a ThoughExperiment class that extends the Experiment
class ThoughtExperiment extends Experiment {
  // Add private property that represents an array of thoughts
  #thoughts = [];
  // the rest of the code for this class will be added here
}
```

   c. Inside this class, add a constructor to the `ThoughtExperiment` class that accepts id, task, thought, startTime, endTime and complete parameters. Inside this constructor invoke a superclass constructor, passing a 0 value for a budget. Use thought parameter value to initialize the first element in the thoughts array.

      Note: Most of these parameters are the same as in the Experiment class, which makes sense, because `ThoughExperiment` is a variant (sub-type) of an Experiment, thus should at least have the same properties. However, it may also introduce additional properties -in this case a "thought" property. Also, notice that there is no parameter representing a budget, because a thought experiment is considered to have a zero budget, which can be hardcoded.

```
// initialise Experiment properties via superclass constructor
// set budget as zero
// initialise first thought
constructor(id, task, thought, startTime, endTime, complete) {
  super(id, task, 0, startTime, endTime, complete);
  this.#thoughts[0] = thought;
}
```

   d. Inside this class, add `get` and `set` accessors for the `thoughts` property. The Get operation should simply return an array of thoughts, while set operation should be adding one thought at a time to this array.

```
// add set and get assessors for a thought array
// return an entire array from the get accessor
get thoughts(){
  return this.#thoughts;
}
// append a thought to an entire via set accessor
set thought(thought) {
  this.#thoughts.push(thought);
}
```

e. Inside this class, add the `toString` function that appends thoughts to the output produced by the parent class version of the `toString`. When concatenating thoughts, place each thought on a new line and prefix it with a "-" symbol.

```
// reuse superclass toString function
// concatenate thoughts as a list of lines
toString() {
  let result = super.toString()+"\nThoughts:";
  for (const thought of this.#thoughts) {
    result += "\n - "+thought;
  }
  return  result;
}
```

3. Add code to test ThoughtExperiment capabilities.
   a. Place this code at the end of the test section of the `main.js` file.
   b. Declare a variable to reference additional experiment object, instantiate a new ThoughtExperiment, and assign it to this variable. Supply the following values:
      - id : 103
      - task : "Predict race results"
      - thought: "Predict the winner of any race."
      - startTime : new Date()
      - endTime : null
      - complete : false

```
// Instantiate and use a Thought Experiment object
let experiment3 = new ThoughtExperiment(103,"Predict race results",
       "Predict the winner of any race.", new Date(), null, false);
```

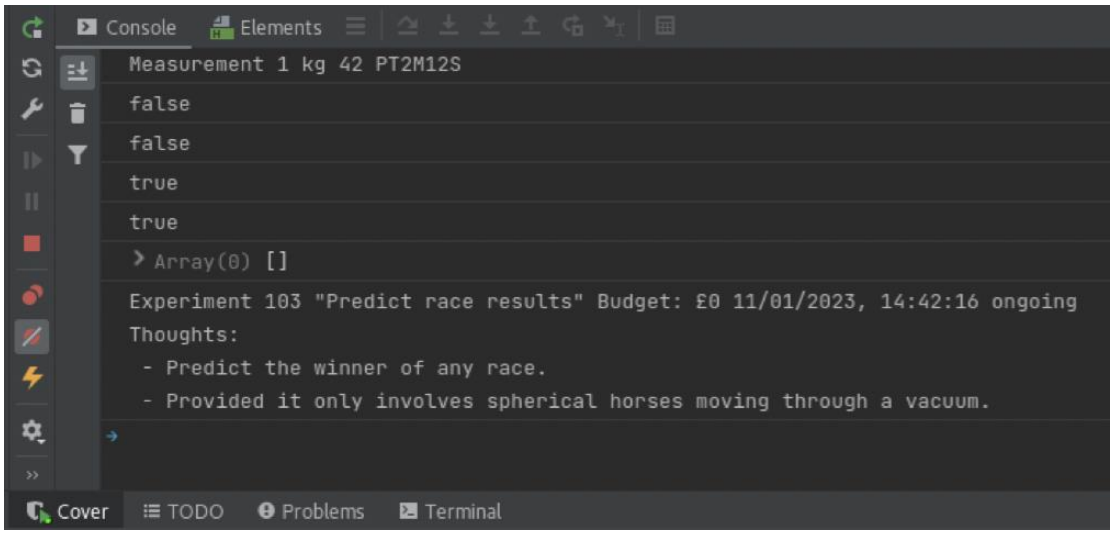
   c. Add one more though to this experiment object.

```
experiment3.thought =
  "Provided it only involves spherical horses moving through a vacuum.";
```

   d. Use the `toString` function to produce text representation of this experiment and print it to the console.

```
console.log(experiment3.toString());
```

4. Test the application logic.
   a. Run the application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.
   b. Observe the JavaScript console output. You should also be able to see the console output produced by the JavaScript code snippet in the console window displayed by WebStorm as well as in the browser's JavaScript console.



5. Alter prototype of the `Experiment` class by adding a function that can find all measurements for this experiment.
   a. Place this code at the end of the class definition section of the `main.js` file, before the object section.
   b. Add the `getMeasurements` function to the Experiment prototype. This function can reuse the existing `getMeasurements` function of the data object. It should use current experiment ID as an argument to locate a set of relevant measurement objects. This set needs to be converted to an array before it is returned from this newly defined function.

```
// Add getMeasurements function to the Experiment prototype
Experiment.prototype.getMeasurements = function() {
  return Array.from(data.getMeasurements(this.id));
}
```

Note: There is no obvious practical reason why such function could not have been directly defined as a part of the Experiment class. However, such dynamic alteration of the prototype could be somewhat justified if you imagine that class definition was designed separately as a part of another program or library, and that the requirement to implement additional functionality has emerged later as part of a separate development effort.

6. Refactor the way in which application locates measurements for a given experiment.
   a. In the `main.js` file, locate lines of code that invoke the `getMeasurements` function upon a data object and replace these with the invocation of the `getMeasurements` function upon corresponding experiment objects.
   b. The first occurrence of this code is in the end of the object section that initializes the measurements array:

```
// Create and populate an array of Measurement objects
// let measurements = [measurement1, measurement2, measurement3];
// let measurements = Array.from(data.getMeasurements(101));
// use getMeasurments function of the Experiment
let measurements = data.getExperiment(101).getMeasurements();
```

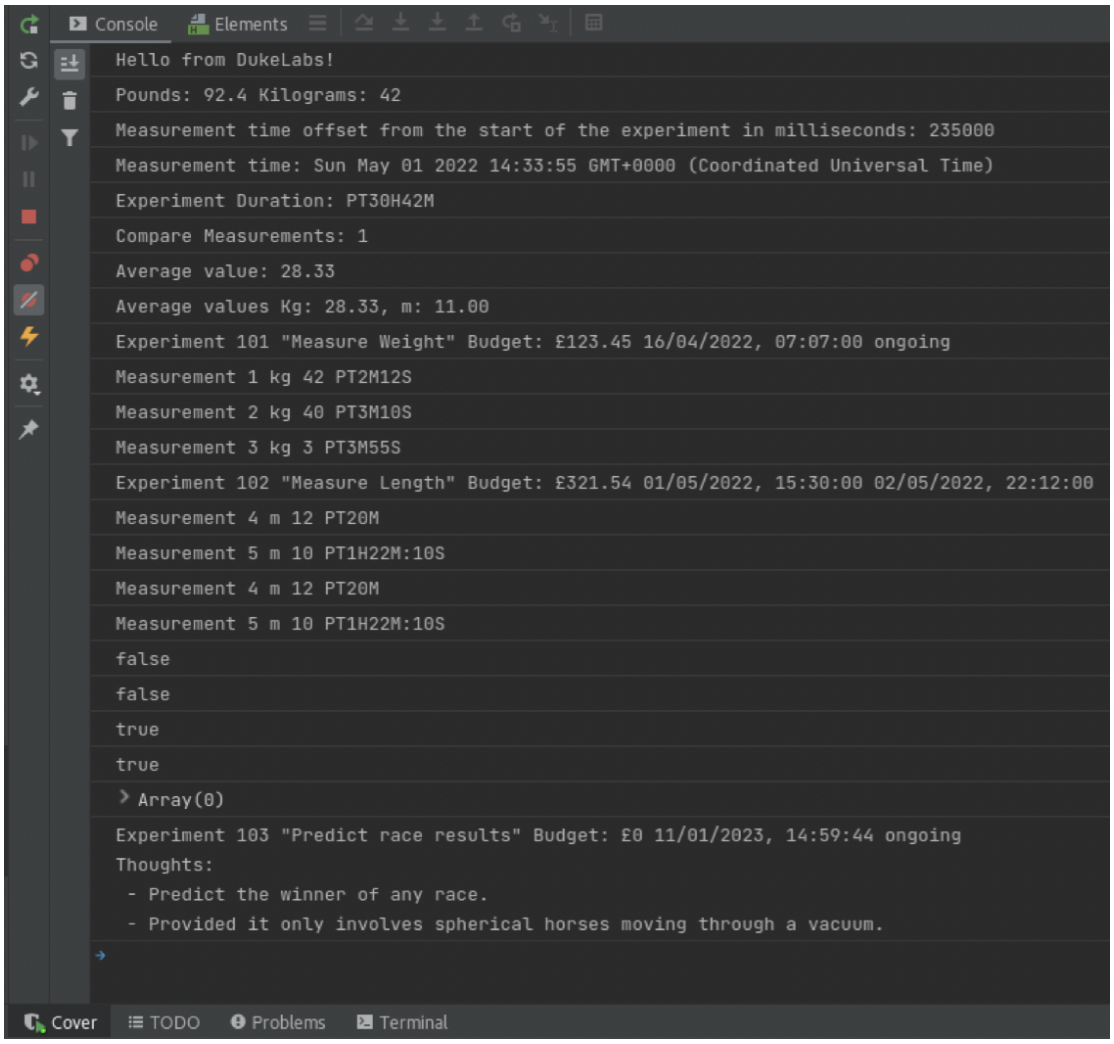
   c. The second occurrence of this code is in the part of the testing section that appends an extra object to the measurements array. In this case, there is an additional opportunity for a code refactoring improvement. Use the array spreading operator, instead of adding an element to the measurements array via the loop with a push function.

```
// Calculate measurements average value version 2
// (per each unit) for all experiments
// for (const measurement of Array.from(data.getMeasurements(102))) {
//     measurements.push(measurement);
// }
// use spread to merge arrays of measurements
// measurements = Array.of(...measurements, ...data.getMeasurements(102));
// use getMeasurments function of the Experiment object
measurements = Array.of(...measurements,
                        ...data.getExperiment(102).getMeasurements());
```

   d. The third occurrence of this code is in the part of the testing section that iterates through the measurements array and print measurements to the console.

```
// for (const measurement of data.getMeasurements(102)) {
//     console.log(measurement.toString());
// }
// use getMeasurments function of the Experiment
for (const measurement of data.getExperiment(102).getMeasurements()) {
    console.log(measurement.toString());
}
```

7. Test the application logic.
   a. Run the application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.
   b. Observe the JavaScript console output. You should also be able to see the console output produced by the JavaScript code snippet in the console window displayed by WebStorm as well as in the browser's JavaScript console.



8. Add `getMeasurements` and `addMeasurement` functions to the `Experiment` class.

   Note: It is generally considered not a good design practice to write code that depends upon dynamically altered prototypes. Also, it makes sense to group functions together with corresponding object to improve code cohesion. In this scenario, it makes sense to incorporate functions that operate on measurements for a given experiment directly into the `Experiment` class.

   a. Place this code inside the `Experiment` class definition.

   b. Add the `getMeasurements` function to the Experiment class. Copy the function definition from the part of code that was dynamically altering the Experiment prototype.

```
getMeasurements() {
    return Array.from(data.getMeasurements(this.id));
}
```

   c. Comment out part of the code that was dynamically altering the Experiment prototype.

```
// Add getMeasurements function to the Experiment prototype
// Experiment.prototype.getMeasurements = function() {
//     return Array.from(data.getMeasurements(this.id));
// }
```

   d. Add the `addMeasurement` function inside the `Experiment` class. Form a new Measurement object using unit and value that are supplied as function parameters and assume current time for when this measurement was taken. Remember that measurement time is recorded as a period of time elapsed from the start of the experiment.

      Hints:

      - Accept unit and value parameters.
      - Calculate time of the measurement by taking current time and subtracting the start time of this experiment.
      - Format time offset milliseconds value as ISO string using the `formatDuration` function.
      - Construct new `Measurement` object with unit, value, and duration parameters.
      - Invoke the `addMeasurement` function upon data object and pass current experiment ID, and the new `Measurement` object as parameters.

```
addMeasurement(unit, value) {
  // calculate time of the measurement
  // as an offset from the start of the experiment
    let duration = formatDuration(new Date().getTime() -
                                  this.#startTime.getTime());
    // create measurement and add it to the experiment
    data.addMeasurement(this.#id, new Measurement(unit, value, duration));
}
```

9. Add code to test the `addMeasurement` and `getMeasurements` functions of the `Experiment` class.

   a. This code should be placed at the end of the test section of the `main.js` file.

b. Invoke the `addMeasurement` function upon the `experiment1` object, supply "kg" as a unit and 3 as a value.

```
// Test addMeasurement and getMeasurements functions of the Experiment
experiment1.addMeasurement("kg",3);
```

Note: Measurement time would be derived automatically based on the assumption that it is an offset from the start of the experiment and the current time.

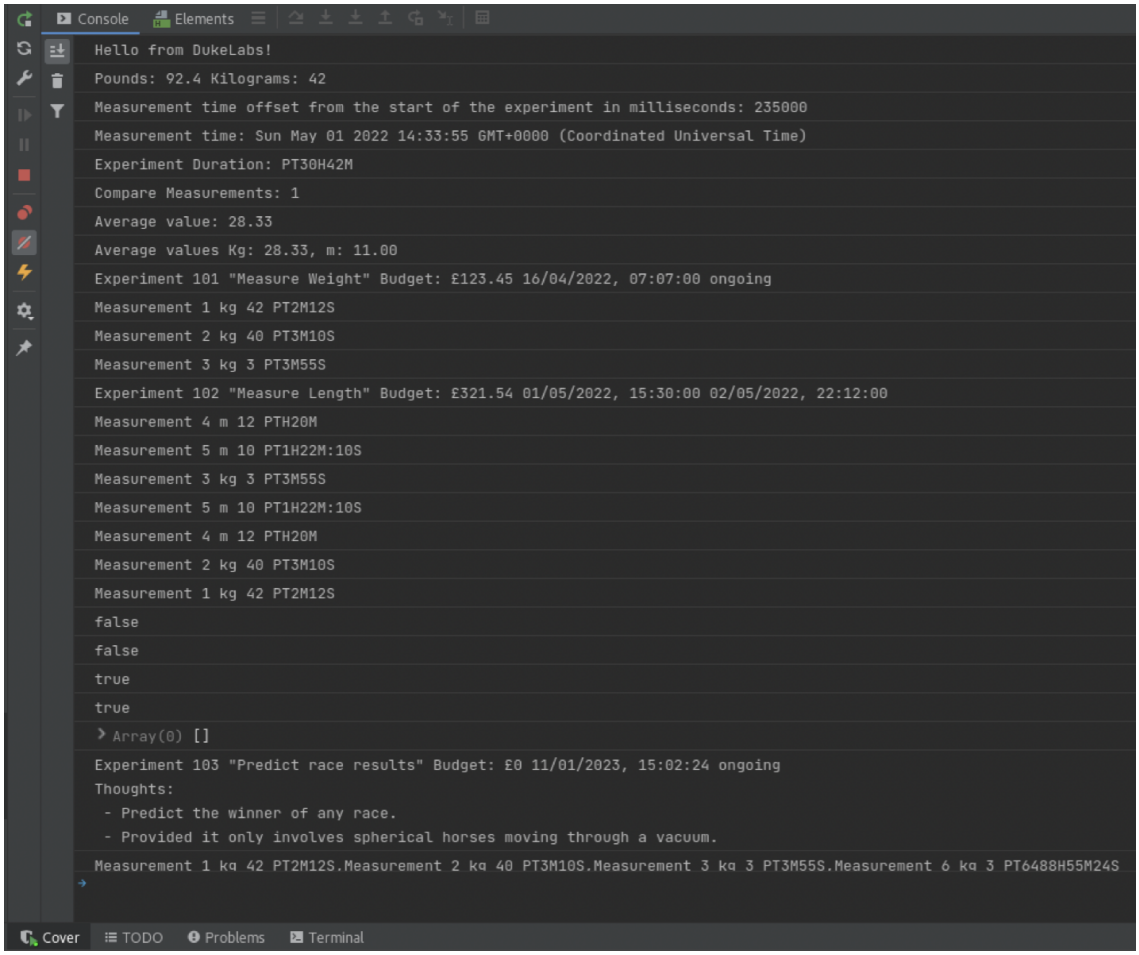
c. Invoke the `getMeasurements` function upon the `experiment1` object to retrieve the array of measurements. Invoke the `toString` operation upon this array and print the result to the console.

```
console.log(experiment1.getMeasurements().toString());
```

10. Test the application logic.

a. Run the application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.

b. Observe the JavaScript console output. You should also be able to see the console output produced by the JavaScript code snippet in the console window displayed by WebStorm as well as in the browser's JavaScript console.



Practices for Lesson 5: JavaScript in a Web Browser Environment

Overview

In these practices, you will:

- Design HTML interface for DukeLabs web application
- Create JavaScript functions to handle events raised by HTML elements of the DukeLabs web application

Practice 5-1: Design HTML Interface

Overview

In this practice, you will design an HTML user-interface for the DukeLabs web application. This should include several fields to display experiment details, a table to display measurements as well as visual representation of the summary information of measurements. You also design a dialog through which users would be able to add measurements to the experiment.

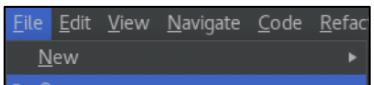
Tasks

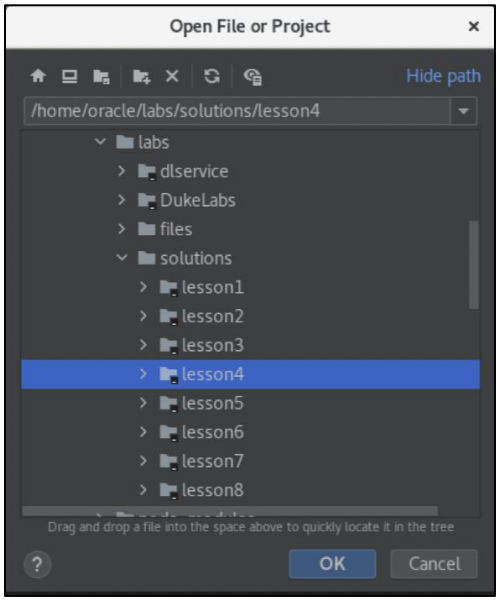
1. If you have not completed previous practice, you may open a solution application for Lesson 4, which is the starting point of the Lesson 5 practice. Otherwise, you may continue to use the same WebStorm project that you have worked on in the previous exercise.

   Hints:

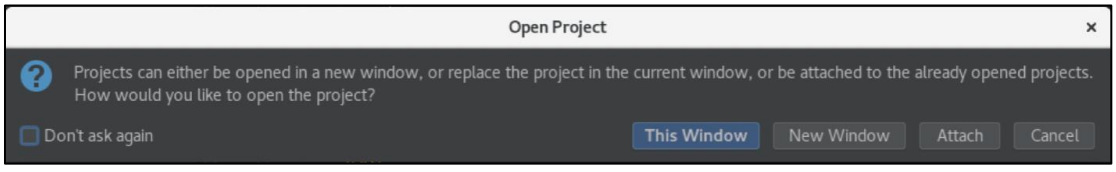
   - Launch WebStorm IDE if it is not already running.

   - Use File → Open menu to and browse to `/home/oracle/labs/solutions/lesson4` folder.
   - Click the "OK" button.





   - Click "This Window" button.



2. Create an HTML form to accommodate fields that represent properties of the experiment.
   a. The experiment form should be placed inside the main section of the `index.html` file, by replacing the phrase, "Experiments and Measurements".
   b. Create a form element inside the main section of the `index.html` page and set this form ID as "exp". Assigning IDs to elements makes it easy to locale these elements from JavaScript functions.

```
<main>
  <form id="exp">
    // form elements will be added here in the next step
  </form>
</main>
```

Note: The form element that surrounds input fields is typically used to enable web page to submit all the form field values to a server-side program. However, in this practice scenario all fields would be handled via JavaScript functions that are executed locally in the context of the same page, which would seem to make the form element redundant. But there is another reason why form element may be useful – all fields inside the form can be reset together with a single function call.

   c. Inside this form add a fieldset and a legend element. Add text "Experiment" inside the legend element. Place a div element inside the fieldset after the legend element. This div will be used as a wrapper for the fields that will be added inside.

```
<form id="exp">
  <fieldset>
    <legend>Experiment</legend>
      <div>
      </div>
      // more fieldsets will be added here in the next step
  </fieldset>
</form>
```

Note: Fieldsets are used to visually group several other elements together, such as in this case, grouping fields of a form. Much like the fieldset, div element are also used to group HTML elements. However, unlike fieldset, div elements are not visually specialized, and generally considered to be a flexible option that groups content together, in a way that allows different visual styling to be applied to this content later. That is, developers can make a div element look like anything they want using CSS, which is why it is so commonly used.

d. Inside this fieldset, after the end of the div element, add two more fieldset elements, one using a ledged "Schedule" and the another "Summary".

```
<fieldset>
  <legend>Experiment</legend>
  <div>
    // form fields will be added here in the next step
  </div>
  <fieldset>
    <legend>Schedule</legend>
  </fieldset>
  <fieldset>
    <legend>Summary</legend>
  </fieldset>
</fieldset>
```

e. Inside the first div element that is inside the experiment field set, add input fields type of text for the id, task, and budget. Mark task and budget fields as disabled. Each form field should be accompanied by a label element that provides a prompt for the corresponding field. Each pair of label and input elements should be grouped together into their own div element. Each input item should be given a unique id, and corresponding label should refer to this id. When assigning id values adopt naming convention where the id value for a field is prefixed with the id of the form to which this field belongs.

```
<div>
  <div>
    <label for="exp_id_field">ID:</label>
    <input type="text" id="exp_id_field">
    // find button will be added here in the next step
  </div>
  <div>
    <label for="exp_task_field">Task:</label>
    <input type="text" id="exp_task_field" disabled>
  </div>
  <div>
    <label for="exp_budget_field">Budget:</label>
    <input type="text" id="exp_budget_field" disabled>
  </div>
</div>
```

Note: All the fields except the id field should be marked as disabled, because the value of the id field would be used as an input to search for a specific experiment. All other fields would then by dynamically populated with relevant properties of the experiment found by the search operation.

f. Inside the div that wraps around the id field and its label, add a submit button with the id "exp_find_button" and the text "Find". This button will later be used allow users to trigger experiment search functionality.

```
<div>
  <label for="exp_id_field">ID:</label>
  <input type="text" id="exp_id_field">
  <input type="submit" id="exp_find_button" value="Find">
</div>
```

Note: This Find button will be later used to execute the experiment search, which will be triggered via a JavaScript `onClick` event. This button is not expected to submit any input fields to the server-side program, because the entire process of searching for the experiment will be fulfilled locally within the JavaScript functions running in the context of this page. This means that it does not have to be a submit button and could be designed as just a simple button. Unlike input type submit, an input type button has not no default functionality, such as submitting a form to a server-side program, which means it does not do anything other than what the JavaScript event handling functions associated with this button are going to do. Nevertheless, in this practice you are asked to make it a submit button. Later in this practice you will be asked to add other types of buttons to the page. This will allow you to explore the assignment of visual properties to different types of elements as part of the practice for the next lesson.

g. Inside the Schedule fieldset, add input fields type of datetime-local to represent the `startTime` and the `endTime` if the experiment. Mark these fields as disabled. Each form field should be accompanied by a label element that provides a prompt for the corresponding field. Each pair of label and input elements should be grouped together into their own div element. Each input item should be given a unique id, and corresponding label should refer to this id. When assigning id values adopt naming convention where the id value for a field is prefixed with the id of the form to which this field belongs.

```
<fieldset>
  <legend>Schedule</legend>
  <div>
    <label for="exp_startTime_field">Start Time:</label>
    <input type="datetime-local" id="exp_startTime_field" disabled>
  </div>
  <div>
    <label for="exp_endTime_field">End Time:</label>
    <input type="datetime-local" id="exp_endTime_field" disabled>
  </div>
  // a checkbox field will be added here in the next step
</fieldset>
```

Note: There is no practical reason why `startTime` and `endTime` fields should be type of datetime-local, because they are intended only to display relevant experiment properties. Different types of input fields correspond to different input value controls, such as a popup calendar that may help users to enter date values, but of course, no such controls are needed if these fields are read-only, so for this reason alone simple text fields would have been sufficient. However, the use of different types of fields would become import in a later practice were you will be asked to apply visual styles to the page and its fields.

h. After the end of the div element that grouped the `endTime` field and its label add another div element to wrap around the field that is going to represent experiment complete status property and the label for it. Mark the check box input item as disabled.

```
<div>
  <label for="exp_complete_field">Complete?</label>
  <input type="checkbox" id="exp_complete_field" disabled>
</div>
```

Note: In a second part of this practice, you will be asked to add functionality to this page that would mark the experiment as complete when this check box will be selected. Of course, this only could happen if a given experiment is still ongoing and has not been marked as complete already. Therefore, this check box should stay disabled for any complete experiment and be dynamically changed to enabled when this form displays an ongoing experiment.

i. Inside a Summary fieldset add a progress field that would later be used to represent an average value of the measurements of this experiment. Its value should be set to be initially 0, and its min and max properties should be set as 0 and 100.

```
<fieldset>
    <legend>Summary</legend>
    <progress id="exp_average_field" min='0' max='100' value='0'>0</progress>
</fieldset>
```

3. Create an HTML table to accommodate rows of data that represent measurements of a given experiment.

a. Inside the main section of the page, after the end of the form element, add a new fieldset using a ledged "Measurements".

```
<fieldset>
  <legend>Measurements</legend>
  // a table element will be added here in the next step
</fieldset>
```

b. Inside this fieldset, after the legend element, add a table element. This table should contain a table header represented by the <thead> element and a table body represented by the <tbody> element. Table header should contain a row of <th> elements each representing a different measurement property, ID, Unit, Value and Time. The content within this table body will be constructed dynamically by JavaScript function what will display measurements. Assign an id to the table body to make it easier to locale from the JavaScript function.

```
<fieldset>
  <legend>Measurements</legend>
  <table>
    <thead>
      <tr><th>ID</th><th>Unit</th><th>Value</th><th>Time</th></tr>
    </thead>
    <tbody id="mea_table">
    </tbody>
  </table>
  // an add measurement button will be added here in the next step
</fieldset>
```

c. After the end of the table, add an "Add Measurement" button. This button should also have an id assigned to it, and it should be disabled by default.

```
<input type="button" value="Add Measurement" id="mea_add_button" disabled>
```

Note: The Add Measurement button will be later used to trigger the appearance of the add measurement dialog. This button is not expected to submit any input fields and is not part of a form, thus its type should not be submit, but just a button. Which means it does not have any default functionality other than what the JavaScript event handling functions associated with this button are going to do. The reason why this button is disabled by default is because it should only be enabled when this page displays an experiment that is still on-going. Because you are not supposed to be able to add measurements to the experiment that is marked as complete.

4. Create a dialog element that will be used to display a form to add a new measurement.

a. Place this dialog element after the end of the footer element of this page.

Note: This dialog is intended to be displayed as a modal dialog that will be placed on top of other content in this page, so its precise location within the document body is not that important.

b. Create a dialog element associated with a unique id. Place a form element with id "mea" inside this dialog.

```
<dialog id="mea_add_dialog">
  <form id="mea">
  // a fieldset will be added here in the next step
  </form>
</dialog>
```

c. Inside the form, add a fieldset with the Add Measurements legend element inside it.

```
<fieldset>
  <legend>Add Measurement</legend>
  // form fields will be placed here in next steps
</fieldset>
```

d. Inside the fieldset, after the legend element, add a select element that displays a list of SI unit of measure as the select options. Each unit of measure should have a value of the SI unit abbreviation and use the full unit's name as option text. Assign unique identity to the select element.

```
<select id="mea_unit_input">
  <option value="s">Second</option>
  <option value="m">Meter</option>
  <option value="kg">Kilogram</option>
  <option value="A">Ampere</option>
  <option value="K">Kelvin</option>
  <option value="mol">Mole</option>
  <option value="cd">Candela</option>
</select>
```

e. Next, after the end of the select element, add a text input field for the value property of the measurement, assign a unique identity to this field.

```
<input id="mea_value_input" type="text">
```

f. Next, add a submit button that with the prompt Ok and assign a unique identity to this it.

```
<input type="submit" value="Ok" id="mea_ok_button">
```

g. Lastly, add a reset button that with the prompt Cancel and assign a unique identity to it.

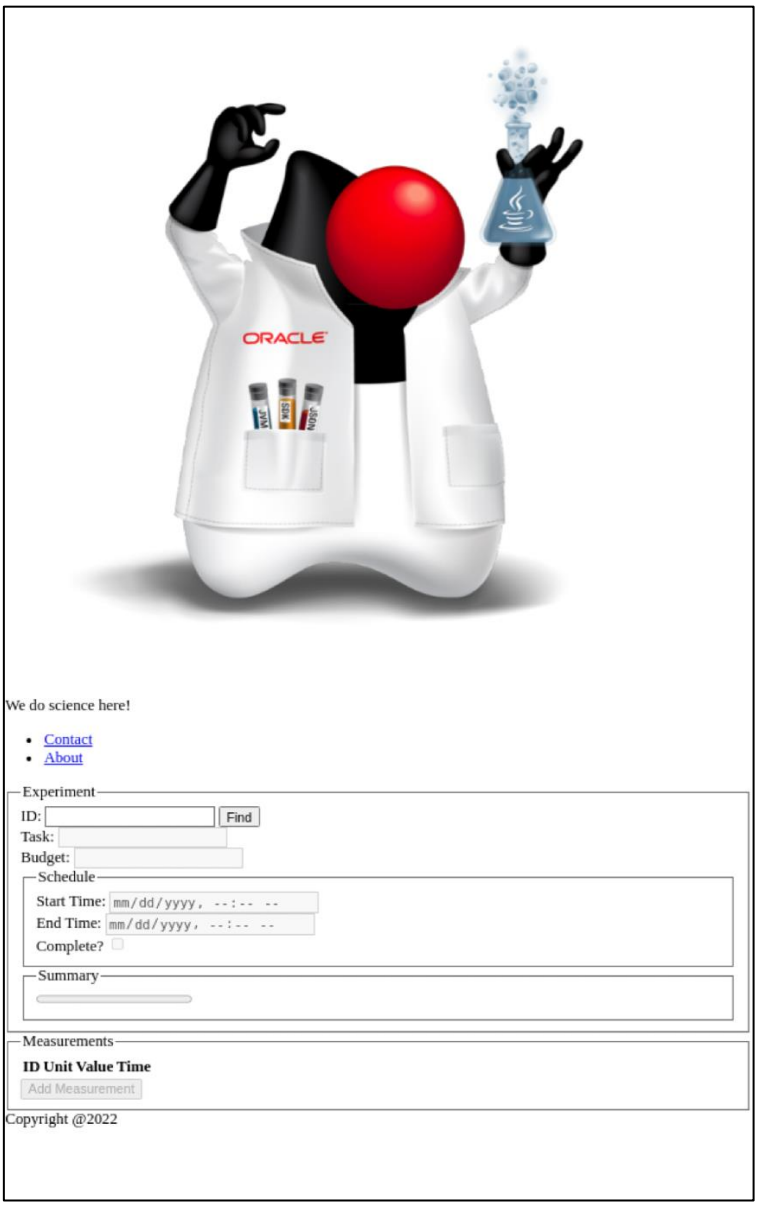
```
<input type="reset" value="Cancel" id="mea_reset_button">
```

5. Test the application html layout design.

Note: It is only possible to observe static html design at this stage of the practice, because the application does not have any event handlers. Creating event handlers is the goal of the second part of the practice. Some of the design elements are going to be disabled, or not visible, because they are intended to be enabled and visualized because of user actions.

a. Run the application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.

b. Observe the HTML page design in the browser.



Practice 5-2: Create Event Handlers

Overview

In this practice, you will write JavaScript functions to handle events produced by the DukeLabs web application. This includes the following logic:

- Find and display an experiment as well as its measurements.
- Compute and visualize summary measurements average value.
- Complete the experiment.
- Display the add measurement dialog and handle the logic of adding a new measurement to the experiment.

Tasks

1. Comment out the Test Section of the `main.js` file.
   a. Open the `main.js` file and locate the section that starts from the /* Test Section */ comment.
   b. Select all code in this section, and press CTRL+/ keys.
2. Organize UI event handling functions.
   a. Create a new section within the `main.js` file to accommodate all functions that would interact with UI elements and events. This could be placed at the end of the `main.js` file.

```
/* --- UI Events and Functions Section --- */
```

b. In this section, create the following functions:
   - The `findExperiment` function that will be searching for an experiment object in response to the find button click event. It should accept event object as an argument.
   - The `displayMeasurements` function that will be displaying measurements for a given experiment. It should accept experiment object as an argument.
   - The `completeExperiment` function that will be updating experiment state to complete in response to the check box change event. It should accept event object as an argument.
   - The `showAddMeasurement` function that will be displaying the add measurement dialog in response to the add measurement button click event. It should accept event object as an argument.
   - The `addMeasurement` function that should process measurement data to add a new measurement to the experiment in response to the ok button click event. It should accept event object as an argument.
   - The `canelMeasurement` function that should dismiss the add measurement dialog in response to the cancel button click event. It should accept event object as an argument.
   - The `resetMeasurementsTable` function that should remove all row displaying data from the measurement table. This function could be triggered when new experiment is found, and a new array of measurements needs to be displayed. Also, this function could be triggered if no experiment is found, and thus no data should be displayed.
   - The `resetExperimentForm` function that should reset all fields of the experiment form. This function could be triggered when new experiment is found and this form, needs to display new values. Also, this function could be triggered if no experiment is found, and thus no data should be displayed.

```
/* --- UI Events and Functions Section --- */
function findExperiment(event) {
}
function displayMeasurements(experiment) {
}
function completeExperiment(event) {
}
function showAddMeasurementDialog(event) {
}
function addMeasurement(event) {
}
function cancelAddMeasurement(event) {
}
function resetMeasurementsTable() {
}
function resetExperimentForm() {
}
```

c. Create an event listener function that is triggered when the browser window loads the document. This function should be associated with the window load event and accept event object as an argument. You may use simple arrow syntax.

```
window.addEventListener("load", (event) => {
  // inside this function you will locate page elements
  // and associate these elements with event handing functions
  // in the next step of the practice
});
```

d. Inside the window load event function, locate the following page elements using their ids. Element ids must match exact values associated with corresponding component that were added to this page in the first part of this practice. Find references for the following elements: experiment find button, experiment complete check box, measurement add button, add measurement dialog submit, and reset buttons.

Note: Giving variables in the JavaScript program names that match corresponding element ids from the HTML document could improve code readability.

```
let exp_find_button = document.getElementById("exp_find_button");
let exp_complete_field = document.getElementById("exp_complete_field");
let mea_add_button = document.getElementById("mea_add_button");
let mea_ok_button = document.getElementById("mea_ok_button");
let mea_reset_button = document.getElementById("mea_reset_button");
```

e. Associate page elements with event handling functions.

- The `findExperiment` function should be associated with the click event of the experiment find button.
- The `completeExperiment` function should be associated with the change event of the experiment complete check box.
- The `showAddMeasurementDialog` function should be associated with the click event of the measurement add button.
- The `addMeasurement` function should be associated with the click event of the add measurement dialog submit button.
- The `cancelAddMeasurement` function should be associated with the click event of the add measurement dialog reset button.

```
exp_find_button.addEventListener("click", findExperiment);
exp_complete_field.addEventListener("change",completeExperiment);
mea_add_button.addEventListener("click",showAddMeasurementDialog);
mea_ok_button.addEventListener("click",addMeasurement);
mea_reset_button.addEventListener("click",cancelAddMeasurement);
```

Note: At this stage of the practice, all UI event handling functions were added, but they still lack actual logic. You will be adding this logic in the next step of this practice.

3. Add implementation logic to the `findExperiment` function.

   a. Inside the `findExperiment` function, add code that prevents default event behavior.

      Note: The reason why you need to prevent default event action is because this function is associated with the click event on the submit button of the form, which by default would try to submit all form fields to the server-side handling program and refresh the page. However, this form is not actually intended to be submitted to a server because all fields are handled by JavaScript functions on the client, and the event should not cause the page to reload.

```
function findExperiment(event) {
  event.preventDefault();
  // the rest of this function logic will be added here
}
```

   b. Next, inside the `findExperiment` function, locate all fields of the experiment form (id, task, budget, `startTime`, `endTime`, complete), as well as the add measurement button. Use relevant element ids to retrieve element references from the document.

      Note: You need these field references to be able to set their values when experiment will be found, and you need a button to be able to enable or disable it depending on if experiment is found and if this experiment is complete or is it still on going.

```
// retrieve target elements from the document
let exp_id_field = document.getElementById("exp_id_field");
let exp_task_field = document.getElementById("exp_task_field");
let exp_budget_field = document.getElementById("exp_budget_field");
let exp_startTime_field = document.getElementById("exp_startTime_field");
let exp_endTime_field = document.getElementById("exp_endTime_field");
let exp_complete_field = document.getElementById("exp_complete_field");
let mea_add_button = document.getElementById("mea_add_button");
```

   c. Next, retrieve the value of the experiment id field, parse this value as integer, and use it to locate an experiment via the `getExperiment` function of the data object.

      Note: All HTML input fields present their values as text, regardless of the element type. So, it does not matter if this is an input type number, text, data etc... , regardless the value property will always present a string value. It may be a good idea to parse and validate user input values first, before using them in any processing.

```
// find Experiment
let exp_id = parseInt(exp_id_field.value);
let experiment = data.getExperiment(exp_id);
```

   d. Create and if/else construct that checks if the experiment object is not found (undefined). If this is the case, display an alert warning user that the experiment with a given id is not found, and reset all the fields of the experiment from as well as the measurements table.

```
if (experiment == undefined) {
  alert("Experiment with id "+exp_id+" not found");
  resetExperimentForm();
  resetMeasurementsTable();
}else{
  // logic that process experiment properties when it is found
  // will be added here in the next
}
```

e. Inside the else clause, assign experiment properties to the corresponding fields in this from.
   - The task property should be assigned as is.
   - The budget property should be formatted as currency before it is assigned.
   - The `startTime` property should be formatted as an ISO String value, and last character of the ISO date format needs to be removed from the formatted date-time value, before it is assigned to a corresponding form field.

   Note: In this practice, `startTime` and `endTime` form elements are input type datetime-local rather than simple text. However, HTML documents always expect all input item values to be presented as text regardless of the element type. In this case, this means that HTML datetime-local fields expect an ISO formatted date and time values to be supplied. Unfortunately, the interpretation of what the ISO date-time format is differs between JavaScript functions and relevant HTML input items.

   The default format of the ISO date-time values produced by the JavaScript `toISOString` function has an extra 'Z' character at the end of the value, which is not expected by the datetime-local input field. There is no practical reason for why this is the case, except poor design and implementation of the JavaScript – HTML compatibility.

```
exp_task_field.value = experiment.task;
exp_budget_field.value = formatCurrency(experiment.budget);
exp_startTime_field.value = experiment.startTime.toISOString().slice(0, -1);
// more logic will be added inside this else clause next
```

f. Continue adding logic inside the `else` clause, by adding another `if/else` statement inside. This new `if` condition should check the value of the complete property of the experiment. If experiment complete property is true, then set the corresponding check box field checked property to true, and set the value of the field that represents the `endTime` of the experiment. Also, make sure that complete experiment check box and add measurement button are disabled.

   Note: Once experiment is complete its status should not be reversed back. Also, it does not make sense to add measurements to a complete experiment.

```
if (experiment.complete) {
  exp_complete_field.checked = true;
  exp_endTime_field.value = experiment.endTime.toISOString().slice(0, -1);
  exp_complete_field.disabled = true;
  mea_add_button.disabled = true;
}else{
  // more logic will be added inside this else clause next
}
```

g. In the `else` block that corresponds to the incomplete experiment, set complete check box checked property to false, the `endTime` field value to be an empty string, enable complete experiment check box, and enable the add measurement button.

```
exp_complete_field.checked = false;
exp_endTime_field.value = "";
exp_complete_field.disabled = false;
mea_add_button.disabled = false;
```

h. After the end of the inner `else` block, just before the end of the outer `else` block, invoke an operation that displays measurements. Pass experiment object as an argument.

```
displayMeasurements(experiment);
```

4. Add the implementation logic to the `displayMeasurements` function

a. Inside the `displayMeasurements` function invoke the `resetMeasurementsTable` function, to make sure there are no measurement records displayed in this table, before you will attempt to add new rows to it.

```
function displayMeasurements(experiment) {
  resetMeasurementsTable();
  // more logic will be added here next
}
```

b. Retrieve an array of measurements for the given experiment object, calculate average measurement value, reserve `max` and `min` variables that will be used later to arrange visual presentation of the average value. The `max` and `min` variables should be initialized as `Number.MIN_VALUE` and `Number.MAX_VALUE`.

```
let measurements = experiment.getMeasurements();
let average = calculateAverageMeasurement(measurements);
let max = Number.MIN_VALUE;
let min = Number.MAX_VALUE;
```

c. Create an `if/else` construct that checks if the measurement length is 0, and if this is the case assign zero to the average property.

Note: The `calculateAgerageMeasurement` function returns a NaN value if measurements array has no elements. An alternative solution could be to design this function to throw an exception instead and handle its response with the `try/catch` construct instead of the `if/else`.

```
if (measurements.length == 0) {
  average = 0;
} else {
  // more logic will be added here next
}
```

d. Inside the `else` clause, retrieve a reference to the measurements table from the HTML document. Add a for loop that iterates through all elements in the measurements array.

```
let mea_table = document.getElementById("mea_table");
for (const measurement of measurements) {
  // more logic will be added here next
}
```

e. Inside the for loop, compute the `max` and `min` measurement values.

Hint: Use `Math.max` and `Math.min` functions to compare `max` and `min` variables with the current measurement value. The `max` function returns the biggest of its arguments, and the `min` function returns the smallest. Assigning biggest and smallest values to the `max` and `min` variables inside this loop would produce the required `min` and `max` values when this loop completes.

```
max = Math.max(max, measurement.value);
min = Math.min(min, measurement.value);
// more logic will be added here next
```

f. Next in this loop, create a table cell (td) element for each measurement object property (id, unit, value and time). Also, create a text node object for each measurement property and append each text node as a child node of a corresponding td element.

```
let id_cell = document.createElement("td");
id_cell.appendChild(document.createTextNode(measurement.id));
let unit_cell = document.createElement("td");
unit_cell.appendChild(document.createTextNode(measurement.unit));
let value_cell = document.createElement("td");
value_cell.appendChild(document.createTextNode(measurement.value));
let time_cell = document.createElement("td");
time_cell.appendChild(document.createTextNode(measurement.time));
```

g. Next in this loop, create a table row (tr) element, append all td elements to this row, and finally append the table row element to the table.

```
let mea_row = document.createElement("tr");
mea_row.appendChild(id_cell);
mea_row.appendChild(unit_cell);
mea_row.appendChild(value_cell);
mea_row.appendChild(time_cell);
mea_table.appendChild(mea_row);
```

h. After the end of the for loop, add logic that calculates values needed to visually represent the average value of the measurements. The average value of the experiment could be any number however big or small it happens to be. However, to graphically represent a value it should be scaled to fit into visual boundaries of the UI element that is going to display it. Assume that the average value needs to be scaled to be in the range between 0 and 100. Therefore, you need to calculate which value between 0 and 100 has the same ratio as the average measurement value between the min and max measurement values.

Hints:

- Subtract min measurement from the average
- Multiply the result by 100
- Divide the result by the difference between max and min measurement values

```
let averageRatio = ((average-min)*100)/(max-min);
```

i. Retrieve the experiment average field from the document and assign the calculated average ratio value to this field.

Note: The experiment average field is designed as a progress element and its min and max values are already set to be 0 and 100.

```
let exp_average_field = document.getElementById("exp_average_field");
exp_average_field.value = averageRatio;
```

5. Add implementation logic to the `completeExperiment` function

a. Inside the `completeExperiment` function, retrieve the following elements from the HTML document: experiment id and complete fields, as well as the add measurement button.

```
function completeExperiment(event) {
  let exp_id_field = document.getElementById("exp_id_field");
  let exp_complete_field = document.getElementById("exp_complete_field");
  let mea_add_button = document.getElementById("mea_add_button");
  // more logic will be added here next
}
```

b. Next, retrieve the value of the experiment id field, parse this value as integer, and use it to locate an experiment via the `getExperiment` function of the data object.

```
let exp_id = parseInt(exp_id_field.value);
let experiment = data.getExperiment(exp_id);
// more logic will be added here next
```

c. Next, produce a new Date object, discard its millisecond precision, and assign this date to the complete accessor property of the experiment object.

```
let date = new Date();
date.setMilliseconds(0);
experiment.complete = date;
// more logic will be added here next
```

d. Next, set properties of document elements:

- Set the complete check box field check property to be true.
- Convert the value of the experiment end time property to an ISO String, discard last character and assign this value to the `endTime` field.
- Disable the complete experiment check box and add measurement buttons.

```
exp_complete_field.checked = true;
exp_endTime_field.value = experiment.endTime.toISOString().slice(0, -1);
exp_complete_field.disabled = true;
mea_add_button.disabled = true;
```

6. Add implementation logic to the `showAddMeasurementDialog` function.

a. Inside the `showAddMeasurementDialog` function, retrieve the add measurement dialog element from the HTML document.

```
function showAddMeasurementDialog(event) {
  let mea_add_dialog = document.getElementById("mea_add_dialog");
  // more logic will be added here next
}
```

b. Invoke the `showModal` function upon the dialog object.

```
mea_add_dialog.showModal();
```

7. Add implementation logic to the `addMeasurement` function

   a. Inside the `addMeasurement` function, add code that prevents default event behavior.

```
function addMeasurement(event) {
  event.preventDefault();
  // more logic will be added here next
}
```

   b. Retrieve experiment id, measurement unit and value input fields from the HTML document.

```
let exp_id_field = document.getElementById("exp_id_field");
let mea_unit_input = document.getElementById("mea_unit_input");
let mea_value_input = document.getElementById("mea_value_input");
```

   c. Next, retrieve the value of the experiment id field, parse this value as integer, and use it to locate an experiment via the `getExperiment` function of the data object.

```
let exp_id = parseInt(exp_id_field.value);
let experiment = data.getExperiment(exp_id);
```

   d. Invoke the `addMeasurement` function of the experiment object, and pass values of measurement unit and measurement value input fields to this function as arguments.

```
experiment.addMeasurement(mea_unit_input.value, mea_value_input.value);
```

   e. Retrieve add measurement dialog and add measurement form elements from the HTML document.

```
let mea_add_dialog = document.getElementById("mea_add_dialog");
let mea_form = document.getElementById("mea");
```

   f. Reset the add measurement form and close the dialog box.

```
mea_form.reset();
mea_add_dialog.close();
```

   g. Invoke the `displayMeasurements` operation, and pass experiment object as an argument.

```
displayMeasurements(experiment);
```

8. Add implementation logic to the `cancelAddMeasurement` function

   a. Inside the `cancelAddMeasurement` function, add code that retrieves the add measurement dialog element from the HTML document.

```
function cancelAddMeasurement(event) {
  let mea_add_dialog = document.getElementById("mea_add_dialog");
  // more logic will be added here next
}
```

   b. Close the add measurement dialog.

```
mea_add_dialog.close();
```

   Note: This function is handling the click event associated with the input type reset button element of the add measurement from. The default event behavior for the reset button is to reset all form fields. Thus, there is no need to explicitly invoke rest function on this from.

9. Add implementation logic to the `resetMeasurementsTable` function.

   a. Inside the `resetMeasurementsTable` function, add code that retrieves the measurement table body element from the HTML document.

```
function resetMeasurementsTable(){
  let mea_table = document.getElementById("mea_table");
  // more logic will be added here next
}
```

   b. Retrieve all child elements from this table body (these are the table rows that display measurements). Iterate through all of these rows and remove each of them from the table.

```
while (mea_table.lastElementChild) {
  mea_table.removeChild(mea_table.lastElementChild);
}
```

10. Add implementation logic to the `resetExperimentForm` function

   a. Inside the `resetExperimentForm` function, add code that retrieves the experiment form, add measurement button, and experiment average field elements from the HTML document.

```
function resetExperimentForm() {
  let exp_form = document.getElementById("exp");
  let mea_add_button = document.getElementById("mea_add_button");
  let exp_average_field = document.getElementById("exp_average_field");
  // more logic will be added here next
}
```

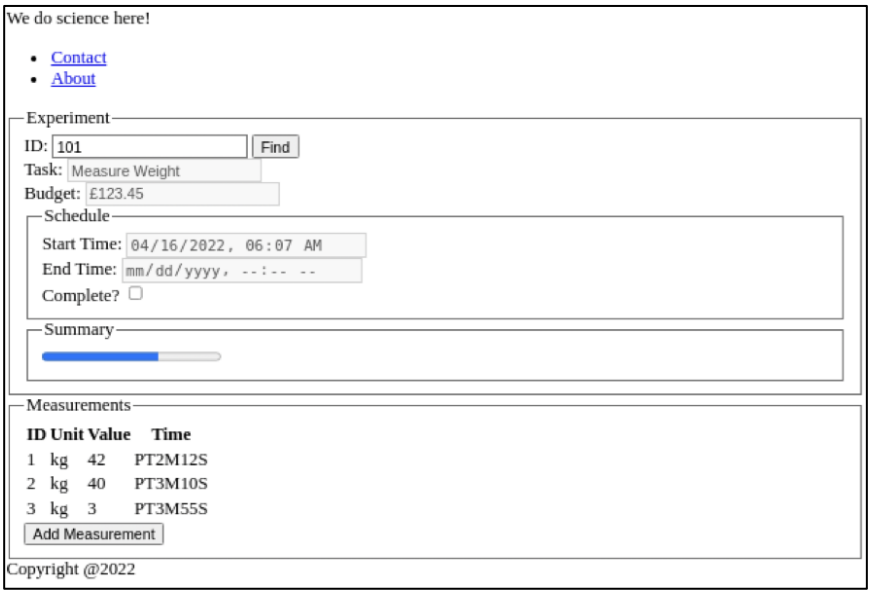
   b. Reset experiment form, set average field value to 0, and disable the add measurement button.

```
exp_form.reset();
exp_average_field.value = 0;
mea_add_button.disabled = true;
```

   Note: Progress element is not actually a form input field as thus is not automatically reset when form reset operations is triggered.

11. Test DukeLabs application.
    a. Run the application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.
    b. Query experiment 101.

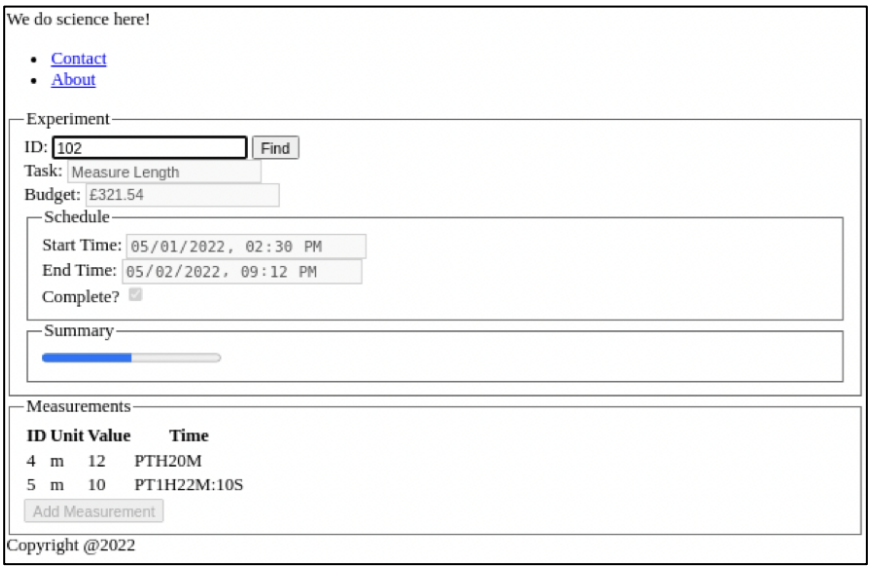
    Hints: On the `index.html` page, enter value 101 into the experiment id field and click the find button.



    Note: You should be able to see the experiment 101, and all its measurements. This experiment is still ongoing. It has no end date, and the complete check box is not selected. Also, notice that add measurement button should be enabled.

c. Query experiment 102.

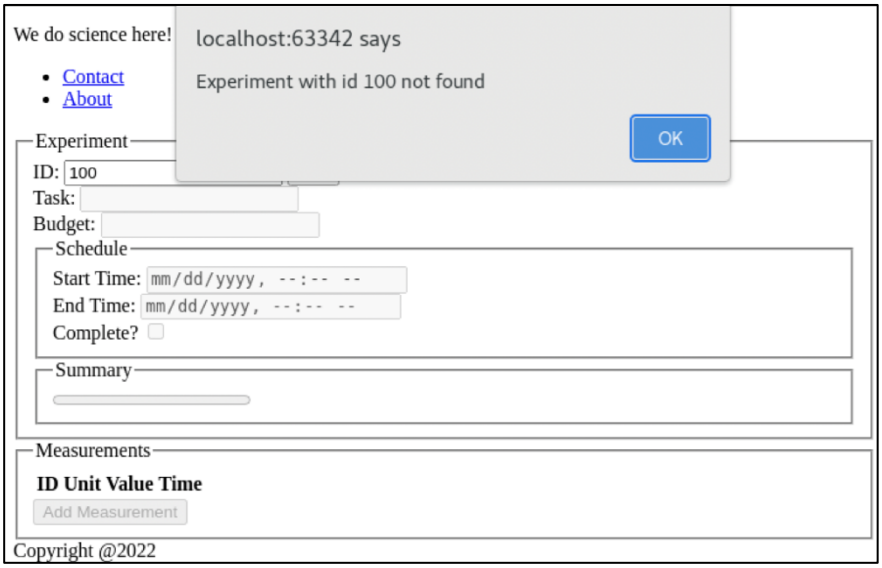
Hints: On the `index.html` page, enter value 102 into the experiment id field and click the find button.



Note: You should be able to see the experiment 102 and all its measurements. This experiment is complete. It has an end date, and the complete check box is selected. Also, notice that add measurement button should be disabled.

d. Produce experiment not found error.

Hints: On the `index.html` page, enter the value that does not match any experiment id, for example 100 into the experiment id field and click the find button.

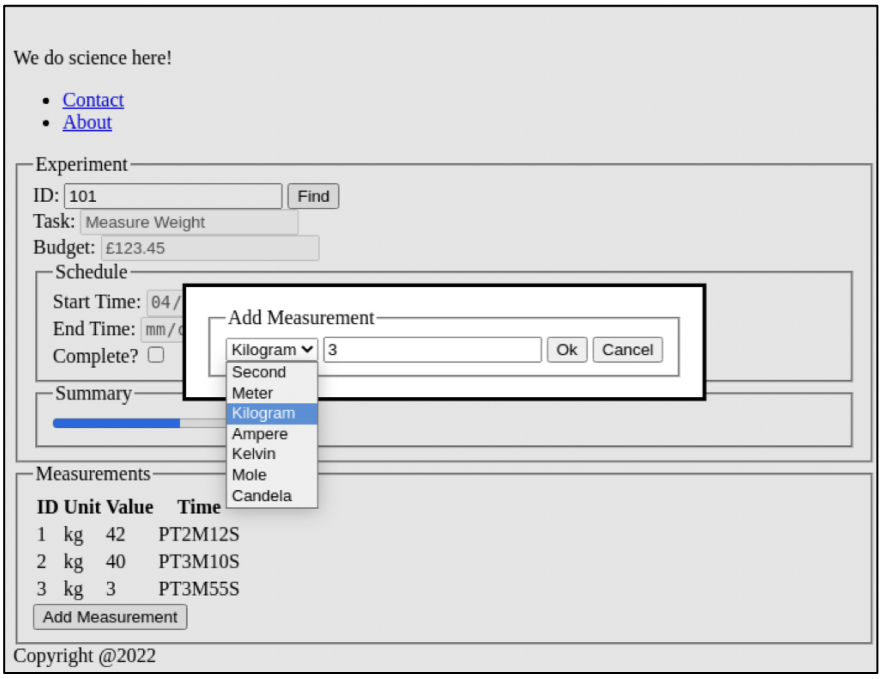


Note: You should be able to see the alert box indicating that experiment with this id was not found and all the content of the page should be reset.

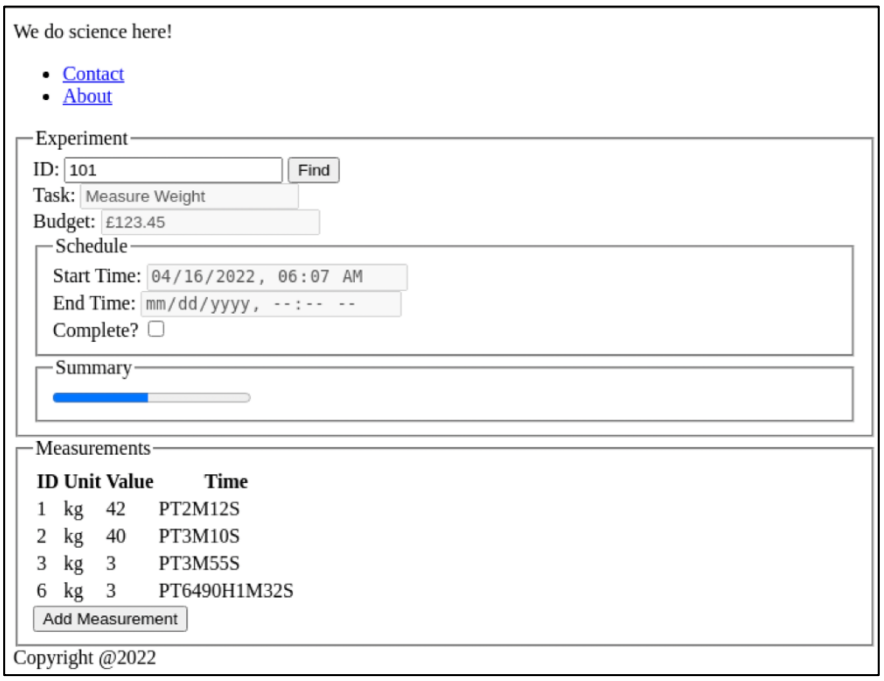
e. Query experiment 101 again.

f. Add a new measurement to this experiment.

Hints: Click the Add Measurement button. The Add Measurement dialog box should appear. Select unit of measure form the drop-down list and enter a value into the input field. For example, unit is kilogram and value is 3.



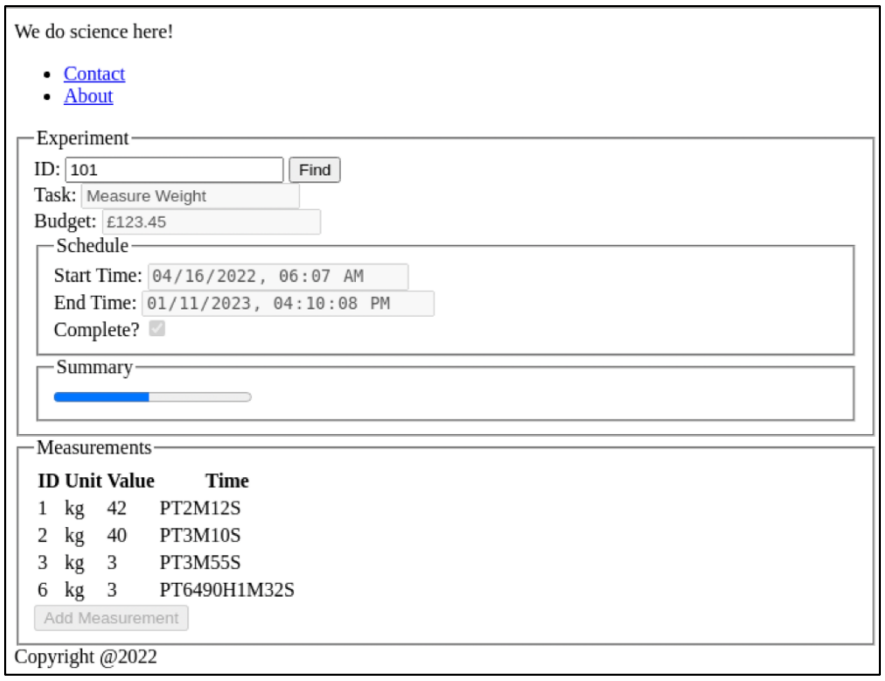
g. Click OK button to submit these values and dismiss the Add Measurement dialog box.



Note: You should be able to see this measurement added as a last row in the measurements table. Also notice the change of the average value display in the summary section when you add a measurement. You may add more measurements to this experiment if you wish.

h. Complete experiment 101.

Hints: Select the complete experiment check box.



Note: This experiment should now change its state to complete. It now has the end date, and the complete check box is selected. Also, notice that add measurement button should be disabled.

Note: If you want to repeat any of the tests, simply reload the `index.html` page. This would reset all experiments and measurements to the starting state, and all the changes you've made would be discarded. This is because all experiments and measurements data is currently only held in memory and is reinitialized from scratch every time the `index.html` is loaded.

Practices for Lesson 6:
Cascading Style Sheets

Overview

In these practices, you will:

- Design CSS document to define styles for the DukeLabs web application
- Apply CSS to html pages of the DukeLabs web application

Practice 6-1: Design and Apply Styles

Overview

In this practice, you will design several styles in the CSS document and apply these styles to HTML pages of the DukeLabs application. You will be instructed to create different types of selectors, referencing element names, element ids, class names, and use pseudo selectors. You will write JavaScript logic to dynamically alter an element style and produce animation effects.

Also, in this practice you will explore different approaches to html element visualizations. Some HTML elements, such as a fieldset, for example, do have distinct default appearances, which of course you may alter. Yet, other elements, such as a div, header, footer and so on do not have a distinct default appearance, which means that you can make them look like anything you need in different cases. Practically, this means that you can make a div element look like a fieldset, or a button, or progress bar, or anything else that you may require. Both approaches, altering default appearance and designing appearance from scratch, will be use in this practice.

Tasks

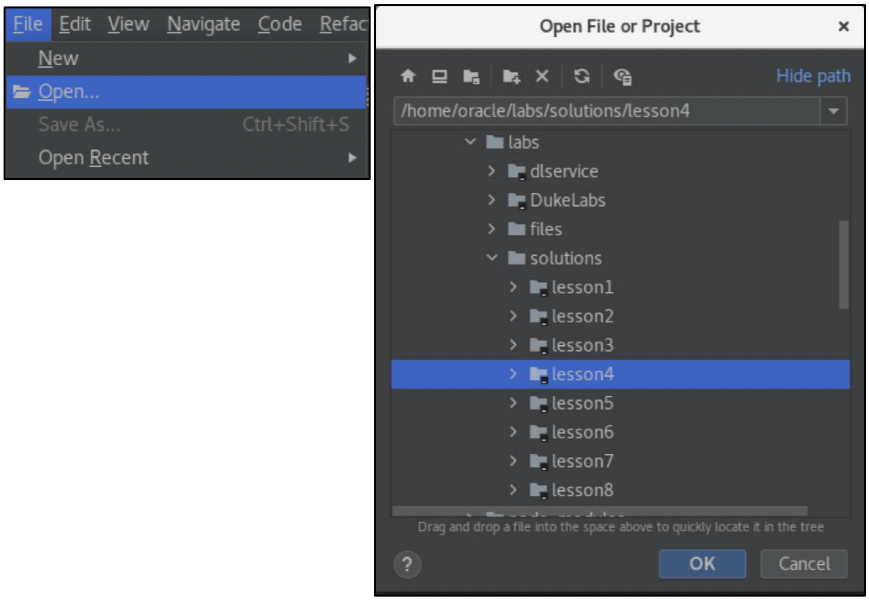
1. If you have not completed previous practice, you may open a solution application for Lesson 5, which is the starting point of the Lesson 6 practice. Otherwise, you may continue to use same WebStorm project that you have worked on in the previous exercise.

   Hints:

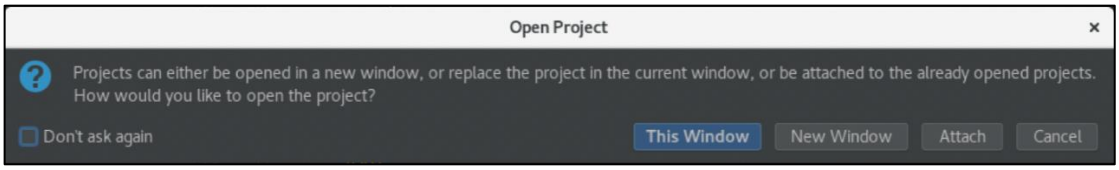
   - Launch WebStorm IDE if it is not already running.

   - Use File -> Open menu to and browse to `/home/oracle/labs/solutions/lesson5` folder.
   - Click the "OK" button.

– Click the "This Window" button.



2. Modify `index.html` page, by assigning visual classes to pages elements, and alter the design of the measurement average display.

   a. Open `index.html` file and locate progress element for the experiment average field and comment out this element.

      Hint: Select relevant lines of code and press CTRL+/ .

```
<!--<progress id="exp_average_field" min='0' max='100'value='0'>0
</progress>-->
```

   b. Create a new div element, assign it an id of "exp_average_box", add another div element inside the first div, and give that an id of "exp_average_value".

```
<div id="exp_average_box">
  <div id="exp_average_value"></div>
</div>
```

   Note: In a later stage of this practice, the outer div element style will feature a border, and the inner div element will be filled with color and sized to be proportional to the average ratio value. Also, you will display the actual average value inside the inner div element. Thus, this element will behave somewhat like a progress element, but you would have much more control over its appearance.

   c. Add style class names to the image and the slogan text located in the page header section.

      Hint: Assign "img_slogan" class name to the img element, and "slogan" to the div element that contains slogan text.

```
<img class="img_slogan" src="resources/duke.png">
<div class="slogan">We do science here!</div>
```

      Note: Actual styles for all the class names applied in this section of the practice will be defined in the `style.css` document in a later stage of this practice.

d. Inside an experiment form (`<form id="exp">`) locate all div elements that contain label and input elements inside and add style class "field" to every one of these div elements.

```
<div class="field">
  <label for="exp_id_field">ID:</label>
  <input type="text" id="exp_id_field">
  <input type="submit" id="exp_find_button" value="Find">
</div>
<div class="field">
  <label for="exp_task_field">Task:</label>
  <input type="text" id="exp_task_field" disabled>
</div>
<div class="field">
  <label for="exp_budget_field">Budget:</label>
  <input type="text" id="exp_budget_field" disabled>
</div>


<div class="field">
  <label for="exp_startTime_field">Start Time:</label>
  <input type="datetime-local" id="exp_startTime_field" disabled>
</div>
<div class="field">
  <label for="exp_endTime_field">End Time:</label>
  <input type="datetime-local" id="exp_endTime_field" disabled>
</div>
<div class="field">
  <label for="exp_complete_field">Complete?</label>
  <input type="checkbox" id="exp_complete_field" disabled>
</div>
```

e. Locale all buttons in this page, this includes input type submit, reset, and button and assign a visual class name of "button" to these elements.

```
<input class="button" type="submit" id="exp_find_button" value="Find">
<input class="button" type="button" value="Add Measurement"
id="mea_add_button" disabled>
<input class="button" type="submit" value="Ok" id="mea_ok_button">
<input class="button" type="reset" value="Cancel" id="mea_reset_button">
```

3. Modify the `about.html` page, to use same stylesheet as the index page, and use similar types of visual elements to arrange it content.

a. Open the `about.html` file and add a reference to the style.css document to the head section of the about page. Remember that `style.css` file is in the resources folder.

```
<link rel="stylesheet" href="resources/style.css">
```

b. Place a fieldset element around all content within the body of the about page and replace the h3 element with the legend element.

```
<fieldset>
<legend>About DukeLabs</legend>
DukeLabs was created in 2020 to handle tracking of experiments and
measurements for the DukeLabs institute.
</fieldset>
```

4. Redefine default styles in the `style.css` document.

   a. Open the `style.css` file located in the resources folder and comment out existing style definition for the header element.

      Hint: Select relevant lines of code and press CTRL+/.

```
/*header {*/
/*    color:red;*/
/*    text-align:center;*/
/*}*/
```

   b. Reset default style settings. Set the following style properties:

      – A default margin of 0.1em for all elements
      – A default line-height of 1 for the page body
      – A default font that inherits font from the page for all input, button, textarea, and select elements.

      Notes:

      – By default, browsers assign styles to various HTML elements. However, these styles are not identical in all browsers. Therefore, you would not be entirely in control of the visual appearance of the page when you start designing your styles, unless you reset browser defaults.
      – In this practice you are making sure that different types of form input controls use same font as the page, and that all components in a page have a certain margin and that text lines are scaled 1 to 1 to the font size.

```
* {
  margin: 0.1em;
}
body {
  line-height: 1;
}
input, button, textarea, select {
  font: inherit;
}
```

5. Define global font, color, and sizing properties, for the root pseudo element, and enable switching of the layout scale based on the dimension properties of reported by the media query.

   a. Create a style for the :root pseudo element and set following variables and properties:

      - A –sizeS –sizeM –sizeL variables set to be 12px, 14px and 16 px
      - A –font variable set to be Helvetica, Arial, of the sans-serif family
      - A –size variable set to be the same value as the –sizeL variable. Use `var` function to reference another variable. Later this variable will be used to implement switching between sizes based on a dimension results returned by the media query.
      - A dark and a light color variables –colorD and –colorL, set to be cornflowerblue and ghostwhite.
      - A color property set to the value of the –colorD variable.
      - A font-family property set to the value of the –font variable.
      - A font-size property set to the value of the –size variable.
      - A font-weight property set to lighter.
      - A width property set to be 32em.

```
:root {
  --sizeS: 12px;
  --sizeM: 14px;
  --sizeL: 16px;
  --font: Helvetica, Arial, sans-serif;
  --size: var(--sizeL);
  --colorD: cornflowerblue;
  --colorL: ghostwhite;
  color: var(--colorD);
  font-family: var(--font);
  font-size: var(--size);
  font-weight: lighter;
  width: 32em;
}
```

   Notes:

      - It is a very common practice to define variables in the css document that other style properties may reference. For example, instead of defining stylistic property for different styles individually, you can make multiple different styles refer to a commonly defined variable. This way a single modification of the variable would have a global effect on all the styles that were referencing it. This approach promotes visual design consistency, simplifies code maintenance, and provides an efficient mechanism to dynamically alter page appearance with a simple variable change, rather than having to update multiple elements individually.
      - Global definitions of colors, fonts and sizes promote visual consistency. In this practice you define a single font, to be used throughout the application, two colors that all other components can use for their background, foreground, or border properties. You also define a base font size in fixed units (px), and then set size of everything else in relative units (em). This way the entire application can be scaled in proportion to the font size.
      - Several fixed font sizes are provided to allow application to be switched into different scales depending on the display dimensions, that can be established using media queries.

b. Create media query elements that reassign the root level –size property value depending on a max-width property of the screen. Set –size to be equal to –sizeM when the max-width is 800px, and –sizeS when max-width is 600px.

```
/* adjust sizing based on a media width */
@media screen and (max-width: 800px) {
  :root {
    --size: var(--sizeM);
  }
}
@media screen and (max-width: 600px) {
  :root {
    --size: var(--sizeS);
  }
}
```

6. Customize the appearance of the nav element and the links it contains.

a. Set nav element display as flex, and align-items as flex-end.

```
nav {
  display: flex;
  align-items: flex-end;
}
```

b. Customize the appearance of the ul element inside the nav element. Set list-style as none, display as flex and justify-content to the right.

```
nav ul {
  list-style: none;
  display: flex;
  justify-content: right;
}
```

c. Customize the appearance of the li element inside the nav element. Set padding-top as 0.2em, margin-top as 0.2em and margin-left as 1em.

```
nav li {
  padding-top: .2em;
  margin-top: .2em;
  margin-left: 1em;
}
```

d. Customize the appearance of the a element inside the nav element. Set color as the value of the –colorL variable, and text-decoration as none.

```
nav a {
  color: var(--colorL);
  text-decoration: none;
}
```

e. Customize the appearance of the a element inside the nav element that is being hovered over (use pseudo selector :hover for the a element inside the nav). Set color as white and font-weight as bold.

```
nav a:hover {
  color: white;
  font-weight: bold;
}
```

7. Arrange page header, and footer elements. The header would contain an image, a slogan text and a nav elements as a grid display. The footer would contain a copyright text.
   a. Define a style for the header element.
      - Set display property as grid.
      - Use grid-display-columns property to set up three columns for this grid. The first column would contain an image, the second column a slogan text and last column a nav element. Set first column to be fixed size of 3em, and other columns to be auto size.
      - Arrange a column-gap to be 0.4em and padding to be 0.2em.
      - Arrange a border to be solid style, the width of 0.1em, the radius of 0.6em.
      - Set background color to reference --colorD (dark color) variable and foreground color to reference --colorL (light color) variable.

```
header {
  display: grid;
  grid-template-columns: 3em auto auto;
  column-gap: 0.4em;
  padding: 0.2em;
  border-width: 0.1em;
  border-style: solid;
  border-radius: 0.6em;
  background-color: var(--colorD);
  color: var(--colorL);
}
```

   b. Define a style for the img_slogan visual class to scale the image to make it fit into specific size. Use flex display and set max-height of 3.2em.

```
.img_slogan {
  display: flex;
  max-height: 3.2em;
}
```

   c. Define a style for the slogan visual class. Sets the font-weight to bold, font-size to be 1.4em, and font-style to be italic. Use align-items property to align content to center. Set height property to 1.6em.

```
.slogan {
  font-weight: bold;
  font-size: 1.4em;
  font-style: italic;
  align-items: center;
  height: 1.6em;
}
```

d. Define a style for the footer element. Footer style should have a similar look as a header, that is, use similar colored rounded box.
   - Set background color to reference --colorD (dark color) variable and foreground color to reference --colorD (light color) variable.
   - Set padding as 1em on the right side, and 0.2em on the top and bottom sides.
   - Arrange a border to be solid style, the width of 0.1em, the radius of 0.6em.
   - Align text to the right.

```
footer {
  background-color: var(--colorD);
  color: var(--colorL);
  padding-right: 1em;
  padding-top: 0.2em;
  padding-bottom: 0.2em;
  border-width: 0.1em;
  border-style: solid;
  border-radius: 0.6em;
  text-align: right;
}
```

8. Arrange display of fieldsets, fields and labels. Each fieldset has a legend and contains a number of div elements that are using the field visual class.
   a. Define a style for the fieldset element.
      - Use a single column, auto sized grid display, with 0.6em padding.
      - Set solid style border to be 0.1em width, a --colorD (dark color) and border-radius of 0.6em.

```
fieldset {
  display: grid;
  grid-template-columns: auto;
  padding: 0.6em;
  border-width: 0.1em;
  border-color: var(--colorD);
  border-style: solid;
  border-radius: 0.6em;
}
```

   b. Arrange legend element to use bold font weight, italic style, a –colorD (dark color) and use 1em left margin.

```
legend {
  font-weight: bold;
  font-style: italic;
  color: var(--colorD);
  margin-left: 1em;
}
```

c. Define a field visual class, to arrange the display of divider elements that wrap around from items.
   - Use a grid display of three columns of 6em, 12em, and 4em widths.
   - Set align-items property to center.
   - Set padding to be 0.05em on top and bottom.

```
.field {
  display:grid;
  grid-template-columns: 6em 12em 4em;
  align-items: center;
  padding-top: 0.05em;
  padding-bottom: 0.05em;
}
```

9. Arrange the display of buttons. Construct two visual classes to condition the display of enabled and disabled buttons.
   a. Define button visual class that arranges button border and color.
      - Set solid style border to be 0.1em width, a --colorD (dark color) and border-radius of 0.2em.
      - Set background color to reference --colorL (light color) variable and foreground color to reference --colorD (dark color) variable.

```
.button {
  border-radius: 0.2em;
  border-color: var(--colorD);
  border-style: solid;
  border-width: 0.1em;
  background-color: var(--colorL);
  color: var(--colorD);
}
```

   b. Define a disabled variant of the button visual class style. Alter border and foreground color properties to be lightgray.

```
.button:disabled {
  border-color: lightgray;
  color: lightgray;
}
```

10. Define style for input and select fields.
    a. Dafne style for input and select elements that sets their color as black. Use solid border, with 0.1em thickness, 0.2em radius and referencing a --colorD (dark color) variable.

```
input, select {
  color: black;
  border: 0.1em solid var(--colorD);
  border-radius: 0.2em;
}
```

    b. Define a disabled variant of the input element style. Alter background property to be ghostwhite.

```
input:disabled {
  background: ghostwhite;
}
```

11. Create style for table header, table data and table row elements.
    a. Create style for the th (table header) elements.
        - Set border appearance to be a rounded solid line border, but only on left and right sides. Use 0.1em border width and 0.6em border radius. This would look as if column headers within a table are placed in rounded brackets.
        - Make this border color reference the --colorD (dark color) variable. Use normal font weight.
        - Make each column width take 25% of the width of the table. This table has 4 columns, so they will appear even width.

```
th {
  border-width: 0.1em;
  border-left-style: solid;
  border-right-style: solid;
  border-radius: 0.6em;
  border-color: var(--colorD);
  font-weight: normal;
  width: 25%;
}
```

    b. Create style for a td (table data) elements. Set their color to be black.

```
td {
  color: black;
}
```

    c. Create style that modifies background color for odd row of the table to reference the --colorL (light color) variable.

```
tr:nth-of-type(odd) {
    background: var(--colorL);
}
```

12. Customize appurtenance of the checkbox element. You task is to completely restyle the check box and design a small animation that plays out when check box changes its state.
    a. Create a style for the input type checkbox element.
        - Set appearance property to none. This is because, to completely overall an element style its default appearance needs to erase first.
        - Set white background color.
        - Set 0 margin.
        - Set width and height to 1.4em.
        - Set solid 0.1em width border, referencing --colorD (dark color) variable, and border-radius of 0.2em.
        - Set display property as grid.
        - Set place-content property as center.

```
input[type="checkbox"] {
  appearance: none;
  background-color: white;
  margin: 0;
  width: 1.4em;
  height: 1.4em;
  border: 0.1em solid var(--colorD);
  border-radius: 0.2em;
  display: grid;
  place-content: center;
}
```

    b. Set up checkbox animation using ::before pseudo selector.
        - You need to assign some content (an empty string will do) to be displayed inside the checkbox, because the default tick appearance has been erased.
        - Set width and height to be 0.8em. - This would make visualization of a checked value within the checkbox appear a little smaller than the checkbox border.
        - Set box-shadow property to be inset, with 1em width and height and reference the --colorD (dark color) variable. - This would make the checkbox checked value appear as a colored box in the center of the checkbox item. This colored box would in fact be achieved as a rendering of the shadow of the checkbox item displayed inside the box.
        - Set transform property to set scale to 0. - This would essentially make the shadow disappear. Which is what needs to happen for the unchecked state of the checkbox.
        - Set transition property to 120ms and make it play whatever animation is defined by the transform property, also make the animation slightly slower at the start and at the end using the ease-in-out property. This would make checkbox value gradually appear and disappear.

```
input[type="checkbox"]::before {
  content: "";
  width: 0.8em;
  height: 0.8em;
  box-shadow: inset 1em 1em var(--colorD);
  transform: scale(0);
  transition: 120ms transform ease-in-out;
}
```

c. Define a style variant for the check state of the checkbox input item. All you need to is set transition property to scale to 1. This is because the entire animation is already defined by the previous style, so all that needs to be done is trigger animation to display the shadow in the checkbox as scale 1 rather than scale 0. These two styles would animate the checkbox when it is checked and unchecked.

```
input[type="checkbox"]:checked::before {
  transform: scale(1);
}
```

13. Create style to visualize the dialog box. The intention is to display the modal dialog, appearing on top of other page elements. The task is to position the dialog and arrange its border, margin and padding properties.
    a. Create style for the dialog element.
       - Set margin and padding to 0.
       - Set top left corner of the dialog to be 17em.
       - Set border as a solid 0.1em line referencing --colorL (light color) variable, and a border-radius of 0.6em.
       - Set the dialog background color to refer --colorL (light color) variable.

```
dialog {
  margin: 0;
  padding: 0;
  top: 17em;
  left: 17em;
  border: 0.1em solid var(--colorL);
  border-radius: 0.6em;
  background: var(--colorL);
}
```

14. Create style to visualize the average value of measurements of a given experiment. This value display is achieved with two div elements. The outer element acts as a box that displays the border of the overall average display. The inner div element acts as a colored box, whose width is scaled to be proportionate to the average measurement value. These styles are using element ids as selectors because they represent a unique user interface component.
    a. Create a style for the exp_average_box (the outer div of the average component).
       - Set border to be a solid line of 0.1em thickness and referencing the --colorD (dark color) variable.
       - Set background color to referencing the --colorL (light color) variable.
       - Set the width property as 12em.

```
#exp_average_box {
  border: 0.1em solid var(--colorD);
  border-radius: 0.2em;
  background-color: var(--colorL);
  width: 12em;
}
```

b. Create a style for the exp_average_value (the inner div of the average component).
   - Set background color to reference the --colorD (dark color) variable.
   - Set color as ghostwhite. This would be used to display the average value as text within this box.
   - Set border-radius and margin properties as 0.2em.
   - Set width to be 0em. - This would make the box not to be visible by default, when no average value is yet calculated. This property would be dynamically changed via the JavaScript to make this box width proportional to the average value of measurements.
   - Set height and line-heigh properties to be 1.2em, and text-align to center. These properties make the text appear centered within the inner colored box.

```
#exp_average_value {
  background-color: var(--colorD);
  color: ghostwhite;
  border-radius: 0.2em;
  margin: 0.2em;
  width: 0em;
  height: 1.2em;
  line-height: 1.2em;
  text-align: center;
}
```

15. Modify code in the `main.js` JavaScript file that handles the display of the average experiment measurements value to use the styled div elements, instead of the progress element.
    a. In the `main.js` file, locate the `displayMeasurements` function and comment out the lines of code that operate on the exp_average_field element. This element has been removed from the page and replaced with the exp_average_value div element.

       Hint: Select relevant lines of code and press CTRL+/

```
// let exp_average_field = document.getElementById("exp_average_field");
// exp_average_field.value = averageRatio;
```

    b. Retrieve the div element with the exp_average_value id from the HTML document.

```
let exp_average_value = document.getElementById("exp_average_value");
```

    c. Set the `style.width` property for this element to be equal to the percentage value equal to the `averageRatio` variable.

```
exp_average_value.style.width = averageRatio + '%';
```

    d. Set the `innerText` property to average variable value.

```
exp_average_value.innerText = average;
```

    e. In the `main.js` file locate the `resetExperimentForm` function and comment out the lines of code that operate on the exp_average_field element.

```
// let exp_average_field = document.getElementById("exp_average_field");
// exp_average_field.value = 0;
```

    f. Retrieve the div element with the exp_average_value id from the HTML document.

```
// reset exp_average_value width to 0 and text to be empty
let exp_average_value = document.getElementById("exp_average_value");
```

g. Set the `style.width` property for this element to be 0%.

```
exp_average_value.style.width = "0%";
```

h. Set the `innerText` property to be an empty string.

```
exp_average_value.innerText = "";
```

16. Test the DukeLabs application appearance.

a. Run the application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.

b. Observe the overall layout and how it differs from the previous practice.

c. Move the cursor on top of the Contact and About items and see how they are highlighted.

d. Navigate to the about page and observe how its appearance has changed compared to the previous practice.

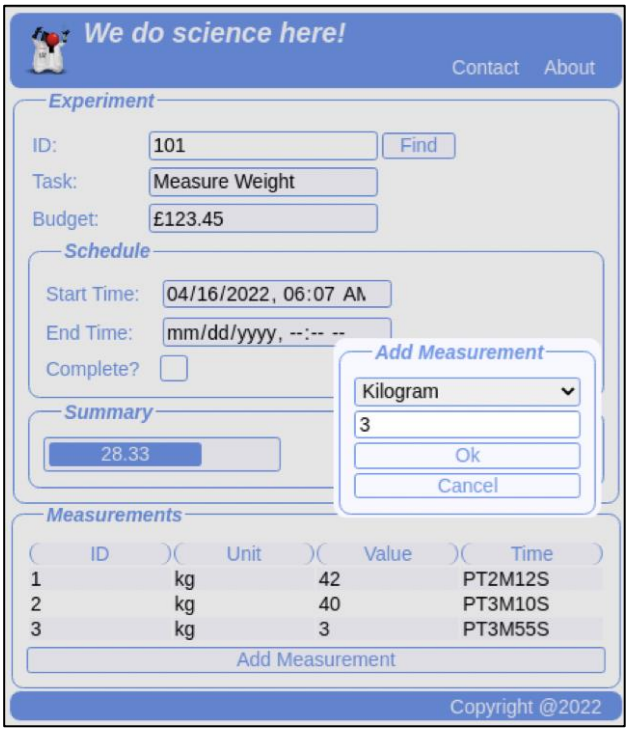
e. Click the browser's back button to return to the index page.

f. Query experiment 101, then 102, then 101 again and observe checkbox animation.

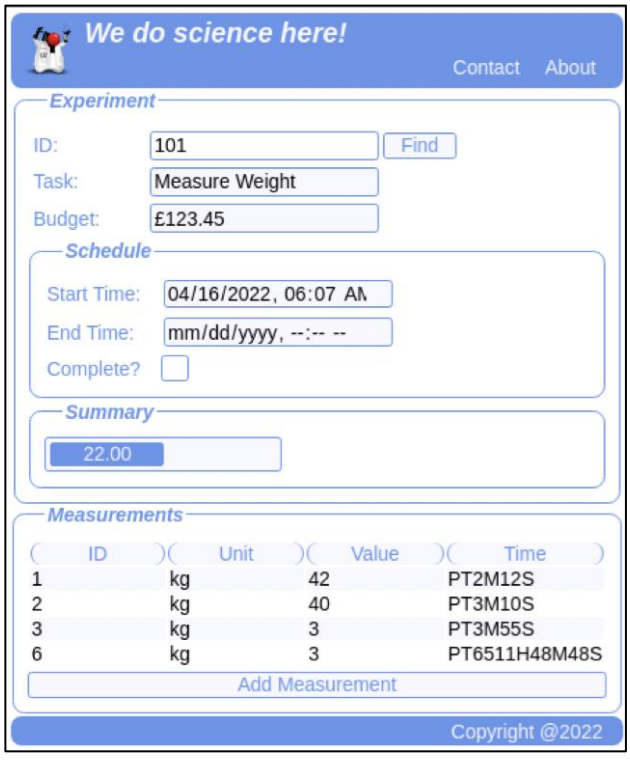
g. When looking at experiment 101, click the Add Measurement button and observe the appearance of the Add Measurement dialog box.

h. Add a new measurement to this experiment.

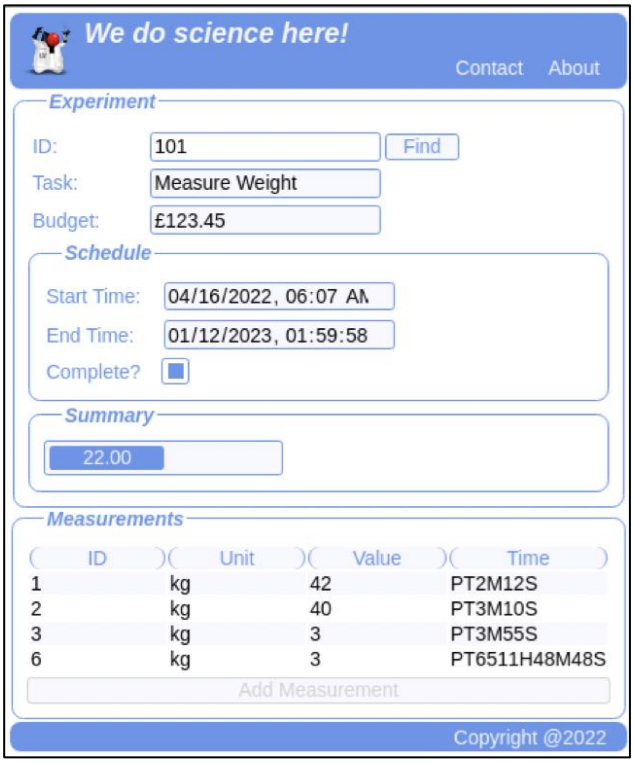
Hints: Click the Add Measurement button. The Add Measurement dialog box should appear. Select unit of measure form the drop-down list and enter a value into the input field. For example, unit is kilogram and value is 3.



i. Click the OK button to submit these values and dismiss the Add Measurement dialog box. Observe changes to the average value visualization.



j. Complete experiment 101. Notice the checkbox animation.



Note: If you want to repeat any of the tests, simply reload the `index.html` page. This would reset all experiments and measurements to the starting state, and all the changes you've made would be discarded. This is because all experiments and measurements data is currently only held in memory and is reinitialized from scratch every time the `index.html` is loaded.

Practices for Lesson 7: Advanced JavaScript

Overview

In these practices, you will:

- Improve application code structure with the use modules
- Handle application errors using exceptions and promises
- Produce animations using timer intervals

Practice 7-1: Improve Application Code Structures

Overview

In this practice, you will perform several code refactoring tasks that explore modules, exception handling, and use promises.

1. Refactor JavaScript code in the DukeLabs application to use modules.

   The idea is to separate code that implements user interface interactions from the code that implements data and business logic handling. This is in line with the model-view-controller design pattern, that stipulates the segregation of code based on its purpose. The advantage of this approach is that developers would be able to work independently on the front-end or the back-end parts of the application, so long as there is clearly defined boundary between these parts. It also opens a possibility to create different application front ends on top of the same back end or modify back-end implementation without affecting the front-end of the application.

   To achieve this, you will be instructed to split all code in the `main.js` file into two modules:

   - The existing `main.js` file would only retain code related to the handling of the UI. This will represent the DukeLabs application front-end.
   - A new module `data_service.js` that would hold all code related to data handling. This code would be moved into the `data_service.js` from the `main.js` file. This will represent the DukeLabs application back end.
   - The data_service module would export only specific objects and functions, that `main.js` would be allowed to import and use.

2. Refactor functions to handle correct and erroneous outcomes.

   The idea is to modify functions in the data_service (back-end) part of the application, so that they would be able to detect data errors and produce different outcomes if such cases.

   To achieve this, you will be instructed to use two different coding techniques:

   - Modify the `addMeasurement` operation of the `Experiment` class to throw an exception if it detects an invalid measurement value.
   - Create a new `findExperiment` function in the `data` object that will return a Promise with two possible outcomes - when the experiment was found, or when it was not.

   You will also have to adjust code to catch relevant errors or handle promise outcomes and report results to the user, in the main.js (front-end) part of the application.

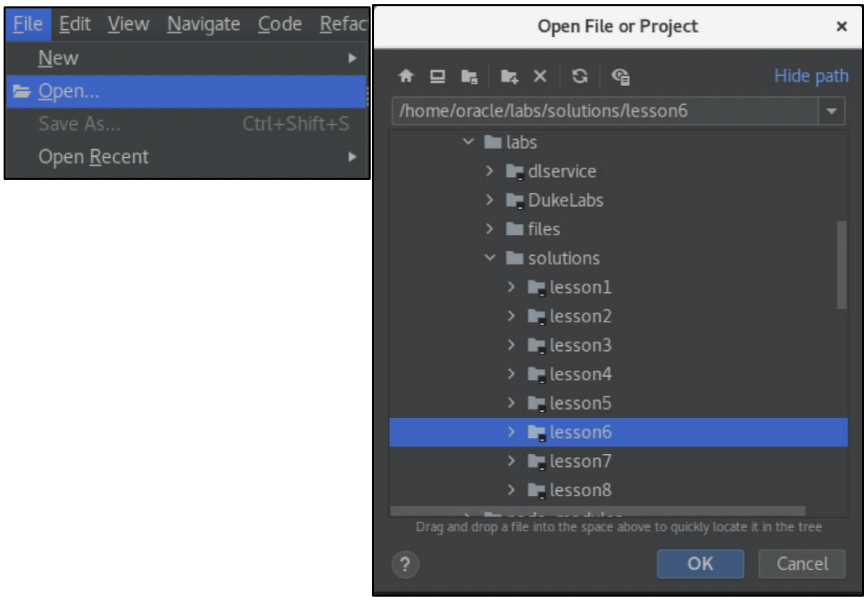
Tasks

1. If you have not completed previous practice, you may open a solution application for Lesson 6, which is the starting point of the Lesson 7 practice. Otherwise, you may continue to use same WebStorm project that you have worked on in the previous exercise.

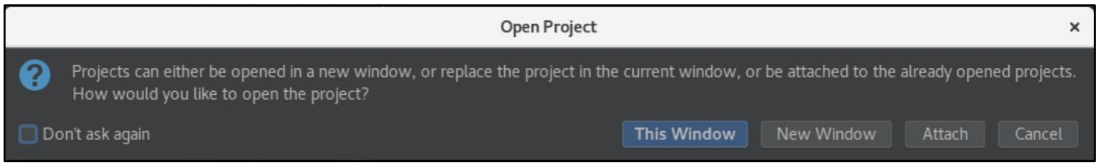
   Hints:

   - Launch WebStorm IDE if it is not already running.

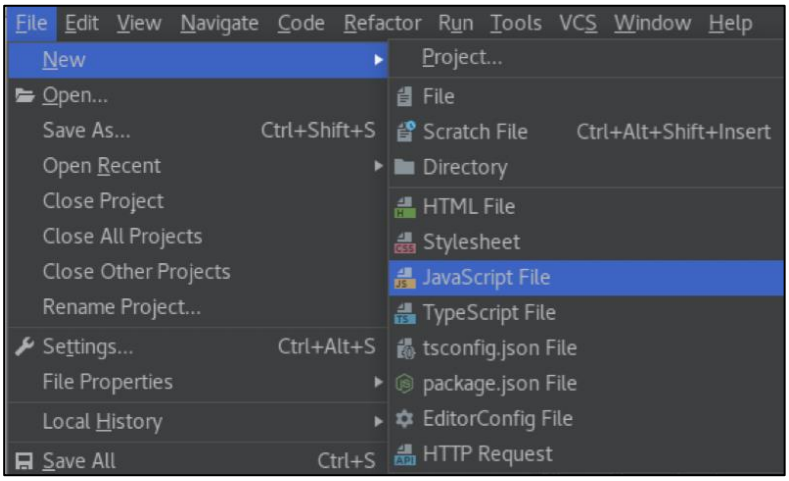
   - Use File -> Open menu to and browse to the `/home/oracle/labs/solutions/lesson6` folder.
   - Click the "OK" button.



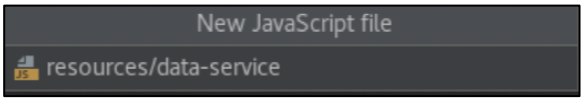
   - Click the "This Window" button.



2. Create JavaScript file to hold the back-end application logic.
   a. Switch back to the WebStorm project window and ensure that the DukeLabs folder is selected.
   b. Use File-> New - > JavaScript File menu.



   c. Set file name as resources/data_service and press the Enter key.



3. Restructure application, by refactoring the data handling, non-UI related code into the `data_service.js` file.
   a. Open the `main.js` file and select all code, except functions located in the part marked as " UI Events and Functions Section".
   b. Cut selected lines of code from the `main.js` file.

      Hint: Select relevant lines of code and press CTRL+x.
   c. Open the `data_service.js` file and paste this code.

      Hint: Press CTRL+v.
   d. Export `data` object, `calculateAverageMeasurement` and `formatCurrency` functions from the `data_service.js` file. Add this export instruction to the `data_service.js` file.

```
export {data, calculateAverageMeasurement, formatCurrency}
```

   e. Open the `main.js` file and add an import instruction for the same objects and functions. Add this import instruction at the start of the `main.js` file.

```
import {data, calculateAverageMeasurement, formatCurrency}
  from "./data_service.js";
```

Note: The data_service module data object functions accept and return Experiment and Measurement objects as parameters and return values. You have already exported and imported this data object, so all its functions are now available to be used in both modules. There is no need to explicitly export and import Experiment and Measurement class definitions, because they are implicitly shared between data_service and main modules.

f. Modify the `index.html` file to reference `main.js` file as a module. Add `type="module"` attribute to the script element referencing the `main.js` file.

```
<script type="module" src="resources/main.js"></script>
```

4. Test DukeLabs application.

Notes:

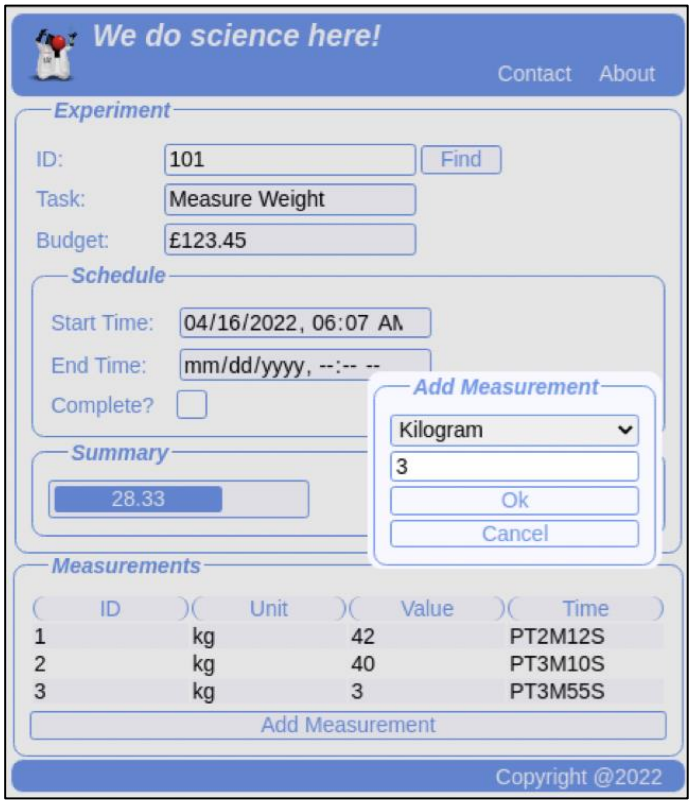
- Application should continue to work exactly as it was in the previous practice, because no actual functionality has been changed so far. The refactoring has only resulted in the improved code structure. This test is only needed to make sure that no errors were made in the application refactoring process and application continues to behave exactly as expected.
- In the next part of the practice, you will be instructed to make changes to the code that would affect application behavior. This test gives you an opportunity to study current application behaviors before they are modified.

a. Run application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.

b. Query experiment 101.

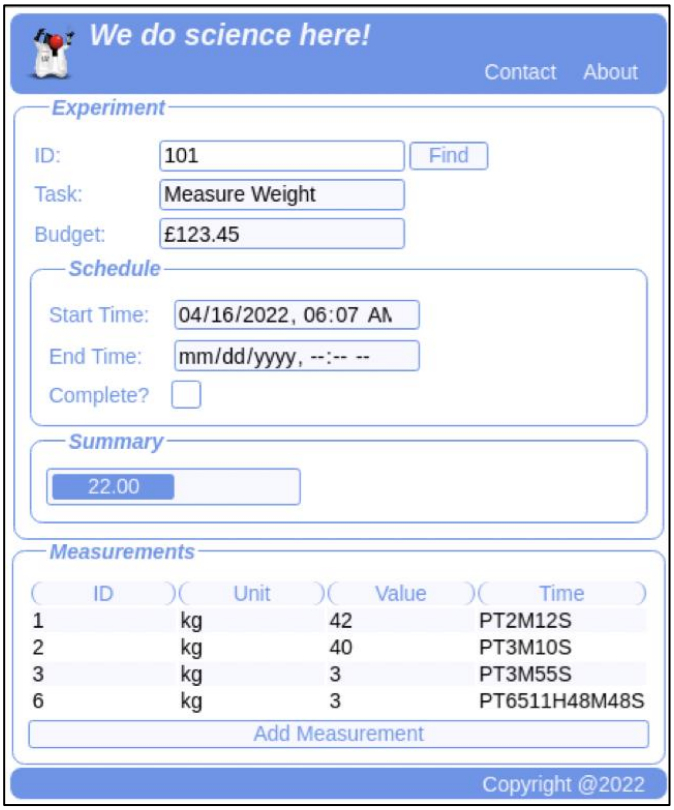
c. Add a new measurement to this experiment.

Hints: Click the Add Measurement button. The Add Measurement dialog box should appear. Select unit of measure form the drop-down list and enter a value into the input field. For example, unit is kilogram and value is 3.



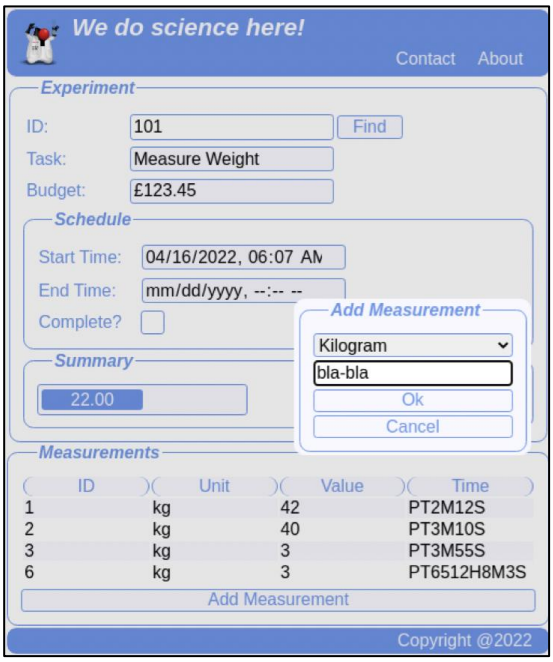
d. Click the OK button to submit these values and dismiss the Add Measurement dialog box.

e. Observe changes to the average value visualization.

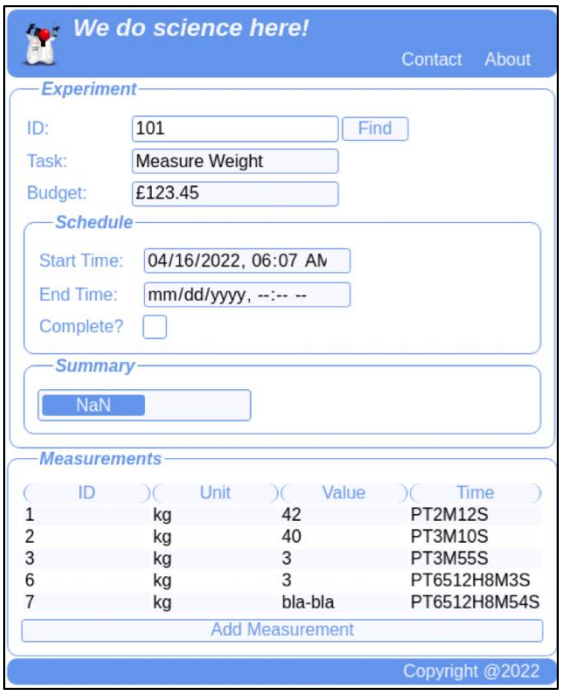


f. Attempt to add an erroneous measurement.

Hints: Click the Add Measurement button. The Add Measurement dialog box should appear. Select unit of measure form the drop-down list and enter an incorrect non-numeric value into the input field. For example, unit is kilogram and value is "bla-bla", or you may just leave the value field empty.



g. Click the OK button to submit these values and dismiss the Add Measurement dialog box.



Note: The application allowed you to submit the erroneous value, which allowed the error to spread to other parts of the application, adversely affecting functionalities, such as an average value calculation. This is what you will try to fix in the next part of this practice.

5. Modify logic of the `addMeasurement` function of the Experiment class to validate the measurement value and throw an exception in case an invalid value is detected.
   a. Open the `data_service.js` file and locate the `addMeasurement` function of the Experiment class.

b. Convert validate if value argument is a valid floating-point number, before using this value to produce a new measurement object. Throw an exception if value is not a valid number.

Hints:

- Just before the line of code that invokes the `addMeasurement` function upon the data object, insert a statement that converts value parameter into a float number
- Check if the result of conversion is not a number and throw the exception if this is the case.
- Use converted value to produce a new measurement.

```
addMeasurement(unit, value) {
  let duration = formatDuration(new Date().getTime() -
                                this.#startTime.getTime());
  // validate if measurement value is a number
  // throw exception when it is not
  let numValue = parseFloat(value);
  if (isNaN(numValue)) {
    throw "Measurement value "+value+" is not a number";
  }
  // create measurement and add it to the experiment
  data.addMeasurement(this.#id, new Measurement(unit, numValue, duration));
}
```

6. Create a new `findExperiment` function in the data object of the data_service module. This function should return a Promise object with two possible outcomes, one for the case when the experiment with a given id was found, and another when it was not.

a. Add a new function called `findExperiment` which accepts the experiment id as an argument.

```
// Find an Experiment with a given ID
findExperiment(eId) {
  // more logic will be added here next
}
```

b. Attempt to retrieve an experiment by invoking the `getExperiment` function upon the current object, passing the experiment id as an argument.

```
let experiment = this.getExperiment(eId);
// more logic will be added here next
```

c. Return a new Promise object that has two outcomes called resolved and rejected.

```
return new Promise((resolved, rejected) => {
  // more logic will be added here next
});
```

d. Test if experiment has been found. In which case trigger the resolved function and pass experiment object as an argument. Otherwise, trigger the rejected outcome and pass an error message informing that the experiment with a given id is not found.

```
if (experiment != undefined) {
  resolved(experiment);
} else {
  rejected("Experiment with id "+eId+" not found");
}
```

7. Modify the `addMeasurement` function in the `main.js` file to catch exceptions that could be thrown by the `addMeasurement` function of the experiment object.

   a. Locate the `addMeasurement` function in the `main.js` file and place a `try /catch` construct inside. This should be placed immediately before the last line of code that invokes displayMeasurements operation. Make the `catch` block logic display an error using an alert.

```
function addMeasurement(event) {
  event.preventDefault();
  let exp_id_field = document.getElementById("exp_id_field");
  let mea_unit_input = document.getElementById("mea_unit_input");
  let mea_value_input = document.getElementById("mea_value_input");
  let exp_id = parseInt(exp_id_field.value);
  let experiment = data.getExperiment(exp_id);
  experiment.addMeasurement(mea_unit_input.value, mea_value_input.value);
  let mea_add_dialog = document.getElementById("mea_add_dialog");
  let mea_form = document.getElementById("mea");
  mea_form.reset();
  mea_add_dialog.close();
  try {
    // code that attempts to add a measurement,
    // reset the form and close the dialog
    //would be added here next
  }catch(error) {
    alert(error);
  }
  displayMeasurements(experiment);
}
```

   b. Move the following functions into a `try` block:

   – The invocation of the `addMeasurement` function upon the experiment object

   – The retrieval of the measurement dialog and form elements from the HTML document

   – The reset of the form and closure of the dialog box

   Hint: Select relevant lines of code available within the `addMeasurement` function and press CTRL+x, then place cursor inside the `try` block and press CTRL+v.

```
try {
  experiment.addMeasurement(mea_unit_input.value, mea_value_input.value);
  let mea_add_dialog = document.getElementById("mea_add_dialog");
  let mea_form = document.getElementById("mea");
  mea_form.reset();
  mea_add_dialog.close();
}catch(error) {
  alert(error);
}
```

   Note: The idea of the `try` block is to terminate the program flow when error is produced. In this case, the dialog box would not be dismissed if the `addMeasurment` function throws an error. Thus, the user will be given a chance to correct their value input in the Add Measurement dialog box.

8. Modify the `findExperiment` function in the `main.js` module to use a Promise returned by the `findExperiment` function of the data object.

   a. Locate the `findExperiment` function in the `main.js` file and comment out the line of code that retrieves the experiment from the data object.

```
// let experiment = data.getExperiment(exp_id);
```

   b. Instead, invoke the `findExperiment` function upon the data object.

   – Create a function that handles the "then" case of the promise, which should accept an experiment object as an argument.

   – Create a function that handles the "catch" case of the promise, which should accept an error object as an argument.

```
// Change find experiment functionality to use a promise
data.findExperiment(exp_id).then((experiment)=>{
  // experiment and measurement display logic will be added here
}).catch((error)=>{
  // error display and the reset of UI elements forms will be added here
});
```

   c. Move lines of code that populate fields in the experiment form, including the `if/else` constructs that process the complete and ongoing experiments and invocation of the `displaysMeasurements` function inside the "then" action function.

   Hint: Select relevant lines of code available within the `findExperiment` function and press CTRL+x, then place cursor inside the `try` block and press CTRL+v.

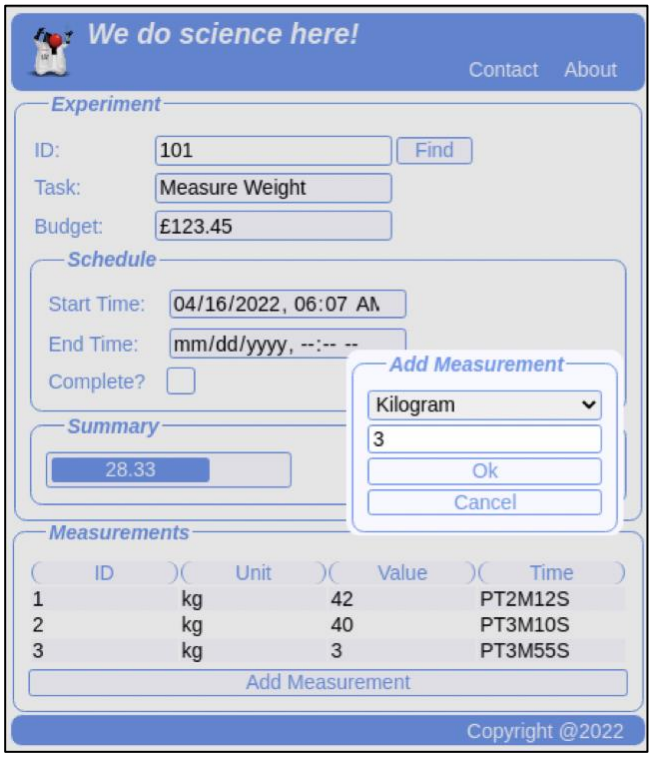
```
// Change find experiment functionality to use a promise
data.findExperiment(exp_id).then((experiment)=>{
  exp_task_field.value = experiment.task;
  exp_budget_field.value = formatCurrency(experiment.budget);
  exp_startTime_field.value = experiment.startTime
    .toISOString().slice(0, -1);
  if (experiment.complete) {
    exp_complete_field.checked = true;
    exp_endTime_field.value = experiment.endTime.toISOString().slice(0, -1);
    exp_complete_field.disabled = true;
    mea_add_button.disabled = true;
  }else{
    exp_complete_field.checked = false;
    exp_endTime_field.value = "";
    exp_complete_field.disabled = false;
    mea_add_button.disabled = false;
  }
  displayMeasurements(experiment);
}).catch((error)=>{
  // error display and the reset of UI elements forms will be added here
});
```

   d. Move lines of code that display an error alert, reset the experiment form, and reset the measurements table inside the "catch" action function.

```
.catch((error)=>{
  alert(error);
  resetExperimentForm();
  resetMeasurementsTable();
});
```

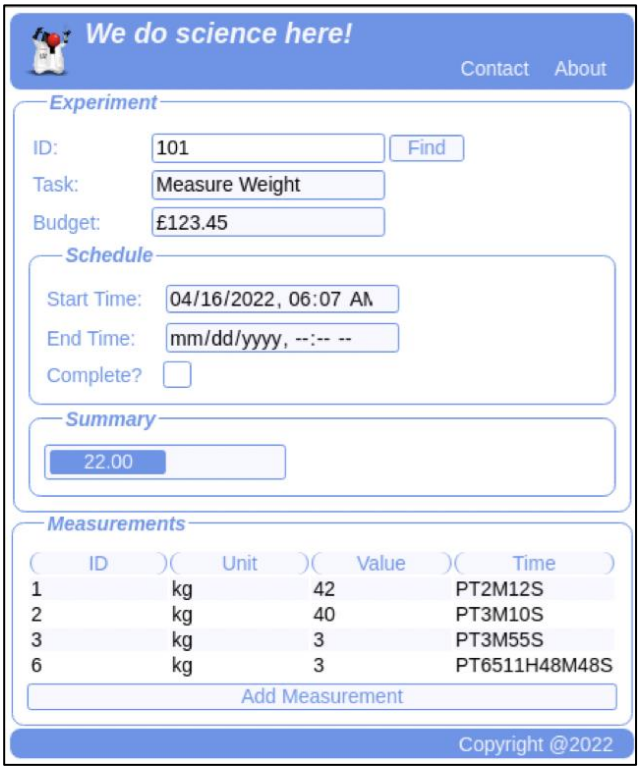
9. Test error-handling functionalities of the DukeLabs application.
   a. Run the application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.
   b. First, make sure "normal" non-erroneous functionality still works as expected. Query experiment 101. This should display the experiment and its measurements.
   c. Add a new measurement to this experiment.

      Hints: Click the Add Measurement button. The Add Measurement dialog box should appear. Select unit of measure form the drop-down list and enter a value into the input field. For example, unit is kilogram and value is 3.



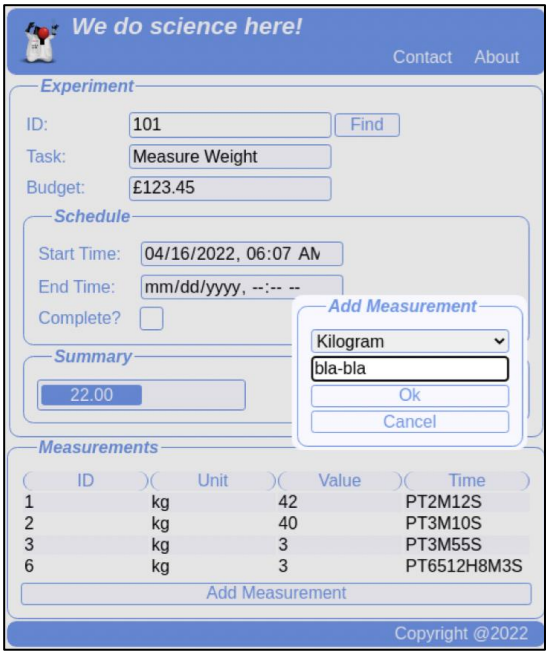
   d. Click the OK button to submit these values and dismiss the Add Measurement dialog box.

e. Observe changes to the average value visualization.

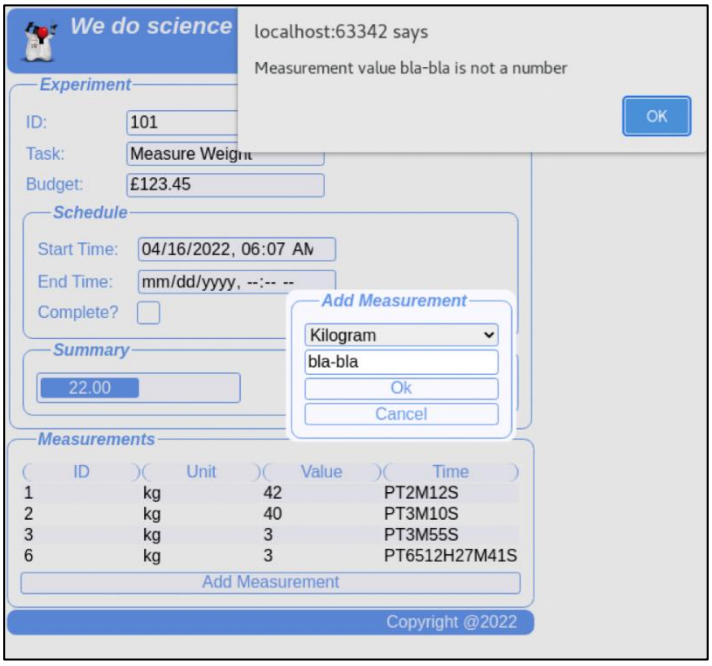


f. Now, test the application's error-handling functionality, by attempting to add an erroneous measurement.

Hints: Click the Add Measurement button. The Add Measurement dialog box should appear. Select unit of measure form the drop-down list and enter an incorrect non-numeric value into the input field. For example, unit is kilogram and value is "bla-bla", or you may just leave the value field empty.



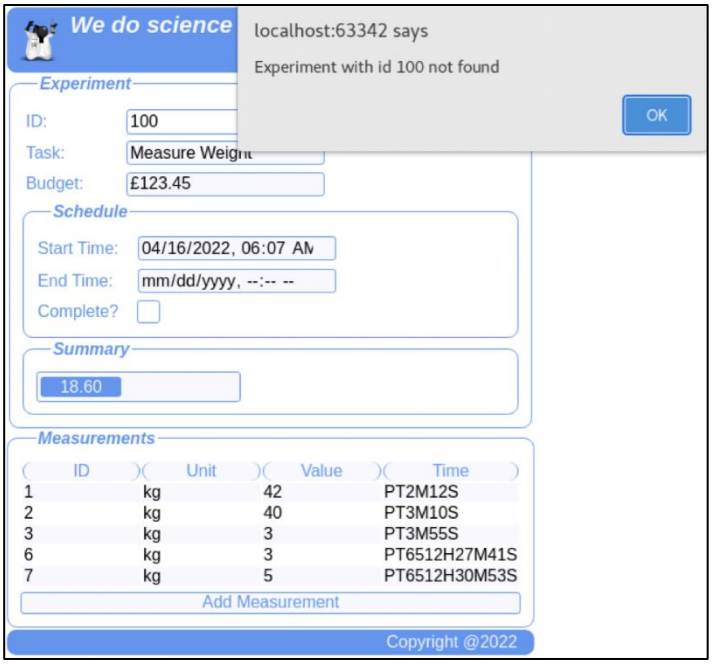
g. Click the OK button to submit these values.



Note: The application should now display an error message that indicates the erroneous value of the measurement. Also, note that the Add Measurement dialog box should reset, but would not be dismissed. This allows the user to enter the correct value.

h. Enter a numeric value, and click OK, or just cancel this dialog box.

i. Test the case when experiment is not found. Enter an invalid experiment id value in the experiment form and click the Find button.



Note: The application should now display an error message that indicates that the experiment with a given id is not found.

Practice 7-2: Use an Interval to Produce Animation Effects

Overview

In this practice, you will use an interval timer to perform animations.

The idea is to add a function to the `main.js` (front-end) part of the application, that will change the visual style of an element. Periodically, trigger this function based on an interval. This will cause a visual animation effect of the UI element which style will be altered by the function.

To achieve this, you will be instructed to:

- Add an additional style class to the `style.css` file that turns image into greyscale
- Add a function to the `main.js` file to toggle this style (add and remove the style) for the duke image element in the header section of the page
- Trigger this function very second using an interval

Tasks

1. Add a new visual style class, to use in image animation.
   a. Open the `style.css` file and add a new style called grey.

```
.grey {
  // style properties will be added there next
}
```

   b. Add a filter property to this style. Assign grayscale(100%) function to this property.

```
.grey {
  filter: grayscale(100%);
}
```

2. Create a JavaScript function to that would perform image animation.
   a. Open the `main.js` file and add a new function called `animateImage`, which accepts an image element as an argument.

```
function animateImage(image) {
  // implementation logic will be added here next
}
```

   b. Inside this function, invoke method toggle upon the classList property of the image element and pass a style class name "grey" as an argument.

```
function animateImage(image) {
    image.classList.toggle("grey");
}
```

   c. Inside the function that implements window load listener, retrieve the image element from the HTML document.

   Hint: This code should be added inside the function that is passed as an argument of the `window.addEventListener` function.

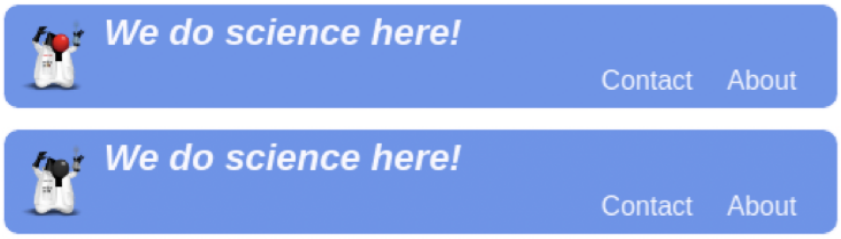
```
let image = document.getElementsByClassName("img_slogan").item(0);
```

d. Trigger the `animateImage` function, with 1 second interval, and pass an image element as an argument.

Hint: Use the `setInterval` function with a 1000 millisecond interval value.

```
setInterval(animateImage, 1000, image);
```

3. Test an interval and image animation functionality.
   a. Run the application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.
   b. Observe the image state altering between color and grayscale every second.



Practices for Lesson 8: Exchanging Data with Servers

Overview

In these practices, you will:

- Modify DukeLabs web application to invoke a REST service to manage experiments and measurement data
- Add chat capability to the DukeLabs web application

At the end of this practice there is a section that describes the server-side logic of the DukeLabs application.

Practice 8-1: Interact with a REST Service

Overview

In this practice, you will alter DukeLabs web application code to invoke REST Service that manages experiment and measurements data. At the start of this practice, you need to launch a server that is implemented in JavaScript, with `node.js`. This server provides REST Service that manages experiment and measurements, which you are going to invoke from the DukeLabs web application.

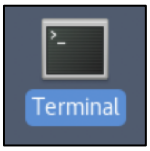
The server-side DukeLabs REST application is in fact a variant of the same code that you've been writing throughout this set of practices. It uses the exact same Experiment and Measurement classes, stores instances of these classes in-memory using the same data object and got identical functions. However, it also has functions that implement REST `get`, `put`, and `post` operations that allow clients to retrieve, experiments and measurements, update experiment marking it as complete, and add new measurements.

Your task will be to write code that invokes this DukeLabs REST Service to retrieve information about experiments and measurements when the page is loaded and store this information within the data object owned by the client. This object would act as client-side data cache from which page can obtain required information that it can display. Your next practice assignment would be to write code that invokes DukeLabs REST service to add measurements or complete experiments.

The server also implements user chat functionality with WebSockets. However, you will write code to interact with this chat server in the second part of this practice.

Tasks

1. Launch DukeLabs Service.
   a. Open a new terminal window if it is not already opened.



   b. Change folder to the location of the DukeLabs Service

      Hint: In the terminal window type `cd /home/oracle/labs/dlservice` and press Enter.

c. Start the `node.js` process executing code of the DukeLabs Service.

Hint: In the terminal window, type `node main.js` and press Enter.

Note:

- The `Node.js` server process should now start and report that it is listening on http://localhost:8080
- If later in this practice you may wish to restart the server, you may type CTRL+C in the terminal window and repeat the `node main.js` command.

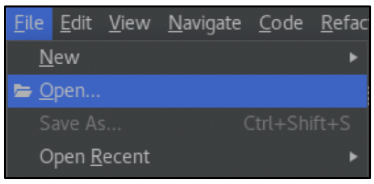
```
[oracle@edvmr1p0 ~]$ cd /home/oracle/labs/dlservice/
[oracle@edvmr1p0 dlservice]$ node main.js
Listening on port http://localhost:8080/
```

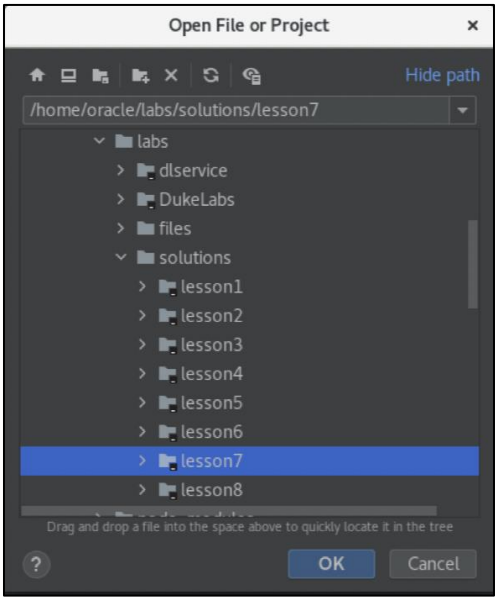
2. If you have not completed previous practice, you may open a solution application for Lesson 7, which is the starting point of the Lesson 8 practice. Otherwise, you may continue to use same WebStorm project that you have worked on in the previous exercise.

Hints:

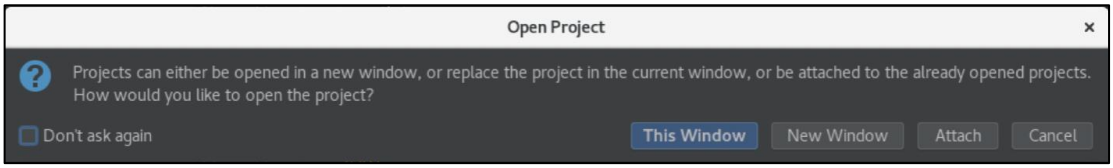
- Launch WebStorm IDE if it is not already running.

- Use File → Open menu to and browse to the `/home/oracle/labs/solutions/lesson7` folder.
- Click the "OK" button.





- Click the "This Window" button.



3. Provide functions to convert Experiment and Measurement objects to JSON as well as produce these objects from JSON data.

   Note: The JavaScript JSON API is unable to correctly produce or restore objects with private properties. Therefore, you need to create an object with public properties before you convert it to JSON. Likewise, you also must manually produce objects with private properties from the JSON string.

   a. Open the `data_service.js` file, locate the Experiment class definition and add a new function called `toJSON` to this class.

```
toJSON () {
  // implementation logic will be added there next
}
```

   b. Convert all properties of the current object to JSON.

      Hints:

      - Create an object that has id, task, budget, startTime, endTime and complete properties.
      - Populate each one of these properties with a corresponding property value from the current object.
      - Use the `JSON.stringify` function to convert this object to JSON.
      - Return the converted JSON value.

```
toJSON () {
    return JSON.stringify({id:this.id, task:this.task, budget:this.budget,
    startTime:this.startTime, endTime:this.endTime, complete:this.complete});
}
```

   c. Add a static function called `fromJSON` to the Experiment class.

```
static fromJSON(json){
  // implementation logic will be added there next
}
```

   d. Produce Experiment object from the JSON string.

      Note:

      - A JSON parse function will produce an object that has only data properties, but no functionalities. One way of adding functions to an object is to alter its prototype. However, because of the internal implementation mechanics of the JavaScript engine, this would not work for prototypes that has private properties, such as the Experiment and Measurement classes. The workaround is to harvest data from the JSON object and use it to programmatically construct an object of appropriate type, which would have all required functionalities.
      - JSON objects do not contain information about property types. Thus, the JSON parse function will produce properties such as date and time as just strings. You will have to manually produce other types from strings when needed.

Hints:

- Use the `JSON.parse` function to convert a JSON string into an object.
- Check if this object complete property is true, in which case use the `createCompleteExperiment` function to produce a new experiment object. Pass id, task, budget, and startTime properties as arguments to this function, and make sure to convert startTime property into a Date object first.
- If complete property is false, use the `createOngoingExperiment` function to produce a new experiment object. Pass id, task, budget, startTime, and endTime properties as arguments to this function, and make sure to convert startTime and endTime properties into a Date object first.
- Return the experiment object.

```
static fromJSON(json){
  let obj = JSON.parse(json);
  if (obj.complete) {
    return Experiment.createCompleteExperiment(obj.id, obj.task, obj.budget,
           new Date(obj.startTime), new Date(obj.endTime));
  }
  return Experiment.createOngoingExperiment(obj.id, obj.task, obj.budget,
         new Date(obj.startTime));
}
```

e. Locate the Measurement class definition in the `data_service.js` file and add a new function called `toJSON` to this class.

```
toJSON () {
  // implementation will be added here next
}
```

f. Convert all properties of the current object to JSON.

Hints:

- Create an object that has id, unit, value, and time properties.
- Populate each one of these properties with a corresponding property value from the current object.
- Use the `JSON.stringify` function to convert this object to JSON.
- Return the converted JSON value.

```
// convert current object to JSON
toJSON () {
  return JSON.stringify({id:this.id,unit:this.unit,
                         value:this.value,time:this.time});
}
```

g. Add a static function called `fromJSON` to the Measurement class.

```
static fromJSON(json){
  // implementation logic will be added there next
}
```

h. Produce Measurement object from the JSON string.

Hints:

- Use the `JSON.parse` function to convert a JSON string into an object.
- Construct new Measurement object using id, unit, value, and time properties.
- Return the measurement object.

```
static fromJSON(json){
  let obj = JSON.parse(json);
  return new Measurement(obj.id, obj.unit, obj.value, obj.time);
}
```

i. Adjust Measurement constructor to use id provided by the server.

Hints:

- Add an `id` parameter to the `Measurement` constructor.
- Place a comment to a line that assigs id as an increment of the maxId variable.
- Instead assign Measurement id property using parameter value.

```
constructor(id, unit, value, time) {
  // this.#id = ++Measurement.#max_id;
  this.#id = id;
  this.#unit = (Measurement.#units.indexOf(unit) == -1) ? null : unit;
  this.#value = value;
  this.#time = time;
}
```

4. Add constant definitions to the `data_service.js` file that will be used to describe communications with the DukeLabs REST service.

a. Create a constant called URL and set its value to be the URL pointing to the DukeLabs REST Service "http://localhost:8080/experiment/"

```
const URL = "http://localhost:8080/experiment/";
```

b. Create a constant called NOBODY_REQUEST that should describe an object, representing an HTTP `GET` request, with a `'Content-Type': 'application/json'` property.

```
const NOBODY_REQUEST =  {
  method: 'GET',
  headers: {
    'Content-Type': 'application/json'
  }
};
```

c. Create a constant called BODY_REQUEST that should describe an object, representing an HTTP request, with empty method and body properties, and with a `'Content-Type': 'application/json'` property.

```
const BODY_REQUEST =  {
  method: '',
  body: '',
  headers: {
    'Content-Type': 'application/json'
  }
};
```

5. Replace code that instantiates experiments and measurements with code that retrieves these objects from the DukeLabs REST Service.

   a. Comment out the section of code that instantiates experiment and measurement objects in the `data_service.js` file.

      Hint: Select appropriate lines of code and press CTRL+/

```
// let experiment1 = Experiment.createOngoingExperiment(101,
//   "Measure Weight", 123.45, new Date(2022,3,16,6,7));
// let experiment2 = Experiment.createCompleteExperiment(102,
//   "Measure Length", 321.54, new Date(2022,4,1,14,30),
//   new Date(2022,4,2,21,12));
// let measurement1 = new Measurement("kg",42,'PT2M12S');
// let measurement2 = new Measurement("kg",40,'PT3M10S');
// let measurement3 = new Measurement("kg",3,"PT3M55S");
// let measurement4 = new Measurement("m",12,"PTH20M");
// let measurement5 = new Measurement("m",10,"PT1H22M:10S");
// data.addExperiment(experiment1);
// data.addExperiment(experiment2);
// data.addMeasurement(experiment1.id,measurement1);
// data.addMeasurement(experiment1.id,measurement2);
// data.addMeasurement(experiment1.id,measurement3);
// data.addMeasurement(experiment2.id,measurement4);
// data.addMeasurement(experiment2.id,measurement5);
```

   b. Create an async `loadMeasurements` function inside the `data` object, that should accept an experiment id as an argument. Use comma to separate this function from others functions inside the object.

```
async loadMeasurements (eId) {
  // implementation will be added here next
}
```

   c. Inside the `loadMeasurements` function, invoke the DukeLabs REST service to obtain a collection of measurements for a given experiment id.

      Hints:

      - Construct a URL that starts with the URL constant value "http://localhost:8080/experiment/, then contains experiment id and ends on "measurements".
      - Fetch measurements from the REST Service.
      - Use NOBODY_REQUEST.
      - Use await operator and place the result of the call into a variable.

```
let response = await fetch(URL+eId+"/measurements", NOBODY_REQUEST);
// more code will be added here next
```

d. Process server response to construct measurement objects.

Hints:

- Check if response status is 200. - Which indicates correct response.
- Extract text value from the response object. - This value should contain JSON data with all measurements that belong to a given experiment.
- Parse this JSON string. -This would produce an array of measurement objects.
- Create a loop to iterate through this array.
- Construct a measurement object using the `fromJSON` function of the Measurement class.
- Inside this loop invoke the `addMeasurement` function upon the data object. This function should accept the experiment id and the measurement object as arguments.

```
if (response.status == 200) {
  let json = await response.text();
  let measurements = JSON.parse(json);
  for (const measurement of measurements){
    data.addMeasurement(eId,Measurement.fromJSON(measurement));
  }
}
```

e. Create an async `loadExperiments` function inside the `data` object, that should accept an array of experiment id values as an argument. Use comma to separate this function from others functions inside the object.

```
async loadExperiments (ids) {
  // implementation will be added here next
}
```

f. Inside the `loadExperiments` function, invoke the DukeLabs REST service to obtain a collection of experiments that match given list of id values.

Hints:

- Create a `for` loop that iterates through all elements in the experiment ids array, that this function has received as an argument.
- Inside this loop construct a URL for each experiment, that starts with the URL constant value "http://localhost:8080/experiment/, and then contains the experiment id.
- Fetch the measurements from the REST Service.
- Use NOBODY_REQUEST.
- Use await operator and place the result of the call into a variable.

```
for (const id of ids) {
  let response = await fetch(URL + id, NOBODY_REQUEST);
  // more code will be added here next
}
```

g. Inside this loop process each response that contains an experiment object.

Hints:

- Check if response status is 200. - Which indicates correct response.
- Extract text value from the response object. - This value should contain JSON data of an experiment with a specific id.
- Parse this JSON string. -This would produce an experiment object.
- Construct an experiment object using the `fromJSON` function of the Experiment class.
- Invoke the `addExperiment` function upon the data object. This function should accept the experiment object as an argument.
- Invoke the `loadMeasurements` function upon the data object. This function should accept the experiment id as an argument.

```
if (response.status == 200) {
  let json = await response.text();
  data.addExperiment(Experiment.fromJSON(json));
  this.loadMeasurements(id);
}
```

h. Create an async `loadData` function inside the `data` object. Inside this function, use the `await` operator to invoke the `loadExperiments` function upon the current object. Pass an array of experiment id values 101 and 102 as arguments to this function. Use comma to separate this function from others functions inside the object.

```
async loadData(){
  await this.loadExperiments([101,102]);
}
```

6. Add a function to the data object that posts a new measurement to the DukeLabs REST Service.

a. Locate the `data` object in the `data_service.js` file and create a `postMeasurement` async function that should accept experiment id, unit of measure and value as arguments. Use comma to separate this function from others functions inside the object.

```
async postMeasurement(eId, unit, value) {
  // implementation code will be added here next
}
```

b. Modify the BODY_REQUEST object, so that its method property is set to POST, and its body property is set to a JSON string that contains unit and value properties.

```
BODY_REQUEST.method = 'POST';
BODY_REQUEST.body = JSON.stringify({
  "unit": unit,
  "value": value
});
// more code will be added here next
```

c. Post this request containing a measurement to the DukeLabs REST Service

Hints:

- Construct a URL that starts with the URL constant value "http://localhost:8080/experiment/, then contains experiment id and ends on "measurements".
- Use fetch function to trigger REST Service call.
- Use BODY_REQUEST.
- Use the `await` operator and place the result of the call into a variable.

```
let response = await fetch(URL+eId+"/measurements", BODY_REQUEST);
// more code will be added here next
```

d. Process service response. The correct response from the DukeLabs Service should contain the JSON string that represent the measurement object that server has created.

Hints:

- Check if you have received a correct response, which is indicated by the response status 200 code.
- Extract text from the response, turn this JSON string into a measurement object using the `Measurement.fromJSON` function and return the result.
- In case, the response is not correct (code is not 200) throw an error indicating that the measurement was not added.

```
if (response.status == 200) {
  let json = await response.text();
  return Measurement.fromJSON(json);
}
throw "Unable to add measurement";
```

7. Add a function to the `data` object that invokes the DukeLabs REST service to complete the experiment. Use comma to separate this function from others functions inside the object.

a. Locate the data object in the `data_service.js` file and create a `completeExperiment` async function that should accept experiment id as an argument.

```
async completeExperiment (eId) {
  // implementation code will be added here next
}
```

b. Modify the BODY_REQUEST object, so that its method property is set to `PUT`, and its body property is empty.

```
BODY_REQUEST.method = 'PUT';
BODY_REQUEST.body = '';
// more code will be added here next
```

c. Post this request to complete the experiment to the DukeLabs REST Service

Hints:

- Construct a URL that starts with the URL constant value "http://localhost:8080/experiment/, and then contains an experiment id.
- Use fetch function to trigger REST Service call.
- Use BODY_REQUEST.
- Use the `await` operator and place the result of the call into a variable.

```
let response = await fetch(URL+eId+"/complete", BODY_REQUEST);
// more code will be added here next
```

d. Process service response. The correct response from the DukeLabs Service should contain the JSON string that represent the experiment completion date-time value.

Hints:

- Check if you have received a correct response, which is indicated by the response status 200 code.
- Extract text from the response, turn this JSON string into a Date object and return the result.
- In case response if not correct (code is not 200) throw an error indicating that the experiment was not completed.

```
if (response.status == 200) {
  let json = await response.text();
  return new Date(JSON.parse(json));
}
throw "Unable to complete the experiment";
```

8. Modify Experiment class to use newly created functions that invoke the REST Service.

a. Locate the `addMeasurement` function inside the Experiment class in the `data_service.js` file and change it to be an async function.

```
async addMeasurement(unit, value) {
  // logic of this function will be modified next
}
```

b. Inside this function comment out the line of code that invokes the `addMeasurement` function upon the data object.

Hint: Select relevant line of code that press CTRL+/.

```
// data.addMeasurement(this.#id, new Measurement(unit, numValue, duration));
```

c. Add a line of code that invokes the `postMeasurement` function.

Hints:

- Pass current experiment id, unit, and value as arguments.
- Assign returned value to the measurement variable.
- Placed this function call at the end of the `addMeasurement` function of the Experiment object.

```
let measurement = await data.postMeasurement(this.id, unit, numValue);
```

d. Next, add a line of code that invokes the `addMeasurement` function upon the data object, pass current experiment id and the measurement object that was returned from the REST service as parameters.

```
data.addMeasurement(this.id, measurement);
```

e. Create new async function called `completeNow` inside the Experiment class.

```
async completeNow() {
  // implementation logic will be added here next
}
```

f. Inside this function assign the `endTime` to the complete accessor of the current object.

Hints:

- The `endTime` value is returned by the REST service in response to the request to complete the experiment. This value is then returned by the `completeExperiment` function of the data object.
- Use await operator to acquire the `endTime` value.

```
this.complete = await data.completeExperiment(this.#id);
```

9. Adjust code in the `main.js` file to utilize `data_service` functions that interact with the DukeLabs REST Service.

a. In the `main.js` file, locate a window load event handling function and add an invocation of the `loadData` function upon the data object as the first line of code within the load event handling function.

```
window.addEventListener("load", (event) => {
  data.loadData();
  // the rest of this function code should not be changed
}
```

Note:

- No more changes are required in the `main.js` file to accommodate the retrieval of the experiment and measurement objects, because these functions operate only on the local data cache maintained by the data_service module. Once data is retrieved from the REST Service, the remaining part of the program can access it locally.
- However, functionality that adds measurements and marks experiment as complete is now implemented through the REST service invocation using async functions that operate on the Promise objects. This requires adjustments to the code of the client that should use promises when handling responses from `addMeasurement` and `completeNow` functions.

b. Locate the `addMeasurement` function in the `main.js` file and comment out the try `/catch` construct inside this function. You will replace this with the code that handles the Promise object outcomes.

Hint: Select appropriate lines of code and press CTRL+/.

```
// try {
// experiment.addMeasurement(mea_unit_input.value, mea_value_input.value);
// let mea_add_dialog = document.getElementById("mea_add_dialog");
// let mea_form = document.getElementById("mea");
// mea_form.reset();
// mea_add_dialog.close();
// }catch(error) {
// alert(error);
// }
// displayMeasurements(experiment);
```

c. Instead of the `try/catch` construct, invoke the `addMeasurement` function upon the experiment object and handle then and catch outcomes of the Promise that this function now returns. The then outcome should perform same actions as were previously performed in the try block, and the catch outcome should perform same action as the catch block.

Hint: You may copy – paste this code from the section that you have just commented out.

```
experiment.addMeasurement(mea_unit_input.value, mea_value_input.value)
  .then(()=>{
    let mea_add_dialog = document.getElementById("mea_add_dialog");
    let mea_form = document.getElementById("mea");
    mea_form.reset();
    mea_add_dialog.close();
    displayMeasurements(experiment);
  }).catch((error)=>alert(error));
```

d. Locate the `completeExperiment` function in the `main.js` file and comment out the lines of code that create new Date object and assign it to the complete accessor of the experiment object. You will replace this with the code that invokes the `completeNow` function and handles the Promise object outcome that it returns.

Hint: Select appropriate lines of code and press CTRL+/.

```
// let date = new Date();
// date.setMilliseconds(0);
// experiment.complete = date;
```

e. Invoke the `completeNow` function upon the experiment object. Add the then outcome handling function. This function should set the experiment complete check box checked value to true, set the `endTime` field value, disable the check box and the add measurement button.

Hint: This code already exists inside the `completeExperiment` function, so it simply needs to be moved into the function that handles the then outcome of a promise. Select relevant lines of code, cut them using CTRL+x and paste inside the then function using CTRL+v.

```
experiment.completeNow().then(()=>{
  exp_complete_field.checked = true;
  exp_endTime_field.value = experiment.endTime.toISOString().slice(0, -1);
  exp_complete_field.disabled = true;
  mea_add_button.disabled = true;
});
```

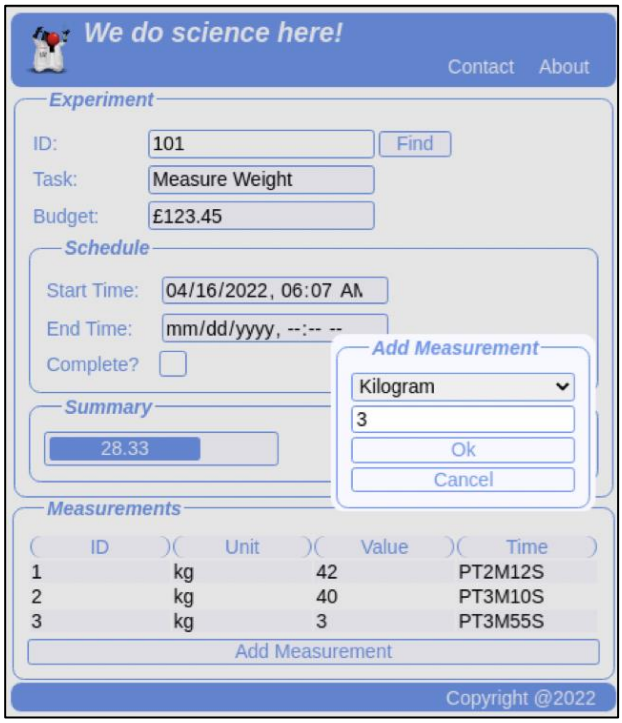
10. Test DukeLabs application.

Notes:

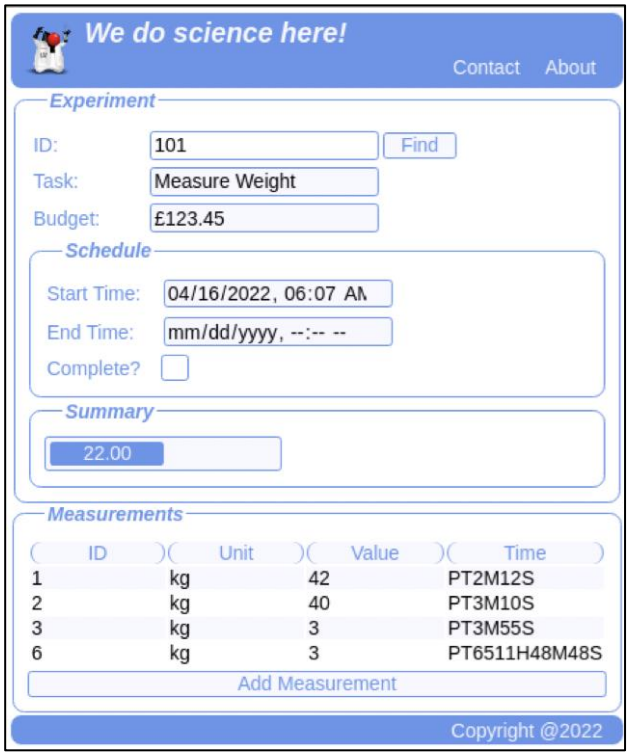
- Visually, application should continue to work exactly as it was in the previous practice, because all code changes related to the way in which the data is managed in the back end.
- The only tangible difference in behavior is the way this application retained its data state. Previously, data was reverted to its original state every time and HTML page was reloaded. This is because the data state was maintained in the JavaScript code running in the client browser. However, now data state is maintained on the server, within the REST Service application running in the `node.js` server. This means that if you add a measurement, or mark experiment as complete – changes would not be discarded when you reload the page. However, you may still reset application data state by restarting the `node.js` server. This version of the DukeLabs REST Service application does not save data into an actual database or a file, but only retains data in memory.
  - Make sure you are running the DukeLabs REST Service application in the `node.js` server. – It should have been started as the first step of this practice.
  - Run client application if it is not already running. Otherwise simply switch to the browser window used in the earlier tests of the application and refresh the page.
  - Query experiment 101.

a. Add a new measurement to this experiment.

Hints: Click the Add Measurement button. The Add Measurement dialog box should appear. Select unit of measure form the drop-down list and enter a value into the input field. For example, unit is kilogram and value is 3.

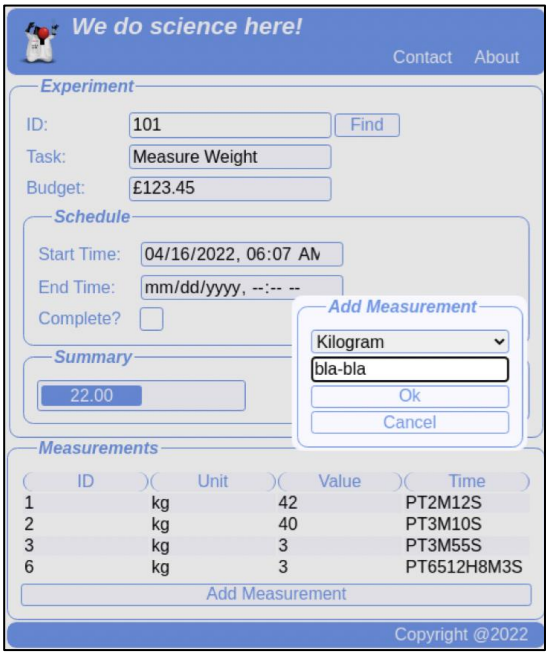


b. Click the OK button to submit these values and dismiss the Add Measurement dialog box. Observe changes to the average value visualization.



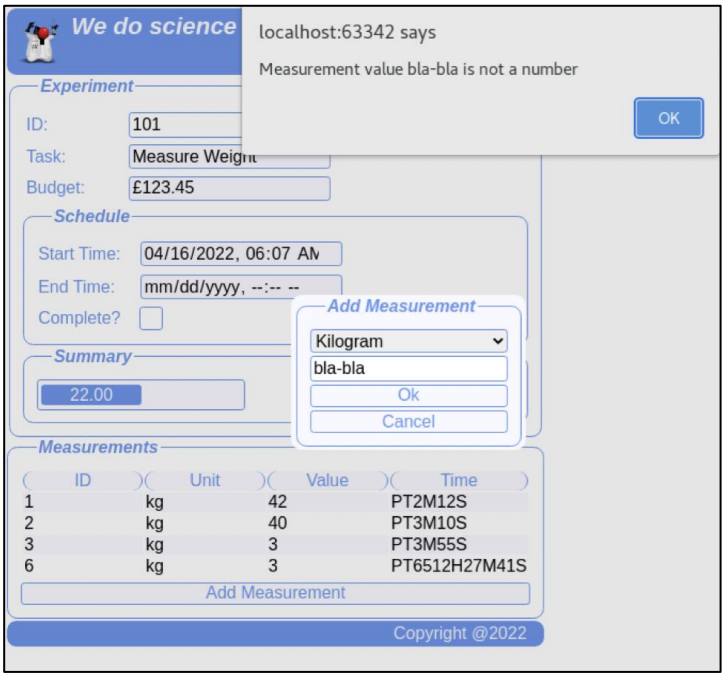
c. Attempt to add an erroneous measurement.

Hints: Click the Add Measurement button. The Add Measurement dialog box should appear. Select unit of measure from the drop-down list and enter an incorrect non-numeric value into the input field. For example, unit is kilogram and value is "bla-bla", or you may just leave the value field empty.



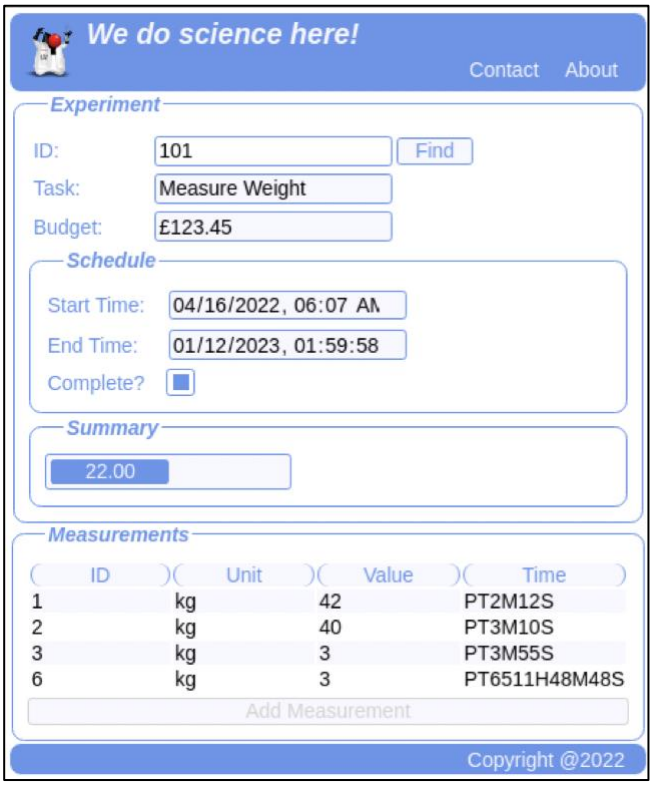
d. Click OK button to submit these values.

Note: Application should now display an error message that indicates the erroneous value of the measurement. Also, note that the Add Measurement dialog box should reset, but would not be dismissed. This allows a user to enter correct value.



e. After that you may either enter a correct numeric value, or simply dismiss this dialog box.

f. Select the complete experiment check box for the experiment 101. This should complete the experiment.



Note: If you want to revert application to its original data state and repeat tests, navigate to the terminal window where you've started the `node.js` server, press CTRL+c to terminate it, and start it again with the `node main.js` command.

Practice 8-2: Persist Data and Interact with a WebSocket Server

Overview

In this practice, you will create an HTML and JavaScript chat application that uses WebSocket communications to implement message exchanges between users. Another task in this part of the practice is to use session storage to memorize the name of the chat user. The idea is that the user should provide a name under which they will be known in the chat when they try to join the chat for the first time. You need to write code in the application to save this name into the session storage and pick it up automatically to allow user to leave and re-enter the chat without having to provide the name again, so long as this happens in the same browser session. Closing and reopening the browser would reset the session store and erase all values saved there.

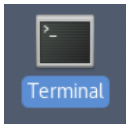
HTML document and the JavaScript file for this chat are provided as part of the practice setup. Your task will be to complete code in the JavaScript functions to perform WebSocket interactions and access the session storage.

Tasks

1. Add `chat.html` and `chat.js` files to your project.
   a. Copy `chat.html` and `chat.js` files from the `/home/oracle/labs/files` folder into the `/home/oracle/labs/DukeLabs/resources` folder.

      Hints:

      - Open a new terminal window.

      - Execute copy commands in the terminal.

      Note: You may have to adjust the target project folder to the actual folder of your project, if you are developing one of the solution projects rather than the original DukeLabs project that was created in the first practice. For example, if you are using a lesson7 solution project, your target path would be labs/solutions/lesson7.

```
cp labs/files/chat.html labs/DukeLabs/
cp labs/files/chat.js labs/DukeLabs/resources/
```

   b. Replace the contact email link to the first list item with a link referencing `chat.html` page.

      Hints:

      - Return to the WebStorm project window and open the `index.html` page.
      - Locate the anchor element referencing the contact email.
      - Replace it with a link pointing to the `chat.html` page.

```
<!-- <a href="mailto:duke@dukelabs.org">Contact</a>-->
<a href="chat.html">Contact</a>
```

2. Study source code of the `chat.html` and `chat.js` files.
   a. Study the `chat.html` page source code. There is no need to write any code, but you need to understand this page structure and its elements, because later in this practice you will be writing code to handle events raised by user action over these elements.

      Notes about the `chat.html` page:

      - Uses same stylesheet as the index and about pages
      - Uses references `chat.js` file, that contains code handling this page events.
      - Uses similar header and footer as the index page but has a different navigation element that points back to the index page
      - Has a div element with id "chat" that is intended to display chat messages
      - Has a form with the "msg" input field to submit new messages
      - Has buttons to join the chat, send messages and leave the chat
      - Has a dialog to enter a name for the user who is about to join the chat
      - This dialog box should be displayed when user clicks the "Join" button, on the condition that the username has not yet been saved into a session store. If name is found in the session store, the dialog box will not be displayed, and user will be allowed to join the chat immediately.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset='UTF-8'>
  <link rel="stylesheet" href="resources/style.css">
  <script type="text/javascript" src="resources/chat.js"></script>
  <title>Chat</title>
</head>
<body>
  <header>
    <img class="img_slogan" src="resources/duke.png">
    <div class="slogan">Let's have a chat!</div>
    <nav>
      <ul>
        <li><a href="index.html">Back</a></li>
        <li><a href="about.html">About</a></li>
      </ul>
    </nav>
  </header>
  <fieldset>
    <form id="cht">
      <legend>Chat</legend>
      <div id="chat"></div>
      <div>
        <label for="msg">Message:</label>
        <input type="text" id="msg" disabled>
      </div>
      <div>
    <input class="button" type="button" id="cht_join" value="Join">
    <input class="button" type="submit" id="cht_send" value="Send" disabled>
    <input class="button" type="reset" id="cht_leave" value="Leave" disabled>
      </div>
    </form>
  </fieldset>
  <footer>Copyright @2022</footer>
```

```
  <dialog id="join_chat_dialog">
    <form id="joi">
      <fieldset>
        <legend>Join Chat</legend>
        <label for="name_input">Enter your name:</label>
        <input id="name_input" type="text">
        <input class="button" type="submit" value="Ok" id="joi_ok">
        <input class="button" type="reset" value="Cancel" id="joi_cancel">
      </fieldset>
    </form>
  </dialog>
</body>
</html>
```

b. Study the `chat.js` source code. In the next practice steps, you will be instructed to fill in gaps in this source code.

Notes about the `chat.js` script:

- Declares two global variables to represent a WebSocket and a Session Store
- Upon the page load event:
  - Initializes the store object as a sessionStrorage
  - Registers event listeners for all buttons in the page
- The function `cancelJoinDialog` is already implemented to close the join dialog box.
- Functions `showJoinDialog`, `okJoinDialog`, `joinChat`, `sendMessage` and `leaveChat` will be implemented later in this practice according to the comments in the source code.

```
let socket;
let store;
window.addEventListener("load", (event) => {
  store = window.sessionStorage;
  let cht_join = document.getElementById("cht_join");
  let cht_send = document.getElementById("cht_send");
  let cht_leave = document.getElementById("cht_leave");
  let joi_ok = document.getElementById("joi_ok");
  let joi_cancel = document.getElementById("joi_cancel");
  cht_join.addEventListener("click", showJoinDialog);
  cht_send.addEventListener("click", sendMessage);
  cht_leave.addEventListener("click", leaveChat);
  joi_ok.addEventListener("click", okJoinDialog);
  joi_cancel.addEventListener("click",cancelJoinDialog);
});
function cancelJoinDialog(event) {
  let join_chat_dialog = document.getElementById("join_chat_dialog");
  join_chat_dialog.close();
}
```

```
function okJoinDialog(event) {
  // Prevent default action.
  // Acquire join dialog, join form, name input elements.
  // Validate the name input value by checking:
  // - If it exists
  // - If its length is not zero.
  // Display an alert and return if name is invalid.
  // If name is valid:
  // - Reset the form.
  // - Close the chat dialog.
  // - Save the name into a session store.
  // - Invoke joinChat function passing name as an argument.
}
function joinChat(user) {
  // Declare a URL that point to the chat server.
  // Open a socket with this URL.
  // Register onopen socket event handler, which should:
  // - Create a new message object containing a user property.
  // - User property value should be picked up from the store object.
  // - Convert message object to JSON and send it to the socket.
  // Register onmessage socket event handler, which should:
  // - Parse event data as a JSON message.
  // - Acquire "chat" element from the document.
  // - Dynamically create new html item containing message text.
  // - Append this item to the "chat" element of the page.
  // Register onerror socket event handler, which should:
  // - Display an alert containing the error.
  // Register onclose socket event handler, which should:
  // - Display an alert stating that chat is now closed.
  // - Clear all messages displayed "chat" element.
  // - Reset all buttons and fields to their original state.
  // Socket is now opened and callback functions are prepared.
  // Disable the join button.
  // Enable all other buttons and a message field.
}
function sendMessage(event) {
  // Prevent default action.
  // Acquire chat form, and message field value from the document
  // Reset the form.
  // Create a new message object containing text property.
  // Assign the message field value to the text property.
  // Convert message object to JSON and send it to the socket.
}
function leaveChat(event) {
  // Perform a normal socket closure.
}
```

3. Implement the `okJoinDialog` function in the `chat.js` script.
   a. Locate the `okJoinDialog` function in the `chat.js` script and prevent it from executing the default action.

      Note: The reason to present the default action is because you are not trying to submit this form to the server using HTTP request, but instead programmatically handle form fields within the client JavaScript logic.

```
function okJoinDialog(event) {
  event.preventDefault();
  // more implementation code will be placed here next
}
```

b. Acquire join dialog, join form, name input elements from the HTML document.

```
let join_chat_dialog = document.getElementById("join_chat_dialog");
let joi_form = document.getElementById("joi");
let name_input = document.getElementById("name_input");
// more implementation code will be placed here next
```

c. Extract the value of the name_input field. Display an alert and return from this function if name does not exist, or its length is zero.

```
let user = name_input.value;
if (user == null || user.length == 0) {
  alert("Invalid name");
  return;
}
// more implementation code will be placed here next
```

d. Reset the form and close the chat dialog box.

```
joi_form.reset();
join_chat_dialog.close();
// more implementation code will be placed here next
```

e. Save the name into a session store.

```
store.setItem("user", user);
// more implementation code will be placed here next
```

f. Invoke `joinChat` function passing name as an argument.

```
joinChat(user);
```

Note: This function algorithm ensures that user does not join the chat unless it has provided a name and saves this name into a session store.

4. Implement the `joinChat` function in the `chat.js` script.

a. Locate `joinChat` function in the `chat.js` script. Open the WebSocket using the "ws://localhost:8080/chat" URL.

```
function joinChat(user) {
  let url = "ws://localhost:8080/chat";
  socket = new WebSocket(url);
  // more implementation code will be placed here next
}
```

b. Register onopen socket event handler, which should:

- Create a new message object containing a user property
- Pick up user property value from the store object
- Convert message object to JSON and send it to the socket

```
socket.onopen = function(evt) {
  let message = {user: store.getItem("user")};
  socket.send(JSON.stringify(message));
}
// more implementation code will be placed here next
```

c. Register onmessage socket event handler, which should:
   - Parse event data as a JSON message
   - Acquire "chat" element from the document
   - Dynamically create new html item containing message text
   - Append this item to the "chat" element of the page

```
socket.onmessage = function (evt) {
  let message = JSON.parse(evt.data);
  let chatElement = document.getElementById("chat");
  let messageElement = document.createElement("div");
  let textItem = document.createTextNode(message.user+": "+message.text);
  messageElement.appendChild(textItem);
  chatElement.appendChild(messageElement);
};
// more implementation code will be placed here next
```

d. Register onerror socket event handler, which should display an alert containing the error.

```
socket.onerror = function (evt) {
  alert(evt.data);
};
// more implementation code will be placed here next
```

e. Register onclose socket event handler, which should:
   - Display an alert stating that chat is now closed
   - Acquire "chat" element from the document
   - Clear all messages displayed in the "chat" element. - There could be many child nodes inside the chat element, corresponding to different chat messages. Your task is to retrieve all these child nodes and remove them from the chat element.
   - Disable cht_join, cht_send, cht_leave buttons
   - Disable the msg field and set its value to be an empty string

```
socket.onclose = function (eve) {
  alert("Chat closed");
  let chatItem = document.getElementById("chat");
  while (chatItem.hasChildNodes()) {
    chatItem.removeChild(chatItem.lastChild);
  }
  document.getElementById("cht_join").disabled = false;
  document.getElementById("msg").disabled = true;
  document.getElementById("cht_send").disabled = true;
  document.getElementById("cht_leave").disabled = true;
  document.getElementById("msg").value = "";
};
// more implementation code will be placed here next
```

f. Disable the join button because chat is now opened.

```
document.getElementById("cht_join").disabled = true;
// more implementation code will be placed here next
```

g. Enable msg field, cht_send and cht_leave buttons.

```
document.getElementById("msg").disabled = false;
document.getElementById("cht_send").disabled = false;
document.getElementById("cht_leave").disabled = false;
```

5. Implement the `sendMessage` function in the `chat.js` script.
   a. Locate the `sendMessage` function in the `chat.js` script and prevent the default action.

```
function sendMessage(event) {
  event.preventDefault();
  // more implementation code will be placed here next
}
```

   b. Acquire cht_form and extract the value of the msg element from the document.

```
let cht_form = document.getElementById("cht");
let value = document.getElementById("msg").value;
// more implementation code will be placed here next
```

   c. Reset the form.

```
cht_form.reset();
// more implementation code will be placed here next
```

   d. Create a message object with a text property that should contain the value extracted from the msg input element.

```
let message = {text:value};
// more implementation code will be placed here next
```

   e. Convert message to JSON and send it to the socket.

```
socket.send(JSON.stringify(message));
```

6. Implement the `leaveChat` function in the `chat.js` script.
   a. Locate the `leaveChat` function in the `chat.js` script.
   b. Close the socket with anormal closure 1000 code.

```
function leaveChat(event) {
  socket.close(1000);
}
```

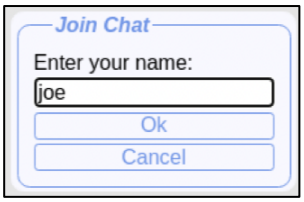
7. Implement the `showJoinDialog` function in the `chat.js` script.
   – Locate the `showJoinDialog` function in the `chat.js` script.
   – Retrieve `user` property from the session store.
   – If user is null, acquire join chat dialog from the page and display this dialog.
   – Otherwise, if the use is not null invoke `joinChat` function passing the user as an argument.

```
function showJoinDialog(event) {
  let user = store.getItem("user");
  if (user == null) {
    let join_chat_dialog = document.getElementById("join_chat_dialog");
    join_chat_dialog.showModal();
  }else{
    joinChat(user);
  }
}
```

8. Test DukeLabs chat.
   a. Make sure you are running the DukeLabs REST Service application in the node.js server. It should have been started as the first step of practice 8.1.
   b. Run the client application if it is not already running; otherwise, simply switch to the browser window used in the earlier tests of the application and refresh the page.
   c. Click the "Contact" link to navigate to the chat page.

   d. On the chat page click the "Join" button.

   e. Enter any name you like in the Join Chat dialog box.



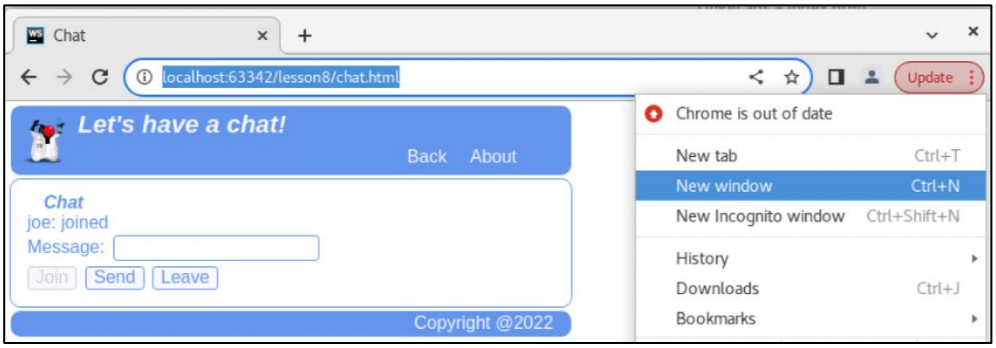
   f. Click Ok.

      Note: You should observe a message indicating that a user has joined the chat.

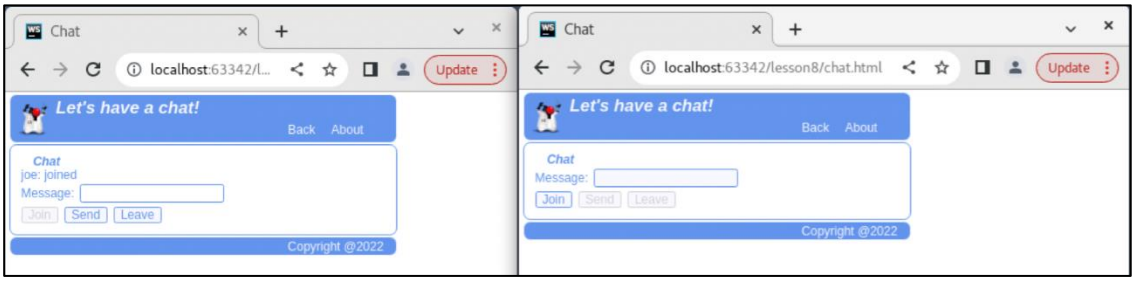
   g. Copy chat page URL from the address field of the browser.
   h. Open a new browser window, paste the previously copied URL into the address field in this new window, and press Enter to navigate to the chat page in this new window.

      Hints:

      - Click the three vertical dots to the side of the "Update" button.
      - Select "New Window" menu.

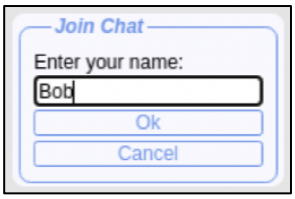


i. Align two browser windows side-by-side.

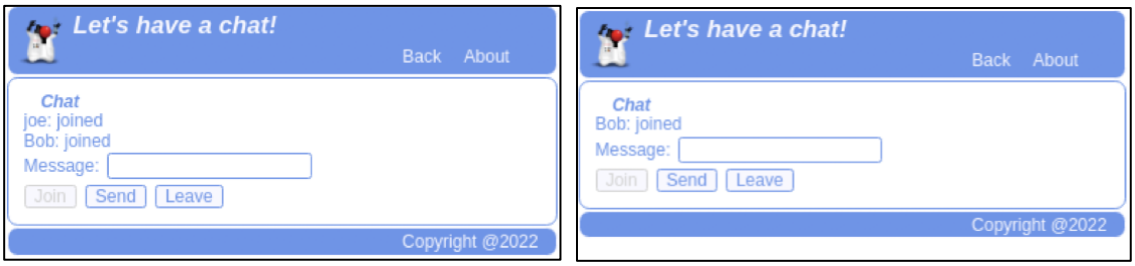


j. Click the "Join" button in the second window.

k. Enter a different name in the Join Chat dialog box.



l. Click Ok.



Notes:

- You should observe a message indicating that a user has joined the chat in both browser windows.
- Each browser window has its own independent socket and its own session storage object, which allows you to test more than one user accessing the chat at the same time.

m. Type a chat message into a chat field. You can use any of the chat windows.

n. Click the "Send" button.

Note: You should observe a message appear in both browser windows.

o. Click the "Leave" button in one of the chat windows.

Notes:

- You should observe an alert indicating that chat has been closed for the browser window in which you clicked the "Leave" button.
- Another browser window should display a message indicating that user has left the chat. your users.

p. Dismiss the alert and observe all messages disappear in the browser window, in which you have clicked the "Leave" button.

q. Click the "Join" button, to re-join the chat.

Notes:

- You should not be prompted to enter the name again, because it could be picked up from the session store. You would only be prompted to enter the name if you open another browser window.
- You should observe a message indicating that a user has joined the chat in both browser windows.

Note: In this practice, you have explored how to save and retrieve data from the session store. Security implementation, including user authentication, is outside of the scope of this course. Therefore, it is very important to note that the way this application handles a name of the user in the chat is not how you should implement security. User can enter any value as the name, and server just accepts this value, with no verification, such as a password or a digital signature validation.

Realistically security implementation must be configured on your server, which should:

- Implement secure connectivity using HTTPS rather than HTTP protocol
- Require users to authenticate first

WebSocket connections automatically inherit authentication information from the HTTP(s) connection. Browser will automatically retain and transmit relevant authentication headers to the server.

You should never put sensitive data, or authentication information into a local or session store.

Practice 8-3: Server-side Application Overview

Overview

There are no practical assignment tasks that you need to complete in this practice section. Its purpose is to give a brief overview of the server-side DukeLabs application that run in the node.js server and implement REST service as well as WebSocket functionalities.

This course focuses on a core JavaScript language feature, and covers client-side application design, leaving the server-side JavaScript programming outside of the scope. Nevertheless, it may be beneficial to understand the behaviors of the server-side part of the DukeLabs application.

DukeLabs Server-side Application Overview

1. Installation prerequisites
   a. A Node.js server and npm package manager must be installed first.
   b. DukeLabs server-side application is using several additional JavaScript packages, that must be installed as well.

```
npm i express
npm i body-parser
npm i cors
npm i websocket
npm i ws
```

2. Application source code with comments:

```
/*
Following section configures HTTP listener and sets up additional APIs that
help implement server-side REST service functionalities.
*/
// import http, body-parser and express APIs
// this enables the use of the additional libraries to implement
// http listener, register handler function for GET/POST/PUT/DELETE etc...
// http methods, parse JSON messages
const http = require('http');
const express = require('express');
const bp = require('body-parser');
// associate http server with express API
const app = express();
const server = http.createServer(express);
// configure listener host and port properties
const hostname = 'localhost';
const port = 8080;
// enable JSON body parser
app.use(bp.json());
```

```
// import cors library and described allowed request origins
const cors = require('cors');
app.use(cors({
    origin: '*'
}));

/*
Following section describes Experiment a Measurement handling
it is practically identical to the client-side code of the DukeLabs
application that you were creating throughout this course.
*/
// that constitutes the business logic of the DukeLabs application
const MILLI_PER_SECOND = 1000;
const SECONDS_PER_MINUTE = 60;
const SECONDS_PER_HOUR = 3600;
// Class describing Experiment data objects
class Experiment {
  #id;
  #task;
  #budget;
  #startTime;
  #endTime;
  #complete;
  constructor(id, task, budget, startTime, endTime, complete) {
    this.#id = id;
    this.#task = task;
    this.#budget = budget;
    this.#startTime = startTime;
    this.#endTime = endTime;
    this.#complete = complete;
  }
  static createCompleteExperiment(id, task, budget, startTime, endTime) {
    return new Experiment(id, task, budget, startTime, endTime, true);
  }
  static createOngoingExperiment(id, task, budget, startTime) {
    return new Experiment(id, task, budget, startTime, null, false);
  }
  toString() {
    return this.#id+" "+this.#task+" "+this.#budget+" "+
           this.#startTime+" "+this.#endTime+" "+this.#complete;
  }
  get id() { return this.#id; }
  get task() { return this.#task; }
  get budget() { return this.#budget; }
  get startTime() { return this.#startTime; }
  get endTime() { return (this.#complete) ? this.#endTime : "ongoing"; }
  get complete() { return this.#complete; }
  set complete(endTime) {
    this.#endTime = endTime;
    this.#complete = true;
  }
  getMeasurements() {
    return Array.from(data.getMeasurements(this.#id));
  }
```

```
  addMeasurement(unit, value) {
    let duration = formatDuration(new Date().getTime() -
                                  this.#startTime.getTime());
    let numValue = parseFloat(value);
    if (isNaN(numValue)) {
      throw "Measurement value "+value+" is not a number";
    }
    let measurement = new Measurement(unit, numValue, duration);
    data.addMeasurement(this.#id, measurement);
    return measurement;
  }
  toJSON() {
    return JSON.stringify({id:this.id, task:this.task, budget:this.budget,
                           startTime:this.startTime, endTime:this.endTime,
                           complete:this.complete});
  }
  static fromJSON(json) {
    let obj = JSON.parse(json);
    if (obj.complete) {
      return Experiment.createCompleteExperiment(obj.id, obj.task,
             new Date(obj.startTime), new Date(obj.endTime));
    }
    return Experiment.createOngoingExperiment(obj.id, obj.task,
          new Date(obj.startTime));
  }
}
// Class describing Measurement data objects
class Measurement {
  static #units = ["s","m","kg","A","K","mol","cd"];
  static #max_id = 0;
  #id;
  #unit;
  #value;
  #time;
  constructor(unit, value, time) {
    this.#id = ++Measurement.#max_id;
    this.#unit = (Measurement.#units.indexOf(unit) == -1) ? null : unit;
    this.#value = value;
    this.#time = time;
  }
  toString() {
    return this.#id+" "+this.#unit+" "+this.#value+" "+this.#time;
  }
  get id() { return this.#id; }
  get unit() { return this.#unit; }
  get value() { return this.#value; }
  get time() { return this.#time; }
  toJSON () {
    return JSON.stringify({id:this.id, unit:this.unit, value:this.value,
                           time:this.time});
  }
  static fromJSON(json){
    let obj = JSON.parse(json);
    return new Measurement (obj.unit, obj.value, obj.time);
  }
}
```

```
// An in-Memory "Database" with insert and search operations
const data = {
 // a Map comprised of a Set of Measurements indexed by an Experiment object
 allData : new Map(),
 // a Map comprised of Experiment objects indexed by ID
 experiments : new Map(),
 // a Map comprised of Measurement objects indexed by ID
 measurements : new Map(),
 addExperiment(experiment) {
   this.experiments.set(experiment.id, experiment);
   this.allData.set(experiment, new Set());
 },
 addMeasurement(eId, measurement) {
   this.measurements.set(measurement.id, measurement);
   this.allData.get(this.getExperiment(eId)).add(measurement);
 },
 findExperiment(eId) {
   let experiment = this.getExperiment(eId);
   return new Promise((resolved, rejected) => {
     if (experiment != undefined) {
       resolved(experiment);
     } else {
       rejected("Experiment with id "+eId+" not found");
    }});
  },
  getExperiment(eId) {
    return this.experiments.get(eId);
  },
  getMeasurement(mId) {
    return this.measurements.get(mId);
  },
  getMeasurements(eId) {
    return this.allData.get(this.getExperiment(eId));
  }
};
function formatDuration(duration) {
  if (duration === 0) {
    return "PT0S";
  }
  let totalSeconds = Math.trunc(duration/MILLI_PER_SECOND);
  let hours = Math.trunc(totalSeconds / SECONDS_PER_HOUR);
  let minutes = Math.trunc((totalSeconds % SECONDS_PER_HOUR)
                           / SECONDS_PER_MINUTE);
  let seconds = Math.trunc(totalSeconds % SECONDS_PER_MINUTE);
  let result = "PT";
  if (hours != 0) {
    result+=hours+"H";
  }
  if (minutes != 0) {
    result+=minutes+"M";
  }
  if (seconds != 0) {
    result+=seconds+"S";
  }
  return result;
}
```

```
// Populate Experiment and Measurement data
data.addExperiment(Experiment.createOngoingExperiment(101, "Measure Weight",
                123.45,new Date(2022,3,16,6,7)));
data.addExperiment(Experiment.createCompleteExperiment(102, "Measure Length",
                321.54, new Date(2022,4,1,14,30), new Date(2022,4,2,21,12)));
data.addMeasurement(101, new Measurement("kg",42,"PT2M12S"));
data.addMeasurement(101, new Measurement("kg",40,"PT3M10S"));
data.addMeasurement(101, new Measurement("kg",3,"PT3M55S"));
data.addMeasurement(102, new Measurement("m",12,"PT1H20M"));
data.addMeasurement(102, new Measurement("m",12,"PT1H22M10S"));

/*
Following section defines functions that handle REST calls for GET, POST and
PUT methods, and starts up the HTTP server.
*/
// HTTP GET retrieve experiment by ID
app.get('/experiment/:id', (req, res) => {
  let eId = parseInt(req.params.id);
  res.statusCode = 200;
  res.setHeader("Content-Type", "application/json");
  res.end(data.getExperiment(eId).toJSON());
});
// HTTP GET retrieve measurements by experiment ID
app.get('/experiment/:id/measurements', (req, res) => {
  let eId = parseInt(req.params.id);
  res.statusCode = 200;
  res.setHeader("Content-Type", "application/json");
  let measurements = data.getExperiment(eId).getMeasurements();
  for(i = 0; i < measurements.length; i++) {
    measurements[i] = measurements[i].toJSON();
  }
  res.end(JSON.stringify(measurements));
});
// HTTP GET retrieve a measurement by ID
app.get('/measurement/:id', (req, res) => {
  let mId = parseInt(req.params.id);
  res.statusCode = 200;
  res.setHeader("Content-Type", "application/json");
  res.end(data.getMeasurement(mId).toJSON());
});
// HTTP POST create new measurement
app.post('/experiment/:id/measurements', (req, res) => {
  let eId = parseInt(req.params.id);
  let experiment = data.getExperiment(eId);
  let json = req.body;
  let measurement = experiment.addMeasurement(json.unit, json.value);
  res.statusCode = 200;
  res.setHeader("Content-Type", "application/json");
  res.end(measurement.toJSON());
});
// HTTP PUT complete experiment with a given id
app.put('/experiment/:id/complete', (req, res) => {
  let eId = parseInt(req.params.id);
  let experiment = data.getExperiment(eId);
  let date = new Date();
  date.setMilliseconds(0);
  experiment.complete = date;
  res.statusCode = 200;
  res.setHeader("Content-Type", "application/json");
  res.end(JSON.stringify(experiment.endTime));
});
```

```
// Use express to start an HTTP server listener on a given host and port
let httpListener = app.listen(port, hostname,
  () => console.log(`Listening on port http://${hostname}:${port}/`));

/*
Following section defines functions that handle WebSocket communications.
*/
// import websocket API
const websocketAPI = require('ws');
// prepare websocket server
const socketServer = new websocketAPI.Server({
    server:httpListener,
    path:"/chat",
    clientTracking: true});
// Map of users indexed by  socket
let userMap = new Map();
// accept socket connections
socketServer.on("connection", (socket) => {
  // create an entry in the userMap that
  // corresponds to the currently opened socket
  // no proper authentication is configured for this nodejs server,
  // so username is undefined and is expected to be
  // picked up from the first message sent by the client
  // this is not a secure coding practice and should be considered only
  // as an example of handling different socket messages
  // authentication and authorisation setup for the nodejs
  // is outside the scope of this set of practices
  userMap.set(socket,undefined);
  // handle all incoming messages
  socket.on("message", (message) => {
    // parse message as JSON
    let messageObject = JSON.parse(message);
    // check if message contains a user property
    // assume its present only in the initial messages
    // save user into the session storage
    // assign property text a message that indicates
    // that user has joined the chat
    if (messageObject.user != undefined) {
      userMap.set(socket, messageObject.user);
      messageObject.text = "joined";
    }
    // transmit message to all sockets opened on this server
    broadcastMessage(socketServer, socket, messageObject.text);
  });
  socket.on("close", (code) => {
    // transmit message that indicates user has left the chat
    // to all sockets opened on this server
    broadcastMessage(socketServer, socket,  "left");
    userMap.delete(socket);
  });
});
```

```
// broadcast messages to all opened sockets
// each message is composed of a user and text properties
// username corresponds to the socket that has posted the message
// text is acquired from a method argument
function broadcastMessage(socketServer, socket, text) {
  let message = {
    user : userMap.get(socket),
    text : text
  }
  // iterate through the collections of clients
  // that represent currently opened sockets of this server
  // convert message object to JSON and send it to each client
  socketServer.clients.forEach((client) => {
    client.send(JSON.stringify(message));
  });
}
```